

Great resource on final exam practice set of problems per module to review after going through all notes for each module:

<https://mcit.notion.site/CIS-5210-Question-Bank-cf290053147a4662834f4ca47ea654d3>

## 16.1 Textbook readings

### 1. Decision Making Under Uncertainty

Decision theory begins with an agent that must choose among actions when the world is uncertain.

There are two main kinds of uncertainty:

- **Uncertainty about the current state of the world:** the agent assigns probabilities  $P(s)$  to possible current states  $s$ .
- **Uncertainty about action outcomes:** even if the current state were known, performing an action  $a$  may lead to different next states  $s'$  with probabilities  $P(s' | s, a)$ .

Thus, the overall probability that action  $a$  leads to outcome state  $s'$  is:

$$P(\text{RESULT}(a)=s') = \sum_s P(s)P(s' | s, a)P(\text{RESULT}(a)=s') = \sum_s P(s)P(s' | s, a)$$

This combines:

- the agent's **beliefs** about where it currently is, and
- the world's **causal dynamics** about what the action will do.

So decision making under uncertainty requires combining **beliefs** (probabilities) and **desires** (utilities).

## 2. Utility Function

To represent preferences over outcomes, decision theory uses a **utility function**  $U(s)$ .

A utility function assigns a real number to a state  $s$  to express how desirable that state is to the agent.

Important idea:

- utility is not the same as probability,
- utility measures **preference/desirability**,
- probability measures **belief/uncertainty**.

## 3. Expected Utility

The **expected utility** of an action is the average utility of its possible outcomes, weighted by their probabilities:

$$EU(a) = \sum_s P(\text{RESULT}(a)=s)U(s)$$

$$EU(a) = \sum_s P(\text{RESULT}(a)=s)U(s)$$

This means:

- list all possible outcomes of action  $a$ ,
- evaluate how good each outcome is,

- weight each by how likely it is,
- sum them up.

Expected utility is therefore the standard way to evaluate actions under uncertainty.

#### 4. Principle of Maximum Expected Utility (MEU)

The central principle of decision theory is the **principle of maximum expected utility**:

This says:

A rational agent should choose the action with the highest expected utility.

This is the formal statement of “do the best thing,” where “best” is interpreted in terms of utility and uncertainty.

#### 5. Why MEU Does Not Solve All of AI

The MEU principle gives a mathematical ideal of rational choice, but it does **not** by itself solve the AI problem.

To apply MEU, the agent still needs to:

- infer the probability distribution over current states  $P(s)$ ,
- know or learn the transition model  $P(s' | s, a)$ ,
- represent actions and outcomes,
- compute utilities of outcomes,

- search or plan when the value of a state depends on future possibilities,
- possibly infer a human user's utility function if acting on behalf of a person.

So decision theory provides a **framework**, not a complete implementation recipe.

## 6. Relationship Between Utility and Performance Measures

MEU connects to the idea of **performance measures** from intelligent agents.

- A **performance measure** scores complete histories of interaction.
- A **utility function** gives values to immediate outcomes or states.

This is a crucial shift:

- performance measures are usually defined **externally** and retrospectively,
- utility functions are used **internally** to guide action selection step by step.

If the utility function correctly reflects the performance measure, then maximizing expected utility leads to the highest expected performance.

So utility functions are the internal mechanism by which rational agents pursue externally defined goals.

## 16.2 The Basis of Utility Theory — Main Point Summary

### 7. Why Utility Theory Needs Justification

At first glance, MEU seems reasonable, but it is not obvious that it is the only rational method for decision making.

Possible questions include:

- Why maximize average utility rather than some other function?



- Why not minimize the worst-case outcome?
- Why not just rank preferences without numbers?
- Why should a utility function even exist?

Utility theory answers these by starting from **constraints on rational preferences** and proving that such preferences imply the existence of a utility function and the validity of MEU.

### 16.2.1 Constraints on Rational Preferences

## 8. Preferences Between Lotteries

Decision theory often deals not with certain outcomes, but with **lotteries**.

A **lottery** is a probabilistic combination of outcomes:

$$L = [p_1, S_1; p_2, S_2; \dots; p_n, S_n] \quad L = [p_1, S_1; p_2, S_2; \dots; p_n, S_n]$$

where:

- $S_i$  are outcomes,
- $p_i$  are the probabilities of those outcomes.

An outcome can itself be:

- a basic state of the world, or
- another lottery.

Thus utility theory studies preferences over uncertain prospects, not just certain states.

## 9. Preference Notation

The book uses the following notation:

- $A \succ B$ : the agent prefers  $A$  over  $B$
- $A \sim B$ : the agent is indifferent between  $A$  and  $B$

- $A \succeq B \iff B \succeq A$ : the agent prefers A to B or is indifferent

The goal is to identify what rational preferences must look like.

## 10. Axioms of Utility Theory

A rational preference relation should satisfy six axioms.

### A. Orderability

For any two lotteries, the agent must be able to compare them.

Exactly one of the following must hold:

- $A \succ B \iff B \not\succ A$
- $A \sim B \iff B \sim A$

So a rational agent cannot refuse to make a comparison.

### B. Transitivity

If:

- $A \succ B \iff B \not\succ A$
- $B \succ C \iff C \not\succ B$

then:

- $A \succ C \iff C \not\succ A$

This prevents cyclic preferences.

Without transitivity, an agent could be exploited by repeated trades. For example, if it prefers  $A \succ B \succ C \succ A$ , someone could repeatedly trade with it in a loop and extract a fee each time until the agent loses all its money.

So transitivity is necessary for coherent choice.

### C. Continuity

If:

- $A \succ B \succ C$

then there must exist some probability  $p$  such that the agent is indifferent between:

- receiving  $B$  for sure, and
- a lottery that gives  $A$  with probability  $p$  and  $C$  with probability  $1-p$

This means intermediate preferences can be represented by some probabilistic mix of better and worse outcomes.

Continuity rules out lexicographic preferences where one consideration is infinitely more important than another.

### D. Substitutability

If the agent is indifferent between  $A$  and  $B$ , then one can be substituted for the other inside a more complex lottery without changing preferences.

So if:

- $A \sim B$

then:

- $[p,A;1-p,C] \sim [p,B;1-p,C]$

This ensures that only the utilities of outcomes matter, not irrelevant descriptions.

The same idea also applies when one option is strictly preferred to the other.

## E. Monotonicity

If:

- $A \succ B$

then among lotteries over AAA and BBB, the agent should prefer the lottery with the **higher probability of AAA**.

So if  $p > q$ , then:

$$[p,A;1-p,B] \succ [q,A;1-q,B]$$

This captures the idea that more chance of a better outcome is better.

## F. Decomposability

Compound lotteries can be reduced using standard probability rules.

For example, a lottery whose outcome is itself another lottery should be equivalent to a flattened single-stage lottery with combined probabilities.

This is sometimes called the “**no fun in gambling**” rule: the structure of the gamble itself has no intrinsic value unless such value is encoded into the outcomes.

This lets us simplify nested lotteries into ordinary ones.

## 11. Why These Axioms Matter

These axioms are not about numbers yet; they are about how rational preferences should behave.

Their importance is that they are strong enough to derive the existence of utility functions and the MEU principle.

### 16.2.2 Rational Preferences Lead to Utility

#### 12. Existence of a Utility Function

A central theorem says:

If an agent's preferences satisfy the utility axioms, then there exists a utility function  $U$  such that:

- $U(A) > U(B)$  iff  $A \succ B$
- $U(A) = U(B)$  iff  $A \sim B$

So a consistent rational preference ordering can always be represented numerically.

This is a foundational result:

Utility is not just an arbitrary assumption; it can be derived from rational preference structure.

#### 13. Utility of a Lottery = Expected Utility

Another theorem says that the utility of a lottery is the expected utility of its outcomes:

This means expected utility is not merely a convenient formula; it is forced by the preference axioms.

Thus:

- if an agent is rational in the sense of satisfying the axioms,
- then it must act as if it maximizes expected utility.

#### 14. Uniqueness Up to Positive Affine Transformation

Utility functions are **not unique**.

If  $U$  represents preferences, then so does:

$$U'(S) = aU(S) + b$$

where:

- $a > 0$
- $b$  is any constant

This is called a **positive affine transformation**.

So utility scales are meaningful only up to:

- shifting by a constant,
- multiplying by a positive constant.

This means there is **no absolute utility scale**.

#### 15. Ordinal vs. Utility-Based Ranking

In deterministic environments, only the ranking of states matters, not the exact numbers. This is sometimes called an **ordinal utility function** or **value function**.

But under uncertainty, the actual numeric differences matter, because expected utility calculations depend on them.

So:

- **certainty** only needs order,
- **uncertainty** requires cardinal structure up to positive affine transformations.

## 16. Utility as an Explanatory Tool

The existence of a utility function does **not** mean an agent explicitly computes it.

A rational system could be implemented in many ways, including simple lookup tables.

Still, by observing an agent's choices, an outside observer can infer a utility function that explains what the agent is effectively optimizing.

So utility theory can be used both:

- normatively, to prescribe rational action,
- descriptively, to model observed choice behavior.

### 16.3 Utility Functions — Main Point Summary

## 17. Utility Functions Can Reflect Any Consistent Preferences

In general, utility theory allows any set of preferences consistent with the axioms.

An agent may have unusual tastes and still be rational, such as:

- preferring prime numbers of dollars,
- preferring a dented old car to a new luxury car.

Rationality does **not** require sensible-seeming goals.

It requires only that preferences be consistent in the appropriate way.

So utility theory is very general.

### 16.3.1 Utility Assessment and Utility Scales

## 18. Preference Elicitation

If we want an AI system to act for a human, we need to determine the human's utility function.

This process is called **preference elicitation**.

It works by:

- presenting choices,
- observing the human's preferences,
- inferring the underlying utility numbers consistent with those choices.

## 19. Utility Scales

Since utility is only defined up to a positive affine transformation, we often choose a convenient scale by fixing two reference outcomes:

- best possible prize:  $u_{\top}$
- worst possible catastrophe:  $u_{\bot}$

A common choice is **normalized utilities**:

This gives a standardized range for comparisons.



## 20. Standard Lottery Method

To assess the utility of some outcome SSS, compare it with a **standard lottery**:

$$[p, u^\top; (1-p), u_\perp] [p, u_{\text{top}}; (1-p), u_{\text{bot}}] [p, u^\top; (1-p), u_\perp]$$

Adjust ppp until the person is indifferent between:

- receiving SSS for sure, and
- the standard lottery.

Then, on a normalized scale, the utility of SSS is ppp.

This is a standard technique for measuring utilities empirically.

## 21. Utility Assessment in Life-and-Death Decisions

Many important real decisions involve lives, health, and safety.

Examples:

- transportation regulation
- medical treatment
- pollution policy
- engineering safety

The text emphasizes that refusing to put a value on life does not avoid tradeoffs; it usually means tradeoffs are made implicitly and often poorly.

Agencies often use the **value of a statistical life (VSL)** to quantify the benefit of safety interventions. The passage notes that U.S. agencies have used values around **\$10 million** (as of 2019).

## 22. Micromort

A **micromort** is a one-in-a-million chance of death.

It is used as a unit for small mortality risks.

By examining how much people are willing to pay to reduce tiny risks, analysts estimate implied valuations of risk and life.

The text notes that observed behavior often values a micromort around tens of dollars.

## 23. QALY

A **QALY** is a **quality-adjusted life year**.

It measures health outcomes by trading off:

- length of life,
- quality of life.

For example, a person may be indifferent between:

- living longer with serious disability,
- living a shorter period in full health.

QALYs are widely used in health economics and medical decision analysis.

## 16.3.2 The Utility of Money

### 24. Money Is Not the Same as Utility

Because money can buy many things, it seems like a natural candidate for utility. But money itself is **not** generally a utility function.

An agent may usually prefer more money to less, which is called a **monotonic preference** for money, but this does not tell us how the agent values uncertain monetary lotteries.

So:

- “more money is better” is not enough,
- we need to know the shape of the utility function over wealth.

## 25. Expected Monetary Value (EMV) vs. Expected Utility

People often reject gambles with higher expected monetary value because utility of money is not linear.

Example:

- sure \$1,000,000
- versus 50% chance at \$0 and 50% chance at \$2,500,000

The gamble has higher **EMV**:

$$0.5(0) + 0.5(2,500,000) = 1,250,000$$
$$0.5(0) + 0.5(2,500,000) = 1,250,000$$

But a rational person may still decline if the sure million has higher utility.

So the correct decision criterion is:

- maximize **expected utility**, not expected money.

## 26. Diminishing Marginal Utility of Wealth

For many people, the utility of money is **concave** over positive wealth:

- the first million matters more than an additional million,
- extra wealth yields smaller and smaller increases in utility.

This is called **diminishing marginal utility**.

The text mentions empirical work suggesting utility may be roughly proportional to the **logarithm of wealth** over certain ranges.

## 27. S-Shaped Utility Curve

The text suggests that overall utility for money may be:

- **concave** in positive wealth,
- but may become **convex** in severe debt or desperate situations.

This creates an **S-shaped** curve:

- risk-averse for gains,
- risk-seeking in extreme loss regions.

So attitudes toward risk may depend on current wealth level.

## 28. Risk Aversion

If utility is concave over a range, then the utility of a lottery is less than the utility of receiving the lottery's expected monetary value for sure:

$$U(L) < U(\text{SEMV}(L))$$

Such an agent is **risk-averse**.

Meaning:

- the agent prefers a sure thing over a gamble with the same EMV.

This explains why people buy insurance.

## 29. Risk Seeking

In some negative-wealth or desperate regions, an agent may become **risk-seeking**:

- willing to accept gambles with unfavorable downside because the current situation is already very bad.

This is still rational if it follows from the utility function.

## 30. Certainty Equivalent

The **certainty equivalent** of a lottery is the guaranteed amount of money the agent would accept instead of taking the gamble.

For example, a person might prefer \$400 for sure over a 50/50 gamble between \$0 and \$1000.

Then:

- certainty equivalent = \$400
- $EMV = \$500$

This difference shows risk aversion.

## 31. Insurance Premium

The difference between:

- the expected monetary value of a lottery,
- and its certainty equivalent

is called the **insurance premium**.

This is the amount a risk-averse person is willing to give up to avoid uncertainty.

Insurance exists because individuals are risk-averse while large insurers are approximately risk-neutral over relatively small individual policies.

### 32. Risk Neutrality

An agent with a **linear** utility function over wealth is **risk-neutral**.

For risk-neutral agents:

- utility is proportional to money,
- maximizing expected utility is the same as maximizing expected monetary value.

For small gambles relative to total wealth, many agents are approximately risk-neutral because utility is locally close to linear.

### 16.3.3 Expected Utility and Post-Decision Disappointment

### 33. Why Choosing the Highest Estimated Utility Can Still Disappoint

Even if expected utility is the correct decision rule, in practice we often use **estimates** of expected utility rather than exact values.

Suppose those estimates are:

- noisy,
- but **unbiased** on average.

Even then, if we pick the option with the highest estimated utility, we are likely to be disappointed.

Why?

Because among many noisy estimates, the highest one is likely to be one that is too optimistic.

### 34. The Optimizer's Curse

This systematic overestimation is called the **optimizer's curse**.

If you evaluate many options and choose the one with the highest estimate, that estimate is probably biased upward simply because:

- some estimates are too low,
- some are too high,
- and selecting the maximum tends to pick an overestimate.

As the number of options  $k$  increases, the expected optimism bias grows.

So even unbiased estimation procedures can lead to biased choice outcomes after optimization.

### 35. Examples of the Optimizer's Curse

The text gives several examples:

- a drug that looks best among thousands of candidates may not be as good as it appears,
- a mutual fund chosen for advertising because of strong past performance is likely flattered by noise,

- a treatment with a small sample and dramatic success may actually be worse than a larger-sample treatment with slightly lower apparent success.

This is a very important real-world lesson:

optimization over uncertain estimates causes systematic overoptimism.

### 36. Bayesian Correction

A way to correct the optimizer's curse is to use a **Bayesian approach**.

Instead of accepting the estimate at face value, treat it as evidence about the true utility and combine it with:

- a prior distribution over likely true utilities,
- a model of estimation error  $P(\hat{EU} | EU)P(\hat{EU} \mid EU)P(EU | \hat{EU})$

Using Bayes' rule, compute the posterior for the true utility.

This pulls extreme estimates back toward more realistic values.

### 16.3.4 Human Judgment and Irrationality

### 37. Normative vs. Descriptive Theory

Decision theory is a **normative theory**:

- it says how rational agents **should** act.

A **descriptive theory** says how real humans **actually do** act.

These are not always the same.



The text notes that humans often appear to violate the predictions of utility theory in systematic ways.

### **38. Allais Paradox**

In the **Allais paradox**, people typically make choices that cannot be represented by any single expected utility function.

They prefer:

- a sure moderate reward over a risky higher-EMV option,
- but then in another related choice prefer the riskier option.

These paired choices are inconsistent with MEU.

This suggests that actual human judgments are influenced by factors outside standard utility theory.

### **39. Certainty Effect**

One explanation is the **certainty effect**:

people disproportionately favor outcomes that are certain.

Even when expected utility analysis would not justify it, a sure gain often feels much more attractive than a nearly sure gain.

Possible reasons:

- reduced mental effort,
- distrust of stated probabilities,
- anticipated regret if a gamble fails.

### **41. Ellsberg Paradox**

In the **Ellsberg paradox**, people prefer known probabilities over ambiguous ones.

This is called **ambiguity aversion**.

Even when standard expected utility theory cannot justify these preferences under any single probability model, humans tend to prefer gambles with known chances over equivalent gambles with uncertain probabilities.

So people dislike not just risk, but also ambiguity.

## **42. Framing Effect**

The **framing effect** occurs when logically equivalent descriptions lead to different choices.

Example:

- “90% survival rate”
- versus “10% death rate”

These mean the same thing, but people often react differently.

This shows that human decision making is sensitive to presentation, not just substance.

## **43. Anchoring Effect**

The **anchoring effect** occurs when an irrelevant reference point influences judgment.

Example:

- an overpriced wine bottle makes a merely expensive bottle seem reasonable by comparison.

People often make relative judgments rather than absolute ones, and anchors can shift those judgments.

#### **44. Human Preferences Are Often Revisable**

The text notes that human inconsistency does not necessarily mean permanent irrationality.

When choices are explained more clearly, many people revise their preferences and become more consistent.

So paradoxes in judgment may partly arise from:

- misunderstanding,
- poor presentation,
- difficulty thinking in abstract numerical forms.

#### **45. Evolutionary Psychology Perspective**

Researchers in **evolutionary psychology** argue that human brains did not evolve to solve word problems stated in formal numerical language.

Instead, human decision mechanisms may work better with:

- concrete experience,
- frequencies,
- visual or intuitive representations.

When problems are presented in more evolutionarily natural formats, human behavior often comes closer to rational standards.

So some apparent irrationality may reflect mismatch between:

- the way the problem is presented,
- and the way human cognition evolved.

#### **Big-Picture Takeaways**

## 46. Central Core of Decision Theory

Decision theory combines:

- **beliefs** about uncertain states and outcomes,
- with **desires** represented by utilities.

A rational agent should choose the action that maximizes **expected utility**.

## 47. Utility Theory Justifies MEU

The MEU principle is not arbitrary. It follows from axioms about rational preferences:

- orderability
- transitivity
- continuity
- substitutability
- monotonicity
- decomposability

These imply that rational preferences can be represented by a utility function and that lotteries must be evaluated by expected utility.

## 48. Utility Is Not the Same as Money

Money is often related to utility, but utility of wealth is generally nonlinear. This explains:

- risk aversion,
- risk seeking in desperate regions,
- certainty equivalents,
- insurance premiums.

## **49. Optimization Under Uncertainty Can Mislead**

Even if estimates are unbiased, choosing the option with the highest estimated utility tends to produce disappointment because of the **optimizer's curse**.

So rational decision making must often account for uncertainty in the estimates themselves.

## **50. Humans Often Deviate from Normative Rationality**

Human judgment is influenced by:

- certainty effect
- ambiguity aversion
- framing
- anchoring
- regret
- presentation format

Thus descriptive human choice often differs from normative expected utility theory.

**Textbook readings 17.1 - 17.2:**

### **17.1 Sequential Decision Problems**

#### **1. Sequential Decision Problems**

Sequential decision problems arise when an agent must make **a sequence of decisions over time** in an uncertain environment.

Example:

A **4×3 grid world** where an agent starts in a specific square and must reach a terminal state:

- Goal states:
  - **+1 reward**
  - **-1 reward**
  
- Nonterminal transitions:
  - **-0.04 reward**

Actions available in each state:

- Up
  
- Down
  
- Left
  
- Right

The agent repeatedly:

1. Observes its current state
2. Chooses an action
3. Moves to a new state
4. Receives a reward

The process continues until a **terminal state** is reached.

## 2. Stochastic Actions

Actions in the environment are **not deterministic**.

Example transition model:

- Intended direction occurs with probability **0.8**
- Side directions occur with probability **0.1 each**

Example:

From state (1,1), action **Up** results in:

- Move to (1,2) with probability **0.8**
- Move to (2,1) with probability **0.1**
- Hit wall and stay in (1,1) with probability **0.1**

Thus, actions have **uncertain outcomes**.

## 3. Transition Model

The transition model specifies the probability of moving between states.

Notation:

$$P(s' | s, a)$$

Meaning:

Probability that performing action **a** in state **s** leads to state **s'**.

Assumption: **Markov property**

The next state depends only on:

- the current state
- the action taken

**It does not depend on previous history.**

#### **4. Reward Function**

Each transition produces a reward:

$R(s,a,s')$

Example in grid world:

- Transition to +1 state  $\rightarrow$  reward **+1**
- Transition to -1 state  $\rightarrow$  reward **-1**
- All other transitions  $\rightarrow$  reward **-0.04**

This negative step reward encourages the agent to reach the goal **quickly**.

#### **5. Markov Decision Process (MDP)**

A **Markov Decision Process (MDP)** models sequential decision problems under uncertainty.

An MDP consists of:

1. **States (S)**
2. **Initial state ( $s_0$ )**
3. **Actions available in each state  $A(s)$**



4. **Transition model  $P(s' | s, a)$**

5. **Reward function  $R(s, a, s')$**

MDPs are widely used in:

- AI
- Robotics
- Economics
- Control theory
- Operations research

## 6. Dynamic Programming

MDPs are usually solved using **dynamic programming**, which:

- Breaks a problem into smaller subproblems
- Solves them recursively
- Stores optimal solutions

This principle is essential for algorithms like:

- Value iteration
- Policy iteration

## Policies

### 7. Policy Definition

A **policy** specifies what action the agent should take in each state.

Notation:

$$\pi(s) \backslash \pi(s) \pi(s)$$

Meaning:

The action chosen in state  $s$ .

A policy is a **complete solution** to the sequential decision problem because it specifies behavior for **all possible states**.

## 8. Optimal Policy

Because outcomes are stochastic, executing a policy produces **different possible histories**.

Thus, policies are evaluated using **expected utility**.

An **optimal policy** is the policy that maximizes expected utility.

Notation:

$$\pi^* \backslash \pi^* \pi^*$$

Multiple optimal policies may exist.

Example:

In the grid world, the agent might choose:

- **safer but longer path**
- **riskier but shorter path**

Both can be optimal depending on parameters.

## 9. Effect of Step Reward

The step reward  $r$  affects the agent's behavior.

Examples:

If **r** is very negative:

- Life is painful
- Agent exits immediately
- Even chooses  $-1$  exit if closer

If **r** moderately negative:

- Agent heads toward  $+1$  but avoids risk

If **r** slightly negative:

- Agent becomes very cautious
- Avoids  $-1$  completely

If **r** positive:

- Agent prefers staying alive forever
- Avoids terminal states
- Infinite reward possible

### 17.1.1 Utilities Over Time

#### 10. Utility of Histories

The utility of an environment history is written as:

$$U_h([s_0, a_0, s_1, a_1, \dots, s_n])$$

This measures the total desirability of a sequence of states and actions.

#### 11. Finite vs Infinite Horizon

## Finite Horizon

There is a fixed deadline **N** or **time limit** to reach the terminal states

After time **N**, outcomes no longer matter.

Policies may depend on time remaining.

These are called **nonstationary policies**.

Example:

If only **3 steps remain**, the agent must take the shortest route.

## Infinite Horizon

There is **no fixed deadline** or no time limit to reach one of the terminal states

Optimal behavior depends **only on the current state**.

Policies are therefore **stationary**.

Stationary policies are simpler and commonly used

## 12. Additive Discounted Rewards

Utility is defined as

Where:

- $\gamma$  = discount factor
- $0 \leq \gamma \leq 1$

The discount factor reflects preference for **earlier rewards**.

### 13. Discount Factor Meaning

If:

$\gamma$  close to **0**

- Only immediate rewards matter

If:

$\gamma$  close to **1**

- Long-term rewards matter more

Economic interpretation:

$$\text{interest rate} = \frac{1}{\gamma} - 1$$

Example:

$$\gamma = 0.9 \rightarrow \text{interest rate} \approx 11.1\%$$

### 14. Reasons for Discounting

Discounting is justified by:

#### 1. Psychology

- Humans prefer near rewards

#### 2. Economics

- Money today can be invested

#### 3. Uncertainty

- Future rewards may never occur

#### 4. **Mathematical convenience**

Prevents infinite utility sums

### 15. **Infinite Reward Problems**

Without discounting, infinite-horizon problems may produce infinite utilities.

Solutions include:

1. **Discounted rewards**
2. **Terminal states with proper policies**

**Average reward per step**

Discounted rewards are usually the easiest.

#### 17.1.2 **Optimal Policies and Utilities of States**

### 16. **Utility of a State**

The utility of a state is the expected sum of discounted rewards when following the optimal policy.

$$U(s) = U^{\pi^*}(s)$$

States closer to positive rewards have **higher utilities**.

### 17. **Maximum Expected Utility Action**

Optimal action selection follows:

$$\pi^*(s) = \arg\max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma U(s')] \\ \pi^*(s) = \arg\max_{s'} \sum_{a} P(s'|s,a) [R(s,a,s') + \gamma U(s')]$$

This is the **principle of maximum expected utility**.

## 18. Bellman Equation

The Bellman equation defines state utilities recursively:

$$U(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma U(s')] \\ U(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma U(s')]$$

Meaning:

Utility of a state equals:

- immediate reward
- plus discounted utility of next state

assuming optimal action.

## 19. Q-Function

The **action utility function**:

$$Q(s,a)$$

represents the expected utility of performing action **a** in state **s**.

Relationship:

$$U(s) = \max_a Q(s,a) \\ U(s) = \max_a Q(s,a)$$

Optimal policy:

$$\pi^*(s) = \arg\max_a Q(s,a) \\ \pi^*(s) = \arg\max_a Q(s,a)$$

### 17.1.3 Reward Scales

## 20. Affine Transformations

Utilities can be transformed without affecting optimal decisions:

$$U'(s) = mU(s) + b \quad U'(s) = mU(s) + b$$

Where:

- $m > 0, b > 0$

## 21. Reward Shaping

Reward functions can also be transformed using a **potential function**  $\Phi(s)$ :

$$R'(s, a, s') = R(s, a, s') + \gamma \Phi(s') - \Phi(s) \quad R'(s, a, s') = R(s, a, s') + \gamma \Phi(s') -$$

$$\Phi(s) \quad R'(s, a, s') = R(s, a, s') + \gamma \Phi(s') - \Phi(s)$$

This transformation **does not change the optimal policy**.

This is known as the **shaping theorem**.

## 22. Purpose of Reward Shaping

Reward shaping helps guide learning by providing intermediate rewards.

Example:

Animal training:

- small rewards reinforce intermediate steps toward the final goal.

### 17.1.4 Representing MDPs

## 23. Tabular Representation

The simplest representation uses tables:



- Transition table:  $P(s' | s, a)P(s' | s, a)P(s' | s, a)$
- Reward table:  $R(s, a, s')R(s, a, s')R(s, a, s')$

Size:

$$|S|^2 |A| |S|^2 |A| |S|^2 |A|$$

This becomes infeasible for large problems.

## 24. Factored Representation

Large MDPs are represented using **Dynamic Decision Networks (DDNs)**.

DDNs extend **Dynamic Bayesian Networks** with:

- action nodes
- reward nodes
- utility nodes

Advantages:

- exponential reduction in complexity
- realistic modeling of large systems

## 25. Example: Robot MDP

State variables include:

- location
- velocity
- battery level

- charging status

Actions include:

- wheel motor power
- plug/unplug charging

The full state space is the **Cartesian product of these variables**.

## 26. Example: Tetris MDP

Tetris can be modeled as an MDP.

State variables:

- board configuration
- current piece
- next piece

State space size:

$$7 \times 7 \times 2200 \approx 10627 \times 7 \times 2^{200} \approx 10^{62} \times 7 \times 7 \times 2200 \approx 1062$$

This is far too large for exact algorithms.

## 17.2 Algorithms for MDPs

Four main methods:

1. **Value iteration**
2. **Policy iteration**
3. **Linear programming**
4. **Online algorithms**

### 17.2.1 Value Iteration

Value iteration solves the Bellman equations iteratively.

Update rule:

Each iteration updates utilities using neighboring states.

Eventually the utilities converge to the optimal values.

### 27. Bellman Update

Each update is called a **Bellman update**.

Utilities propagate gradually across the state space.

### 28. Convergence

Value iteration converges because the Bellman update is a **contraction mapping**.

A contraction brings values closer together each iteration.

Error decreases by factor  $\gamma$  each iteration.

Thus convergence is **exponentially fast**.

## 29. Error Bounds

If the update difference is small:

$$||U_{i+1} - U_i|| < \epsilon(1-\gamma)/\gamma \implies ||U_{i+1} - U_i|| < \epsilon(1-\gamma)/\gamma$$

then the utility estimate is within  $\epsilon$  of the true utility.

## 30. Policy Loss

Even if utility estimates are inaccurate, the resulting policy may already be optimal.

Policy loss measures how much utility is lost by using the approximate policy instead of the optimal one.

### 17.2.2 Policy Iteration

Policy iteration alternates two steps:

#### Policy Evaluation

Compute utilities assuming the current policy is followed.

#### Policy Improvement

Update the policy using maximum expected utility.

This continues until the policy stops changing.

## 31. Policy Evaluation Equation

For a fixed policy:

$$U(s) = \sum_{s'} P(s' | s, \pi(s)) [R(s, \pi(s), s') + \gamma U(s')] \\ U(s) = \sum_{s'} P(s' | s, \pi(s)) [R(s, \pi(s), s') + \gamma U(s')] \\ U(s) = \sum_{s'} P(s' | s, \pi(s)) [R(s, \pi(s), s') + \gamma U(s')]$$

These equations are **linear**, so they can be solved efficiently.

## 32. Modified Policy Iteration

Instead of exact evaluation, approximate evaluation is used.

This greatly reduces computational cost.

## 33. Asynchronous Policy Iteration

Utilities do not need to be updated for all states simultaneously.

Updating subsets of states can still converge to the optimal policy.

Policy iteration and value iteration are two fundamental [Dynamic Programming](#) algorithms used to solve [Markov Decision Processes \(MDPs\)](#) by finding an optimal policy. While both are guaranteed to converge to the same optimal solution, they differ in their internal mechanics and efficiency. [GeeksforGeeks +4](#)

### Core Differences

| Feature  | Policy Iteration                                                             | Value Iteration                                                                      |
|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>Approach</b>                                                                             | Alternates between <b>Policy Evaluation</b> and <b>Policy Improvement</b> .  | Directly updates the <b>Value Function</b> until it reaches a fixed point.           |
| <b>Iteration Loop</b>                                                                       | Two nested loops: evaluation is an iterative sub-process for a fixed policy. | A single loop: updates values and improvement happen simultaneously.                 |
| <b>Convergence</b>                                                                          | Converges when the policy stops changing (usually in few iterations).        | Converges when the value function stabilizes within a tiny threshold ( $\epsilon$ ). |
| <b>Computation</b>                                                                          | High cost per iteration but typically requires fewer total iterations.       | Lower cost per iteration but often requires many more total iterations.              |
| <b>Policy Output</b>                                                                        | An explicit policy is maintained and updated at every step.                  | The policy is extracted only after the value function has converged.                 |

### Algorithm Mechanics

- **Policy Iteration:** Starts with a random policy and follows two steps:
  1. **Evaluation:** Calculates the exact value function  $V^\pi(s)$  for the current policy by solving a system of linear equations or using iterative updates.
  2. **Improvement:** Updates the policy by choosing the best action (greedy) based on those calculated values.

2. **Improvement:** Updates the policy by choosing the best action (greedy) based on those calculated values.

- **Value Iteration:** Combines the two steps. In each sweep across all states, it applies the Bellman Optimality Equation to update each state's value to the maximum possible expected reward of all available actions.  Medium +4

## When to Use Each

- **Use Policy Iteration** for problems with **small state spaces** where you can afford the linear system solving in the evaluation step, or when you have a good initial policy that can lead to rapid convergence.
- **Use Value Iteration** for **large state spaces** where evaluating a full policy at each step is too computationally expensive. It is often faster in practice because it doesn't wait for exact policy evaluation before moving on.  GeeksforGeeks +4

\* **Value iteration** general complexity of  $O(S^2A)$  per iteration.

**Policy iteration** general complexity:  $O(S^3 * S^2A)$  which is a lot more per iteration than value iteration so slower per iteration. But requires less iterations!\*

**Policy Iteration practice problem to look over carefully!!!**

Explanation:

a: (true) the living reward is more of a game-based feature that either rewards or punishes an agent for remaining alive in the game. A living reward under 1.0 encourages an agent to leave the game, while a living reward over 1.0 incentivizes the agent to remain in the game

b: (true) the discount rate  $\gamma$  is similar to the tension between interest rates, inflation, and returns on investment. The a  $\gamma < 1.0$  on a future reward makes it less valuable to the agent (i.e. inflation/interest), while  $> 1.0$  causes future rewards to be more valuable, similar to returns earned on an investment.

c: (true) The noise  $\delta$  is what introduces stochasticity and risk into the action-taking process. The higher the noise, the lower the probability that an agent will successfully complete the directed action and the higher the probability of an undesirable outcome. With  $\delta = 0.0$  leads to deterministic outcomes, and encourages the agent to take risks because the probability of a bad outcome is reduced. With  $\delta = 0.5$ , the agent has a 50-50 chance of successfully completing the directed action, making its life a coin-flip, and thus discouraging risk-taking

d: (false) From the textbook, "A discount factor of  $\gamma$  is equivalent to an interest rate of  $(\frac{1}{\gamma}) - 1$ ." This means that  $\gamma = 0.9$  is equivalent to an interest rate of 11.1%, while a  $\gamma = 0.1$  would be a 900% interest rate! Note, importantly, that this interest rate is punitive, such as with a credit card. A factor  $> 1.0$  yields a negative number, which can be seen as the rate of return on an investment: the longer the agent is willing to wait to collect the reward, the more valuable it becomes.

## Problems with Value Iteration

Value iteration repeats the Bellman updates:

Problem 1: It's slow –  $O(S^2A)$  per iteration

$s, a, s'$

Problem 2: The “max” at each state rarely changes

Problem 3: The policy often converges long before the values

## Policy Iteration

Alternative approach for optimal values:

- **Step 1: Policy evaluation:** calculate utilities for some fixed policy (not optimal utilities!) until convergence

- **Step 2: Policy improvement:** update policy using one-step look-ahead with resulting converged (but not optimal!) utilities as future values

- Repeat steps until policy converges

This is **policy iteration**

- It's still optimal!

- Can converge (much) faster under some conditions

## Comparison

Both value iteration and policy iteration compute the same thing (all optimal values)

In value iteration:

- Every iteration updates both the values and (implicitly) the policy
- We don't track the policy, but taking the max over actions implicitly recomputes it

In policy iteration:

- We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them)
- After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
- The new policy will be better (or we're done)

Both are **dynamic programs** for solving MDPs

## Policy Iteration

Evaluation: For fixed current policy  $\pi$ , find values with policy evaluation:

- Iterate until values converge:

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$

Improvement: For fixed values, get a better policy using policy extraction

- One-step look-ahead:

$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$

**Summary: MDP Algorithms**

So you want to....

- Compute optimal values: use value iteration or policy iteration
- Compute values for a particular policy: use policy evaluation
- Turn your values into a policy: use policy extraction (one-step lookahead)

These all look the same!

- They basically are – they are all variations of Bellman updates
- They all use one-step lookahead expectimax fragments
- They differ only in whether we plug in a fixed policy or max over actions

### 17.2.3 Linear Programming

MDPs can also be solved using linear programming.

Goal:

Minimize utilities subject to Bellman constraints:

$$U(s) \geq \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')] \quad U(s) \geq \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')] \\ U(s) \geq \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')]$$

for every state and action.

This method runs in **polynomial time**, but is often slower than dynamic programming in practice.

### 17.2.4 Online Algorithms

Offline algorithms compute full policies before acting.

Online algorithms compute decisions **during execution**.

Used when state spaces are extremely large.

Example:



- Tetris with  $106210^{62}$  states

### 34. Expectimax Planning

Decision trees alternate between:

- **max nodes** (agent decisions)
- **chance nodes** (environment randomness)

Utilities propagate upward through the tree.

### 35. $\epsilon$ -Horizon

Future rewards become negligible after some depth **H**.

Depth needed

This limits search depth.

### 36. Monte Carlo Sampling

Large branching factors can be handled by sampling possible outcomes instead of enumerating them.

Expected value approximated by sample averages.

### 37. Real-Time Dynamic Programming (RTDP)

RTDP focuses computation on states likely to be visited.

Instead of solving the entire MDP, it solves a **sub-MDP** around the current state.

### 38. Monte Carlo Tree Search (UCT)

UCT explores future actions using simulated playouts.

Advantages:

- works well in huge state spaces
- useful for problems like Tetris

However, performance depends heavily on:

- simulation quality
- number of playouts

**Additional utility theory info found in recitation 6 (HW6) slides!!!**

### **Textbook readings 17.3 (Module 8)**

#### **17.3 Bandit Problems – Main Point Summary**

##### **1. What is a Bandit Problem?**

A **bandit problem** models the tradeoff between:

- **Exploitation** → Choose the action currently believed to give the highest reward.
- **Exploration** → Try less-tested actions to gain information that may improve long-term reward.

The classic metaphor is the **n-armed bandit (slot machine)**:

- Each arm has an unknown reward distribution.
- Each pull gives a stochastic reward.
- The goal is to maximize total (discounted or undiscounted) reward over time.

##### **Real-World Applications**

Bandit problems model decisions such as:

- Choosing medical treatments

- Selecting investments
- Funding research projects
- Online ad placement
- A/B testing systems

## 2. Formal Definition

Each arm  $M_i$  is modeled as a **Markov Reward Process (MRP)**:

- States:  $S_i$
- Transitions:  $P_i(s' | s)$
- Rewards:  $R_i(s, s')$

The overall bandit problem is an **MDP** where:

- State space = Cartesian product of all arms' states
- Actions = choose one arm to pull
- Only the selected arm changes state
- Discount factor =  $\gamma$

Key property:

Arms are **independent**, except that only one can be used at a time.

## 3. Deterministic Example Insight

Even if one arm looks better long-term, an optimal strategy may:

- Start with one arm,
- Switch at an optimal stopping time,
- Then stick with another arm.

Important insight:

**Optimal policies may involve switching arms at the right time**, not committing permanently to one arm.

#### 4. One-Armed Bandit & Stopping Time

A **one-armed bandit**:

- One uncertain arm  $M$
- One constant arm  $M_\lambda$  giving fixed reward  $\lambda$

Equivalent interpretation:

Pulling  $M$  has an **opportunity cost** of  $\lambda$ .

Optimal strategy:

- Pull  $M$  up to stopping time  $T$
- Then switch permanently to  $M_\lambda$

This leads to the definition of the

#### 5. Gittins Index

The **Gittins index** is the key theoretical result.

It defines a value for each arm based on:

Maximum expected discounted reward per unit of discounted time

#### Interpretation

- Numerator = discounted utility
- Denominator = discounted time
- Ratio = utility per discounted time unit

## The Remarkable Result

### Optimal Policy:

At every step, pull the arm with the **highest Gittins index**.

Important properties:

- Each arm's index depends only on that arm
- First decision:  $O(n)$
- Later decisions:  $O(1)$
- Provides a simple optimal solution to discounted bandits

## 6. Computing the Gittins Index

To compute it:

- Convert the arm into a **restart MDP**
- Add an action to restart the process
- Solve via value iteration
- Multiply optimal value by  $1 - \gamma$

For deterministic sequences, the index is the maximum ratio across stopping times.

## 7. Bernoulli Bandit

One of the most important special cases.

Each arm:

- Produces reward 1 with probability  $\mu_i$
- Produces reward 0 otherwise
- $\mu_i$  is unknown

State representation:

- $sis\_isi$  = number of successes
- $fif\_ifi$  = number of failures
- Updated via Bayesian updating (Beta prior)

### Key Insight

The Gittins index reflects:

- Estimated mean reward
- An **exploration bonus** for uncertain arms

Example:

An arm with slightly worse average reward but fewer trials may have a higher index because uncertainty makes it worth exploring.

## 8. Approximately Optimal Policies

Computing exact Gittins indices is often hard.

Two major approximate approaches:

### A. Upper Confidence Bound (UCB)

Where:

- $\mu^i \hat{\mu}_i$  = empirical mean
- $N_i$  = times arm pulled
- $N$  = total pulls
- Second term = exploration bonus

Choose arm with highest UCB.

Properties:

- Balances exploitation and exploration
- Regret grows as  $O(\log N)$
- Near-optimal performance

## B. Thompson Sampling

Method:

- Sample a value for each arm from its posterior distribution
- Pick the arm with highest sampled value

Properties:

- Bayesian
- Regret grows as  $O(\log N)$
- Simple and highly effective

## 9. Regret

Regret = difference between:

- Optimal clairvoyant policy (always pick best arm)
- Your learning algorithm

Fundamental result:

No algorithm can achieve regret growing slower than  $O(\log N)$

Both UCB and Thompson sampling achieve this optimal rate.

## 10. Selection Problems (Different from Bandits)

Bandits:

- Goal = maximize cumulative reward

Selection problems:

- Goal = identify best option as fast as possible.
- No cost for bad trials beyond fixed testing cost.

Examples:

- Choosing best drug after testing
- Hiring decisions
- Supplier selection

Important result:

No index policy (like Gittins) exists for selection problems.

Key difference:

Bandits assume failures cost real utility.

Selection problems only care about the final decision.

## 11. Bandit Superprocess (BSP)

Generalization:

- Each arm is a full **MDP**, not just an MRP.
- Only one arm can be worked on at a time.
- Represents **multitasking**.

Examples:

- Project management
- Construction planning
- Teaching multiple students



## Critical Insight

Global optimal policy  $\neq$  Local optimal policies.

Reason:

Opportunity cost changes incentives.

Because:

- Delaying reward in one task delays reward in all tasks.
- Leads to greedier behavior in each component MDP.

Local optimality and global optimality coincide only when:

$$\gamma=1 \setminus \gamma=1$$

## 12. Solving BSPs

Full Cartesian product MDP is exponential  $\rightarrow$  impractical.

Instead:

- Use **upper and lower bounds**
- Use opportunity cost modeling
- Use dominating policies (relaxed MDPs)

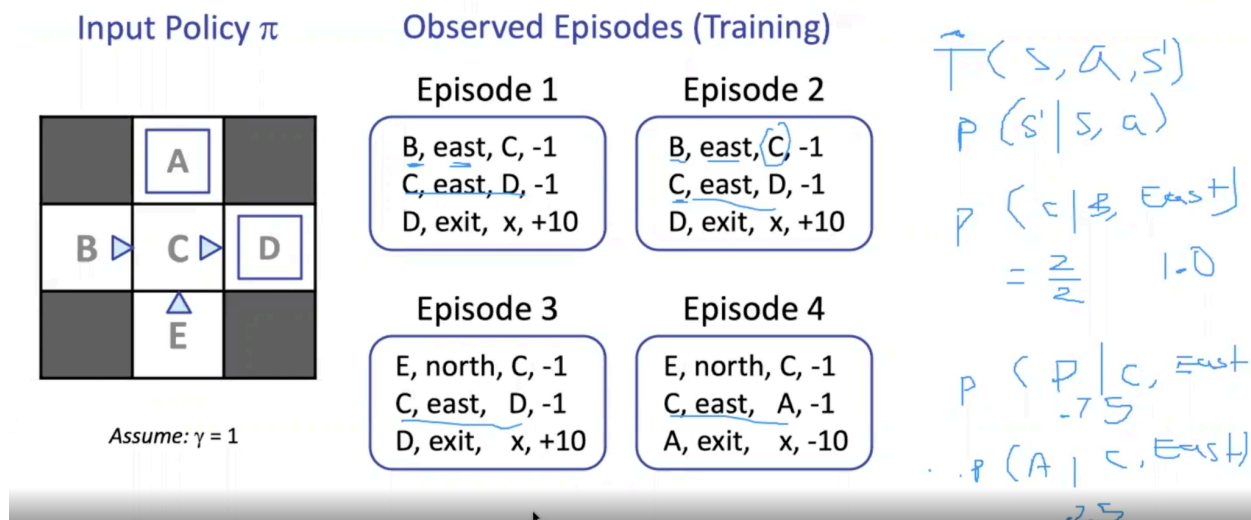
Combine with look-ahead search

This allows solving very large BSPs efficiently.

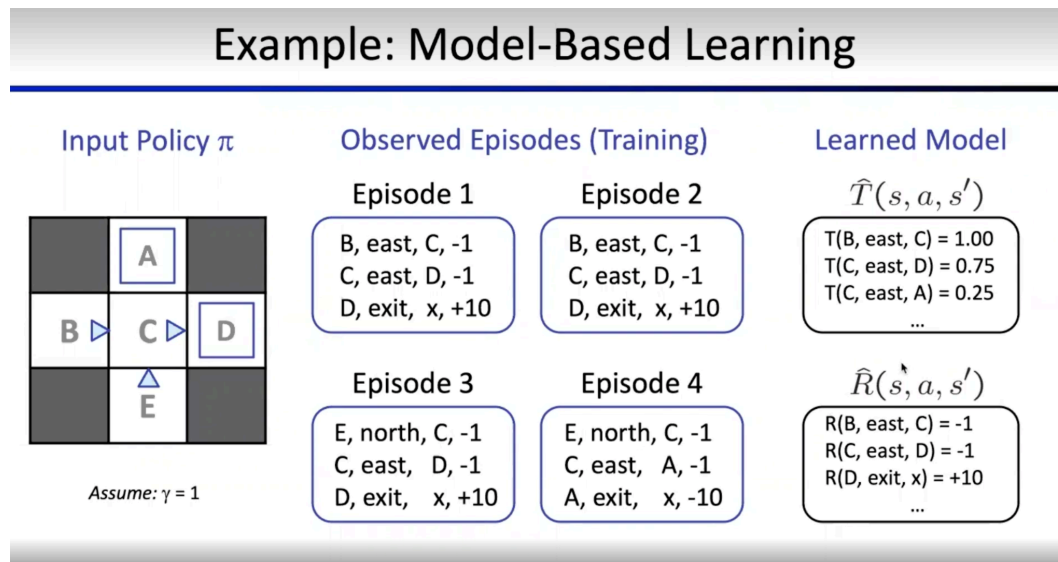
**Estimating transition function for reinforcement learning example:**

Introduction to Reinforcement Learning part 2 video from module 8!!!!

# Example: Model-Based Learning



Estimating reward function for reinforcement learning example:



Textbook  
readings  
22.1:

Chapter 22

## – Reinforcement Learning

### Section 22.1: Learning from Rewards — Detailed Main-Point Summary

#### 1. Supervised Learning vs Reinforcement Learning

##### Supervised Learning

In supervised learning:

- The agent learns from example input–output pairs.

- These examples are provided by a teacher.
- The goal is to learn a function mapping inputs to correct outputs.

Example:

Chess learning with supervised learning:

- Input → chess board position
- Output → correct move

Training data could come from:

- millions of recorded grandmaster games

However, supervised learning has major limitations for complex decision problems.

## **Problems with Supervised Learning for Complex Tasks**

**Even with large datasets, supervised learning struggles because:**

### **1. State space is extremely large**

Example:

- Possible chess positions  $\approx 10^{40}$
- Available examples  $\approx 10^8$

Thus most board states encountered in real games:

- do not appear in the training set

### **2. The system does not understand goals**

Supervised learning only teaches:

- which move was played

It does not explain why.

The agent lacks knowledge about:

- long-term goals (e.g., checkmate)
- consequences of actions

### **3. Limited real-world datasets**

For real-world tasks:

- large labeled datasets often do not exist

Example:

There is no database of millions of expert decisions for:

- driving
- robotics
- real-world planning tasks

## **2. Reinforcement Learning (RL)**

**Reinforcement learning solves this problem by allowing an agent to learn from interaction with the environment rather than labeled examples.**

Definition:

Reinforcement learning is a learning method where an agent:

1. interacts with the environment
2. takes actions
3. receives rewards
4. learns how to maximize long-term reward.

### **Connection to Markov Decision Processes**

Reinforcement learning uses the same goal as MDPs (Chapter 17):

maximize expected sum of rewards

But the key difference:

In MDP solving

The agent is given:

- transition model
- reward function

In Reinforcement Learning

The agent:

- **does not** know the model
- must learn it through experience

### **Key RL Concept**

The agent must act in order to learn.

Example:

Learning a new game:

- You don't know the rules
- You try actions
- Eventually someone says “You lose”

This feedback becomes the learning signal.

### **3. Advantages of Reinforcement Learning**

From an AI system design perspective, RL is often easier to set up than supervised learning.

#### **Advantage 1: Reward signals are easy to define**

**Example rewards:**

**Car racing:**

- **finish race**
- **crash**

**These reward signals are usually simple to program.**

#### **Advantage 2: No expert demonstrations required**

Supervised learning requires:

- expert labels for every example.

Reinforcement learning only requires:

- a reward signal.

Even non-experts can define rewards.

## **4. Sparse Rewards**

Many RL environments provide rewards only rarely.

This is called sparse rewards.

Example:

Chess reward:

- only at the end of the game

Car racing reward:

- only when race ends

Most states provide no feedback.

### **Why Sparse Rewards Are Difficult**

Sparse rewards make learning harder because:

- the agent receives little information about which actions were good or bad.

The credit assignment problem becomes difficult:

Which earlier action caused the eventual success or failure?

## **5. Intermediate Rewards**

Learning becomes easier when additional rewards are provided during the task.

Examples:

Tennis or cricket

Reward:

- each point scored

### **Robot crawling**

Reward:

- forward movement

**These additional signals guide the agent and accelerate learning.**

## **6. Reinforcement Learning Applications**

Reinforcement learning has become extremely powerful when combined with deep learning.

Important modern applications include:

Video games

Agents can learn to play Atari games from raw pixels.

## 7. Types of Reinforcement Learning Approaches

Reinforcement learning algorithms can be grouped into two main categories:

1. Model-based reinforcement learning
2. Model-free reinforcement learning

## 8. Model-Based Reinforcement Learning

In model-based RL:

The agent uses a model of the environment.

The model includes:

- transition probabilities
- reward structure

### Transition Model

The model describes:

$$P(s'|s,a)P(s'|s,a)P(s'|s,a)$$

Probability that action  $a$  in state  $s$  leads to state  $s'$ .

### Learning the Model

Two cases exist:

Known model

Example:

Chess program knows the rules.

Unknown model

Agent must learn the model by observing outcomes of actions.

Utility Learning

Model-based RL often learns a utility function:

$U(s)U(s)U(s)$

which represents the expected future reward starting from state  $s$ .

This is exactly the value function from Chapter 17.

## 9. Model-Free Reinforcement Learning

Model-free RL does not attempt to learn the environment model.

Instead, it directly learns how to behave.

## 10. Action-Utility Learning

In action-utility learning, the agent learns a function:

$Q(s,a)Q(s,a)Q(s,a)$

called the Q-function.

Meaning of  $Q(s,a)$

$Q(s,a)Q(s,a)Q(s,a)$

represents the expected total reward obtained if:

- the agent starts in state  $s$
- takes action  $a$
- and then follows an optimal policy.

Decision Rule

The agent chooses the action with the highest Q-value:

$a^* = \operatorname{argmax}_a Q(s,a) \quad a^* = \arg\max_a Q(s,a)$

Q-learning

The most famous action-utility learning algorithm is Q-learning.

It learns the Q-function through experience.

Q-learning is widely used because:

- it does not require a model
- it can learn optimal behavior directly.



## 11. Policy Search

Another model-free approach is policy search.

Instead of learning Q-values or utilities, the agent directly learns the policy.

### Policy Representation

A policy is a function:

$$\pi(a|s)$$

that maps states directly to actions.

### Active Reinforcement Learning

The agent must learn:

1. the utilities
2. which actions to choose

This introduces the major RL challenge:

### Exploration

The agent must explore the environment to discover:

- good actions
- bad actions.

## 13. Exploration Problem

If the agent always chooses the best-known action:

- it may miss better alternatives.

Therefore the agent must balance:

### Exploitation

Using the best-known action.

### Exploration

Trying new actions to gather information.

## 14. Scaling Reinforcement Learning

To handle complex problems, several techniques are used:

## **Inductive learning**

Learning general patterns from experience.

## **Deep learning**

Neural networks approximate:

- value functions
- policies

## **Pseudorewards**

Additional artificial rewards to guide learning.

## **Hierarchical actions**

Complex behaviors broken into smaller sub-actions.

## **15. Other Learning Approaches**

The chapter also introduces:

### **Policy search methods**

Directly optimizing policies.

### **Apprenticeship learning**

Learning behavior by observing demonstrations rather than reward signals.

## **Key Takeaways**

1. Reinforcement learning allows agents to learn by interacting with the environment and receiving rewards.
2. Unlike supervised learning, RL does not require labeled training examples.
3. The agent's goal is to maximize the expected sum of rewards.
4. Sparse rewards make learning difficult, but intermediate rewards can help.
5. RL approaches are divided into:
  - model-based RL
  - model-free RL

### **6. Model-free RL includes:**

- Q-learning (action-utility learning)
  - policy search
7. Reinforcement learning requires balancing exploration vs exploitation.
  8. Modern RL systems combine reinforcement learning with deep learning to solve complex tasks like games, robotics, and decision-making problems.

MDP's is **offline** planning while RL is **online** learning!!!

## Textbook readings 22.2 on Temporal Learning:

### 1. Core Idea of Temporal-Difference (TD) Learning

Temporal-difference learning is a method for **learning state utilities directly from experience** without needing to fully solve the MDP model first. Instead of computing utilities from a known transition model, TD learning uses **observed transitions between states** to gradually adjust utility estimates so they become consistent with the **Bellman-style relationship** between neighboring states.

The main intuition is:

- If the agent moves from state  $s$  to state  $s'$ ,
- and receives reward  $r$ ,
- then the utility of  $s$  should move toward:  

$$r + \gamma U(s') + \gamma U(s) - U(s)$$

This means a state's estimated value should agree with the immediate reward plus the discounted value of its successor.

### 2. Why TD Learning Is Useful

TD learning is an alternative to solving the MDP through full model-based methods such as ADP. It is useful because:

- It is **simpler**
- It requires **less computation per observation**
- It does **not require a transition model**
- It learns directly from actual state transitions experienced in the environment

So, instead of learning the transition probabilities and then solving Bellman equations exactly, TD learning updates utilities incrementally from real experience.

### 3. Basic TD Update Rule

When the agent observes a transition from state  $s$  to state  $s'$  under a fixed policy  $\pi$ , it updates the utility estimate of  $s$  using:

$$U^\pi(s) \leftarrow U^\pi(s) + \alpha [R(s, \pi(s), s') + \gamma U^\pi(s') - U^\pi(s)]$$

### Meaning of each part:

- $U^\pi(s)$ : current estimate of the utility of state  $s$
- $\alpha$ : learning rate
- $R(s, \pi(s), s')$ : immediate reward from going from  $s$  to  $s'$
- $\gamma U^\pi(s')$ : discounted utility of the next state
- **Bracketed term**: the **TD error**

## 4. TD Error

The quantity

$$R(s, \pi(s), s') + \gamma U^\pi(s') - U^\pi(s)$$

is called the **temporal-difference error**.

It measures how wrong the current utility estimate is:

- If the quantity is **positive**, then  $U(s)$  is too low and should increase.
- If it is **negative**, then  $U(s)$  is too high and should decrease.

So TD learning works by using the error between:

- current estimate of a state's value, and
- a one-step "better" estimate based on the reward plus next-state value.

## 5. Why It Is Called "Temporal-Difference"

It is called temporal-difference learning because the update is based on the **difference between utility estimates at successive times/states**. The agent compares:

- what it previously thought about state  $s$ ,
- with what the next moment suggests based on  $s'$ .

Thus, TD uses **successive time steps** to improve estimates.

## 6. Local Adjustment Toward Bellman Equilibrium

TD methods try to make utility estimates satisfy the local equilibrium condition implied by the Bellman equation. In passive reinforcement learning, the correct utility values should satisfy the fixed-policy Bellman constraints. TD does not solve all those equations globally at once. Instead, it makes **small local adjustments** after each observed transition.

Over time, these local corrections push the utility estimates toward the true values.

## 7. Why Updating from Only One Observed Successor Still Works

A subtle point is that the true Bellman equation depends on **all possible successor states and their probabilities**, but the TD update uses only the **one successor actually observed**.

At first, this may seem problematic because:

- a rare transition could cause a large update,
- even though it is not representative of the usual case.

But this works in the long run because:

- rare transitions happen rarely,
- common transitions happen often,
- so across many experiences, the average updates reflect the true transition probabilities.

Thus, the expected behavior of TD learning converges toward the correct utility values.

## 8. Importance of the Learning Rate $\alpha$

For TD learning to converge properly, the learning rate should **decrease over time** as a state is visited more often.

Why?

- Early on, the agent should learn quickly from new data.
- Later, once it has seen a state many times, updates should become smaller and more stable.

This decreasing step-size helps utility estimates eventually converge rather than endlessly fluctuating.

## 9. Passive TD Learner

In passive reinforcement learning, the policy  $\pi$  is fixed. The passive TD learner:

- follows the given policy,

- observes transitions and rewards,
- updates utility estimates for visited states.

The passive TD learner only needs:

- the current state
- the previous state
- the reward
- a utility table
- visit counts for learning-rate adjustment

It does **not** need to learn the full transition model or reward model.

## 10. Comparison with Passive ADP

TD learning and ADP are closely related because both try to make utility estimates agree with successor states. But they differ in important ways.

### ADP

- Learns a transition model and reward model
- Uses those models to solve Bellman equations
- Updates utilities based on **all possible successors**, weighted by probabilities
- More accurate and stable
- More computationally expensive

### TD Learning

- Does not learn a transition model
- Updates from **one observed successor at a time**
- Simpler and cheaper computationally
- Learns more noisily and with higher variability
- Usually slower than ADP in terms of learning precision

So TD can be viewed as a **simpler, cheaper approximation** to ADP.

## 11. Performance Characteristics of TD Learning

According to the text, TD learning:

- does not learn quite as fast as ADP,
- has higher variability,
- but requires much less computation per observation.

This means TD is attractive when:

- computation is limited,
- the model is unknown,
- or the state space is too large for exact model-based methods.

## 12. Key Advantage: No Transition Model Needed

One of the biggest advantages of TD learning is that it does **not require a transition model**.

Instead:

- the environment itself provides the information through observed transitions.

This is important because in many realistic problems:

- the transition probabilities are unknown,
- difficult to estimate,
- or too expensive to model exactly.

TD bypasses that requirement entirely.

## 13. TD as a First Approximation to ADP

The text describes TD as a **crude but efficient first approximation to ADP**.

Reason:

- ADP updates utilities by fully propagating changes through the model.
- TD only makes one local update per observed transition.

So ADP is more thorough, while TD is faster and cheaper.

## 14. Pseudoexperience

The text explains that ADP's multiple utility adjustments can be interpreted as if the agent were learning from additional **imagined experiences**, called **pseudoexperiences**.

Idea:

- If the agent has a transition model,
- it can simulate possible transitions that might happen,
- then use those simulated transitions to perform extra TD-style updates.

This improves learning because the agent is not limited to only real observed transitions.

So pseudoexperience bridges model-free TD and model-based ADP:

- TD learns from real experience,
- ADP can be thought of as learning from both real and simulated experience.

## 15. Approximate ADP

In very large state spaces, full ADP becomes infeasible because there are too many states and too many value updates to compute exactly.

Approximate ADP speeds this up by:

- limiting the number of utility adjustments after each observed transition,
- ignoring tiny updates that do not matter much,
- focusing computation on the most important changes.

This keeps learning efficient while still approximating the benefits of ADP.

## 16. Prioritized Sweeping

A major heuristic for approximate ADP is **prioritized sweeping**.

**Main idea:**

Only perform the most important utility updates first.

It prioritizes states whose likely successors have just had large utility changes.

Why this helps:



- If a successor's value changes a lot, predecessor states may also need significant adjustment.
- So computation is focused where it matters most.

This makes approximate ADP much more efficient while preserving strong learning performance.

## 17. Why Approximate ADP Can Be Better Early in Learning

Early in learning, the transition model is usually still inaccurate. That means:

- there is little value in solving the model very precisely,
- because the model itself is still wrong.

Approximate ADP helps by:

- avoiding expensive exact computation too early,
- using a minimum adjustment threshold,
- gradually becoming more precise as the model improves.

This avoids wasting effort on exact calculations based on a poor early model.

## Big-Picture Takeaways

- TD learning updates utility estimates directly from **observed transitions**.
- It uses the formula:  

$$U(s) \leftarrow U(s) + \alpha(r + \gamma U(s') - U(s))$$

$$U(s) \leftarrow U(s) + \alpha(r + \gamma U(s') - U(s))$$
- The bracketed term is the **TD error**.
- TD is **model-free** for utility learning: it does not need a transition model.
- It is simpler and computationally cheaper than ADP, but noisier and often less stable.
- ADP and TD are closely related: TD is essentially a lightweight approximation to ADP.
- **Pseudoexperience** and **prioritized sweeping** are ways to combine the efficiency of TD with some of the stronger propagation benefits of model-based approaches.

**Textbook readings 22.3 on Active Reinforcement Learning:**

## 22.3 Active Reinforcement Learning — Very Detailed Main-Point Summary

### 1. Passive vs. Active Reinforcement Learning

A **passive reinforcement learning agent** follows a **fixed policy** and only learns how good that policy is. It does not choose its own actions. By contrast, an **active reinforcement learning agent** must both:

- **learn about the environment**, and
- **decide which actions to take**

So active reinforcement learning is harder because the agent is not just evaluating a policy; it is trying to **find the optimal policy** while interacting with an initially unknown environment.

### 2. What the Active Agent Must Learn

In passive learning, the agent only needed to learn the transition behavior of the actions dictated by the fixed policy. In active learning, the agent must learn a **complete transition model** for **all actions**, because it now has freedom to choose among them.

The active ADP agent must therefore learn:

- the transition probabilities  $P(s' | s, a)$  for every state-action pair
- the reward function
- the utility values corresponding to the **optimal policy**, not just a fixed one

These optimal utilities satisfy the **Bellman optimality equation**:

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')] \\ U(s) = \max_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')]$$

This equation says:

- the utility of a state is the maximum expected return achievable from that state,
- assuming the agent chooses the best action available.

Once the model is learned, the agent can solve for the utilities using methods like:

- **value iteration**
- **policy iteration**

Then it can choose actions by:

- one-step look-ahead using the utility function, or

- directly following the policy found by policy iteration.

But this creates a major question:

Should the agent always execute the action that currently appears optimal in its learned model?

The answer is **no**, because doing so can prevent the agent from discovering better actions

### 22.3.1 Exploration

## 3. Why Always Acting Optimally Is Not Enough

A natural idea is:

- learn a model,
- compute the optimal action in that model,
- and always take that action.

An agent that does this is called a **greedy agent**.

A greedy agent always takes the action that it **currently believes** is best. This sounds reasonable, but it often fails because the learned model is still incomplete or wrong. Acting greedily can cause the agent to get stuck with a **suboptimal policy** simply because it never explores alternatives enough to learn that something better exists.

### Important insight:

The action that is optimal in the **learned model** may be suboptimal in the **true environment**.

So even if the agent is “optimal” relative to what it currently knows, it may still behave badly in reality.

## 4. Why the Greedy Agent Can Converge to the Wrong Policy

The greedy agent can settle too early on a path that seems good and then stop exploring. Once that happens:

- it no longer visits untried or rarely tried states,
- it never improves its knowledge about those parts of the state space,
- and it may permanently miss the true optimal policy.

This shows that actions have **two roles**:

1. They produce **rewards**

## 2. They produce **information**

A purely greedy agent considers only reward, not information value.

That is the same basic issue seen in **bandit problems**:

- **exploitation** = choosing the action that currently looks best
- **exploration** = trying actions that might reveal better long-term options

A successful active RL agent must balance both.

## 5. GLIE: Greedy in the Limit of Infinite Exploration

To solve the exploration problem, the text introduces the concept of a **GLIE** scheme:

### Greedy in the Limit of Infinite Exploration

A GLIE strategy must satisfy two conditions:

- **Every action in every state must be tried an unbounded number of times**
- **In the limit, the behavior should become greedy**

This means:

- early in learning, the agent explores enough to avoid missing good actions,
- later in learning, once enough information has been gathered, it increasingly exploits what it has learned.

This is enough to guarantee that the agent can eventually discover the optimal policy, even if the exploration schedule is not itself optimal.

## 6. A Simple GLIE Strategy

One basic GLIE scheme is:

- choose a random action at time step  $t$  with probability  $1/t$
- otherwise act greedily
- Epsilon greedy strategy

This works because:

- the probability of random exploration decreases over time,
- but never so fast that exploration completely stops,
- so every action continues to be tried infinitely often.

However, although this approach eventually converges, it may be **very slow**.

## 7. Better Exploration Through Utility Optimism

A better idea is to encourage the agent to try actions that have not been explored much, while still preferring actions that appear valuable.

This is done using an **exploration function** that modifies the Bellman update to include optimism for underexplored actions. **Optimistic utility estimate**  $U(s) + \frac{1}{N(s,a)}$ , which acts as though unexplored actions might lead to very good outcomes.

where:

- $N(s,a)$  = number of times action  $a$  has been tried in state  $s$
- $f(u,n)$  = exploration function
- $u$  = estimated utility
- $n$  = visit count

## 8. Exploration Function

The purpose of the **exploration function** is to balance:

- **greed**: preference for actions with high estimated utility
- **curiosity**: preference for actions that have not been tried much

So the function should be:

- **increasing in utility  $u$** , and
- **decreasing in count  $n$**

A simple version is:

where:

- $R + \frac{1}{N_e}$  is an optimistic estimate of the best possible reward
- $N_e$  is a threshold number of exploration attempts

This means:

- if an action has not yet been tried enough times, treat it as very promising,
- once it has been tried enough, use its actual estimated utility.

This forces the agent to explore each state-action pair at least  $N_e$  times.

## 9. Why the Optimistic Utility Must Be Propagated

A subtle but very important point is that the right-hand side uses  $U+U^++U+$  rather than ordinary  $UUU$ .

This matters because:

- once the agent has tried nearby actions many times, those actions no longer look novel,
- but if unexplored regions farther away are still optimistic,
- that optimism should propagate backward to encourage the agent to move toward those regions.

So the agent is not only encouraged to try unfamiliar actions directly, but also to move toward states that may lead to unexplored territory.

This makes exploration much more effective.

## 10. What Matters Most: Policy Loss vs. Utility Error

The text notes that with optimistic exploration, the **policy loss** can go to nearly zero quickly even if the **RMS error in utility estimates** remains somewhat high.

This is important conceptually:

- the agent does not need to know exact utilities everywhere,
- it mostly needs to learn enough to choose the right actions in important states.

In particular, the agent may stop caring about poorly rewarding parts of the state space once it knows they are bad and avoidable.

Main lesson:

Perfect utility estimates are less important than learning a near-optimal policy.

## 22.3.2 Safe Exploration

### 11. The Problem with Free Exploration

So far, the discussion assumes exploration is harmless. In many toy problems or simulations, this is fine:

- if the agent makes mistakes, it just learns from them,
- then it resets and tries again.

But in the real world, exploration may be **dangerous**.

Examples:

- a self-driving car crashing

- a robot falling into a ditch
- damaging equipment
- entering a state from which recovery is impossible

This means exploration is not just a question of learning speed; it is also a question of **safety**.

## 12. Irreversible and Absorbing States

The text highlights two especially dangerous categories:

### Irreversible actions

Actions after which no sequence of later actions can restore the original state.

### Absorbing states

States where:

- no future action changes anything meaningful,
- no useful reward can be obtained,
- the agent is effectively stuck forever.

These are especially problematic in real systems because exploratory mistakes may create permanent damage.

## 13. Why Maximum-Likelihood Models Can Be Unsafe

The standard ADP learner estimates the model using **maximum-likelihood estimation** and then acts as if that estimated model were correct.

This can be dangerous because:

- early data may be misleading,
- the model may be highly uncertain,
- rare catastrophic outcomes may not yet have been observed.

So the agent may treat a dangerous action as safe simply because it has not yet seen the bad consequences.

Example intuition:

If an agent has ignored a red light a couple of times without incident, it may incorrectly conclude that ignoring red lights is safe.

Thus, acting solely according to the best current estimate can be reckless.

## 14. Bayesian Reinforcement Learning

One solution is **Bayesian reinforcement learning**.

Instead of assuming one model is correct, the agent maintains:

- a **prior** distribution over possible models  $P(h)P(h)P(h)$
- a **posterior**  $P(h|e)P(h|e)P(h|e)$  after seeing evidence  $e$

If the agent were to stop learning now, the best policy would be the one that maximizes **expected utility under the posterior**:

$$\pi^* = \arg\max_{\pi} \sum_h P(h|e) U_h \pi^* = \arg\max_{\pi} \sum_h P(h|e) U_h \pi^* = \arg\max_{\pi} \sum_h P(h|e) U_h \pi^*$$

This means the agent chooses actions that perform well on average across plausible models.

**Big idea:**

Bayesian RL explicitly accounts for **uncertainty in the model**.

## 15. Exploration as a POMDP

If the agent expects to continue learning in the future, then the problem becomes even harder:

- actions influence not only rewards,
- but also future observations,
- which in turn update beliefs about the model.

So the exploration problem becomes an **exploration POMDP**, where:

- belief states are distributions over possible environment models

In principle, this exploration POMDP can be solved in advance, yielding an optimal exploration strategy for any future percept sequence.

In practice:

- this is usually **wildly intractable**.

Still, it provides the correct conceptual foundation:

exploration is fundamentally a planning problem under uncertainty about the model.

## 16. Bayesian Methods Still Do Not Guarantee Safety

An important warning from the text:

Being perfectly Bayesian does **not** automatically make the agent safe.

Why not?

Because unless the prior already represents the possibility of danger, the agent may still take exploratory actions that lead to catastrophe.



So Bayesian optimality is not the same as guaranteed safety.

## 17. Robust Control

Another approach is **robust control theory**.

Instead of assigning probabilities to models, the agent considers a **set of possible models**  $\mathcal{H}$ , and chooses the policy that performs best in the **worst case**:

$$\pi^* = \arg\max_{\pi} \min_{h \in \mathcal{H}} U_h(\pi) \quad \pi^* = \arg\max_{\pi} \min_{h \in \mathcal{H}} U_h(\pi)$$

This is a **minimax** approach:

- assume an adversary picks the worst plausible model,
- choose the policy that does best against that worst case.

This can prevent catastrophically unsafe behavior.

The text connects this to the **logical Wumpus agent**, which refuses to enter states that could contain a pit or Wumpus. That is robust control behavior: maximize worst-case utility over all logically possible models.

## 18. Drawback of Robust Control

The problem with worst-case reasoning is that it can be **too conservative**.

If the agent assumes the world is always trying to harm it, it may avoid acting at all.

Example:

A self-driving car that assumes every other car will deliberately crash into it may decide never to leave the garage.

So safe exploration requires balancing:

- caution,
- reward,
- and the value of learning.

## 19. Role of Human Knowledge in Safety

The text notes that human knowledge is often essential for safe reinforcement learning.

Two important ways humans help:

### A. Demonstrations / teacher trajectories

A human can provide example actions so the system starts with reasonable behavior instead of exploring blindly.

## B. Explicit safety constraints

A human can specify rules that are enforced externally to the RL learner.

Example:

A safety controller may override the learning system whenever the system reaches a state from which unsafe actions could lead to irrecoverable failure.

So the learning agent is free to explore only in states where safety can still be guaranteed.

## 20. Moving from Active ADP to Active TD

An active TD agent would need to choose actions as well as learn. One way to do this would be:

- learn a transition model,
- estimate utilities  $U(s)$ ,  $U(s)$ ,  $U(s)$ ,
- choose actions by one-step look-ahead.

This would make active TD similar to active ADP.

But **Q-learning** avoids this by learning something more useful directly.

## 21. What Q-Learning Learns

Instead of learning state utilities  $U(s)$ ,  $U(s)$ ,  $U(s)$ , Q-learning learns an **action-utility function**:

$Q(s,a)$ ,  $Q(s,a)$ ,  $Q(s,a)$

This is the expected total discounted reward obtained by:

- taking action  $a$  in state  $s$ ,
- then acting optimally afterward.

Once the agent knows  $Q(s,a)$ ,  $Q(s,a)$ ,  $Q(s,a)$ , it can act optimally by choosing:

$\arg\max_a Q(s,a)$

This means:

- no transition model is needed,
- no look-ahead computation is required at decision time.

## 22. Bellman Equation for Q-Values

The Bellman equation for Q-values is:

$$Q(s,a) = \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma \max_{a'} Q(s',a')] \\ Q(s,a) = \sum_{s'} P(s'|s,a) \left[ R(s,a,s') + \gamma \max_{a'} Q(s',a') \right]$$

This says:

- the value of taking action  $a$  in state  $s$
- equals the expected immediate reward
- plus the discounted value of the best action in the next state.

## 23. Q-Learning Update Rule

From this Bellman equation, the TD update for Q-learning is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$
$$Q(s,a) \leftarrow Q(s,a) + \alpha [R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

### Meaning:

- update after taking action  $a$  in state  $s$  and landing in  $s'$
- compare the current estimate  $Q(s,a)$  with a one-step target:
  - immediate reward
  - plus discounted value of the best next action

The difference between these is the **Q-learning TD error**.

## 24. Why Q-Learning Is Important

The most important feature of Q-learning is what it **does not require**:

- no transition model  $P(s'|s,a)$  is needed
- not for learning
- and not for action selection

This makes Q-learning a **model-free** reinforcement learning method.

That is a major advantage in complex environments where:

- modeling transitions is difficult,
- state spaces are huge,
- or the environment is not well understood.

## 25. Limitation of Q-Learning

Because Q-learning does not have a model, it cannot explicitly “look ahead.”

This means it can struggle when:

- rewards are sparse,
- long sequences of actions are needed before reward appears,
- and credit assignment is difficult.

So model-free methods are flexible, but they may learn more slowly in long-horizon sparse-reward settings.

## 26. Exploratory Q-Learning Agent

The exploratory Q-learning agent:

- stores Q-values for state-action pairs,
- stores counts  $N(s,a)$  for exploration,
- updates Q-values after each transition,
- then chooses the next action using the same exploration function  $\epsilon$  used in exploratory ADP.

So exploration in Q-learning can also be driven by optimism or reduced randomness over time.

If a simpler strategy like decaying random exploration is used, then state-action count statistics are less necessary.

## 27. SARSA

A close relative of Q-learning is **SARSA**, named for the sequence:

- **State**
- **Action**
- **Reward**
- **State**
- **Action**

SARSA uses the update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R(s,a,s') + \gamma Q(s',a') - Q(s,a)]$$

The key difference:

- **Q-learning** uses the value of the **best possible next action**
- **SARSA** uses the value of the **actual next action taken**

So SARSA updates from the actual behavior trajectory.

## 28. Q-Learning vs. SARSA

This distinction matters mainly when **exploration** is happening.

**If behavior is greedy:**

- both algorithms are effectively the same

**If exploration occurs:**

- **Q-learning** still backs up from the best next action, even if the agent does something exploratory
- **SARSA** backs up from the actual exploratory action taken

Therefore:

- if exploration leads to a bad outcome, **SARSA penalizes accordingly**
- **Q-learning may not**, because it assumes future behavior is optimal

## 29. Off-Policy vs. On-Policy

This leads to the major conceptual distinction:

**Q-learning = Off-policy**

It learns:

What is the value of this action if, after this, I behave optimally?

So it is learning about the greedy target policy even while behaving differently during exploration.

**SARSA = On-policy**

It learns:

What is the value of this action if I continue following the policy I am actually using?

So SARSA reflects the performance of the actual behavior policy, including its exploratory choices.

## 30. When Each Is Preferable

**Q-learning is better when:**

- you want flexibility,
- you may use many different exploration policies,
- you care about learning the value of optimal behavior.

#### **SARSA is better when:**

- your future behavior is constrained by exploration or external influences,
- or other agents/programs partly determine your actions,
- so you want to learn what will actually happen under the current behavior policy.

### **31. Why Q-Learning and SARSA Learn More Slowly than ADP**

The text notes that both Q-learning and SARSA eventually learn the optimal policy in the example world, but they do so more slowly than the ADP agent.

Reason:

- ADP has a model and can propagate information through many states consistently.
- Q-learning and SARSA make only **local updates** from individual transitions.
- They do not enforce global consistency through a learned model.

#### **Textbook readings 22.4 - 22.5:**

### **22.4 Generalization in Reinforcement Learning — Main Point Summary**

#### **1. Why Tabular RL Breaks Down**

So far, utility functions and Q-functions have been represented in **tables**, with one value per state or state-action pair. This works for small environments, but it fails in large state spaces because:

- there are too many states to store explicitly,
- the agent cannot visit all of them enough times,
- learning would be far too slow.

Example:

- Grid worlds are small enough for tables.
- Backgammon has about  $10^{20}$  states, so tabular learning is impractical.

This motivates **function approximation**, where the agent learns a compact approximation of the utility function or Q-function instead of storing exact values for every state.

## 2. Function Approximation

Function approximation means representing  $U(s)$  or  $Q(s,a)$  with a parameterized function rather than a table.

Example linear approximation for utilities:

$$\hat{U}(s) = \theta_1 f_1(s) + \theta_2 f_2(s) + \dots + \theta_n f_n(s) \\ \hat{U}(s) = \theta_1 f_1(s) + \theta_2 f_2(s) + \dots + \theta_n f_n(s)$$

where:

- $f_1, \dots, f_n$  are features of the state,
- $\theta_1, \dots, \theta_n$  are learned parameters.

Main idea:

Instead of learning a value for each individual state, the agent learns a **small number of parameters** that define a general pattern across states.

This makes RL practical in huge state spaces.

## 3. Inductive Generalization

The most important benefit of function approximation is not just compact storage, but **generalization**.

The agent can:

- learn from states it has visited,
- then infer approximate values for states it has never seen.

This is called **inductive generalization**.

Example:

A backgammon agent can reach champion-level play while exploring only a tiny fraction of the total state space because it generalizes from experience.

## 4. Function Approximation Plus Look-Ahead Search

Approximate value functions can also be combined with **look-ahead search**.

This helps because:

- the function approximator only has to be good enough to guide search,
- the search can refine decisions locally,
- fewer experiences are needed to learn useful behavior.

So function approximation and search often work well together.

## 22.4.1 Approximating Direct Utility Estimation

### 5. Direct Utility Estimation with Function Approximation

In direct utility estimation, the agent runs trials and records, for each visited state, the **total reward from that state until termination**.

Each such pair:

- state  $s$
- observed return  $u_j(s)$

becomes a supervised learning example.

So direct utility estimation can be turned into an ordinary supervised regression problem.

### 6. Example Linear Utility Function

For the  $4 \times 3$  world, suppose state features are just coordinates  $(x,y)$ . Then we can approximate utility

This is linear in the parameters.

The agent learns the parameters so predicted utilities better match observed returns.

### 7. Widrow–Hoff Rule / Delta Rule

To update parameters online after each trial, use gradient descent on squared error.

If:

- $u_j(s)$  = observed return from state  $s$  on trial  $j$ ,
- $\hat{U}_\theta(s)$  = predicted utility,

then define error:

$$E_j(s) = (U^\theta(s) - u_j(s))^2$$

Gradient descent gives update:

$$\theta_i \leftarrow \theta_i + \alpha [u_j(s) - \hat{U}_\theta(s)] \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i}$$

This is the **Widrow–Hoff rule**, also called the **delta rule**.

For the linear approximator:



## 8. Why This Enables Generalization

Updating the parameters based on one state automatically changes predictions for many other states too.

That is the key difference from tabular learning:

- tabular update changes only one entry,
- function approximation changes a whole region of state space.

This speeds learning if the chosen hypothesis space is appropriate.

## 9. Importance of Good Features

Function approximation works well only if the chosen features can represent the true utility function reasonably well.

## 10. Nonlinear Features Inside Linear Models

A linear approximator can still model nonlinear structure if the **features themselves are nonlinear**.

Example feature:

$$f_3(x,y) = (x-x_g)^2 + (y-y_g)^2$$

This feature measures distance to the goal.

So the approximator remains linear in parameters, but can represent more complex shapes through clever feature design.

## 22.4.2 Approximating Temporal-Difference Learning

### 11. Function Approximation for TD Learning

The same function approximation idea can be applied to **TD learning**.

Instead of fitting observed full returns, TD adjusts parameters to reduce the **temporal difference error** between successive states.

For utilities:

For Q-learning:

$$\begin{aligned} \theta_i &\leftarrow \theta_i + \alpha [R(s,a,s') + \gamma \max_{a'} Q^{\theta}(s',a') - Q^{\theta}(s,a)] \frac{\partial Q^{\theta}(s,a)}{\partial \theta_i} \\ \theta_i &\leftarrow \theta_i + \alpha [R(s,a,s') + \gamma \max_{a'} Q^{\theta}(s',a') - Q^{\theta}(s,a)] \frac{\partial Q^{\theta}(s,a)}{\partial \theta_i} \end{aligned}$$

These are the approximate versions of the standard TD and Q-learning updates.

## 12. Convergence Properties

For **passive TD learning** with a **linear function approximator**, convergence can be shown to the best possible approximation within the function class.

But for:

- active learning,
- nonlinear approximators,
- neural networks,

things become much less stable.

The parameters can sometimes diverge to infinity, even when a good solution exists.

So RL with general function approximators is much harder and less well-behaved than ordinary supervised learning.

## 13. Catastrophic Forgetting

A major problem with function approximation is **catastrophic forgetting**.

Example:

Suppose a self-driving car learns that the center of the road is safe and gradually stops visiting the dangerous edges. Over time, because training data now comes only from the middle, it may “forget” the edges are dangerous.

Then small parameter changes can make it drift off the road and crash.

So function approximation can overwrite old important knowledge when new training data is narrow or unbalanced.

## 14. Experience Replay

A common solution is **experience replay**.

Idea:

- store old trajectories from earlier learning,
- replay them later during training,
- make sure the agent continues learning from parts of state space it no longer visits often.

This helps preserve old knowledge and reduces catastrophic forgetting.

In modern deep RL, experience replay is one of the most important stabilization tools.

## 15. Function Approximation for Model Learning

Function approximation is also useful in **model-based RL** for learning transition models.

In fully observable environments:

- learning the next state from the current state and action is a supervised learning problem.

So supervised learning methods from earlier chapters can be used to learn the model.

A learned model then supports:

- look-ahead search,
- internal simulation,
- improved utility or Q approximations without costly real-world experience.

## 16. Partially Observable Environments

In partially observable environments, model learning is harder because the next percept does not directly reveal the underlying state.

If the hidden variables and their structure are known:

- a dynamic Bayesian network can be used,
- EM can learn the parameters.

But learning hidden state structure itself is still very difficult.

The text notes that deep recurrent neural networks can sometimes invent useful hidden structures automatically.

### 22.4.3 Deep Reinforcement Learning

## 17. Why Go Beyond Linear Approximation

There are two main reasons linear function approximation may be insufficient:

1. No good linear approximation exists.
2. We may not know which features to invent.

These are really the same issue:

- linear models are only as good as the features,
- and in many domains good features are unknown.

## 18. Deep Neural Networks as Function Approximators

To solve this, RL can use **deep neural networks** as nonlinear function approximators.

Advantages:

- can model very complex utility or Q-functions,
- can take raw input such as images,
- may automatically discover useful features.

This reduces dependence on hand-designed features.

A reinforcement learning system using deep networks is called **deep reinforcement learning**.

## 19. Deep RL Training

Even though the function is much more complicated, the updates are still based on gradients like those used in supervised learning.

The necessary gradients for TD/Q-learning updates are computed using **backpropagation**.

So deep RL combines:

- RL objectives and update rules,
- with deep learning optimization machinery.

## 20. Successes and Limitations of Deep RL

Deep RL has achieved major successes, including:

- expert-level play in video games,
- defeating the human world champion in Go,
- robotic control for complex tasks.

But it still has major problems:

- training can be unstable,
- performance can be difficult to reproduce,

- agents may fail unpredictably when the environment changes slightly,
- commercial use is still relatively limited compared with other deep learning areas.

So deep RL is powerful, but fragile.

#### 22.4.4 Reward Shaping

### 21. Credit Assignment Problem

In environments with sparse rewards, the agent may execute a huge number of primitive actions before receiving feedback.

Then it has to figure out which earlier actions were responsible.

This is the **credit assignment problem**.

### 22. Reward Shaping

A standard solution is **reward shaping**:

- provide additional rewards during learning,
- reward intermediate progress toward the real goal.

These additional rewards are called **pseudorewards**.

Examples for a soccer robot:

- reward touching the ball,
- reward moving the ball toward the goal.

Reward shaping can dramatically speed learning.

### 23. Danger of Bad Pseudorewards

The danger is that the agent may optimize the pseudorewards rather than the true task.

Example:

Instead of scoring, the robot may learn to stand next to the ball and vibrate, generating many “ball contact” rewards.

So pseudorewards must be chosen carefully.

### 24. Potential-Based Reward Shaping

Chapter 17 provided a safe form of reward shaping:

$$R'(s,a,s')=R(s,a,s')+\gamma\Phi(s')-\Phi(s) \quad R'(s,a,s') = R(s,a,s') + \gamma\Phi(s') - \Phi(s)$$

where  $\Phi(s)$  is a potential function.

This is important because it **does not change the optimal policy**.

So the agent learns faster, but the true objective remains unchanged.

## 25. Designing the Potential Function

The potential function can encode useful progress measures, such as:

- distance to the goal,
- possession of the ball,
- achievement of subgoals.

Example:

For a soccer robot,

- extra potential for having possession,
- extra potential for reducing the ball's distance to the opponent's goal.

This helps learning while preserving correct long-term behavior.

## 22.4.5 Hierarchical Reinforcement Learning

### 26. Motivation for HRL

Another way to handle long action sequences is to break complex behavior into a hierarchy of smaller tasks.

This is **hierarchical reinforcement learning (HRL)**.

Instead of learning one huge flat policy, the agent learns:

- high-level decisions,
- lower-level skills,
- subskills within those skills.

This is similar to hierarchical task network planning.

### 27. Example: Soccer

Scoring a goal can be decomposed into subgoals such as:

- obtain possession,
- pass to a teammate,
- receive a pass,
- dribble,
- shoot.

Each of those can be decomposed further into lower-level motor behaviors.

This makes learning easier because shorter subproblems are more manageable.

## 28. Keepaway Example

The text uses **keepaway**, a simplified soccer game:

- three “keepers” try to retain possession,
- two “takers” try to steal the ball.

The game is implemented in the RoboCup simulator and is a standard HRL benchmark.

## 29. Partial Program

HRL begins with a **partial program** that provides the high-level structure of behavior.

The program includes unspecified choices to be learned.

Example top-level keeper behavior:

- if I have the ball: choose among PASS, HOLD, DRIBBLE
- otherwise: choose among STAY, MOVE, INTERCEPT-BALL

So human designers provide the behavioral scaffolding, while the agent learns the detailed choices.

## 30. Choice Points and Learning

A **choose( . . . )** operator specifies places where learning must decide what to do.

Learning may associate a Q-function with each choice point.

Once learned, the program becomes a complete policy by choosing the highest-valued option at each choice.

So HRL combines:

- human-supplied structural knowledge,
- with RL for optimization of choices.

### 31. Why HRL Is Useful

Humans may not know how to compute the exact kick speed or pass direction to maximize success, but they often do know the rough behavioral structure.

HRL exploits this:

- humans specify the organization,
- RL fills in the details.

This can be far easier than learning everything from scratch.

### 32. Joint State Space

The theory of HRL uses a **joint state space** consisting of:

- physical state sss,
- machine state mmm

So a joint state is  $(s,m)(s,m)(s,m)$ .

The machine state includes internal program information such as:

- which subroutine is active,
- where the program counter is,
- local/global variable values,
- routine arguments.

This matters because behavior depends not only on the physical world but also on where the agent is inside its own program.

### 33. Choice States

A **choice state** is a joint state where the program reaches a choice point.

Between choice states, actions may still occur, but they are fully determined by the existing program structure rather than active decision making.



So HRL turns the problem into a Markovian decision problem over these higher-level choice states.

### **34. HRL as an MDP / Semi-MDP**

The corresponding HRL decision problem has:

- states = choice states in joint space,
- actions = choices available in the partial program,
- reward = expected sum of physical rewards between choice states,
- transition model = induced by both physical dynamics and program execution.

Because a choice may trigger a temporally extended behavior, the resulting process is technically a **semi-Markov decision process**.

### **35. Benefits of HRL**

HRL can be extremely effective.

In the keepaway example:

- the hierarchical agent learns to keep possession indefinitely against the standard opponent,
- greatly outperforming earlier approaches.

Key advantage:

lower-level skills are not fixed routines; they adapt depending on the full machine state and current context.

So the same lower-level behavior can be reused flexibly in different higher-level situations.

### **36. Shaping Rewards in HRL**

HRL also creates new opportunities for reward shaping.

Because the internal program state is explicit in the joint state space, shaping rewards can depend on internal concepts like:

- intended pass recipient,
- subgoal being pursued.

These concepts are not part of the physical world alone, so standard flat RL cannot easily use them.

This is a major advantage of HRL.

### 37. Additive Decomposition

Hierarchy also induces an **additive decomposition** of utility.

Overall utility is the sum of rewards gathered during subtasks.

Example:

- reward during PASS-TO(Ali)
- plus reward during MOVE-INTO-SPACE

Each piece depends on a smaller number of variables, which makes learning easier than learning one giant monolithic utility function.

This is analogous to factorization ideas in Bayesian networks.

## 22.5 Policy Search — Main Point Summary

### 38. Basic Idea of Policy Search

Policy search is another RL approach.

Instead of learning utilities or Q-values and then extracting a policy, policy search learns the policy directly by adjusting its parameters to improve performance.

Core idea:

- keep changing the policy while performance improves,
- stop when no more improvement is found.

### 39. Parameterized Policies

A policy is a mapping from states to actions.

For practical RL, policies are represented with relatively few parameters.

Example:

$$\pi^*(s) = \arg\max_a Q^\theta(s,a) \quad \hat{\pi}(s) = \arg\max_a \hat{Q}_\theta(s,a) \quad \pi^*(s) = \arg\max_a Q^\theta(s,a)$$

where each  $Q^\theta(s,a)$  may itself be linear or nonlinear.

Policy search then adjusts  $\theta$  to improve performance.

### 40. Policy Search vs Q-Learning

Even if the policy is represented using Q-functions, policy search is **not the same as Q-learning**.

### Q-learning:

tries to make  $Q^\theta \approx Q_{\theta}$  close to the true optimal Q-function  $Q^*$ .

### Policy search:

tries to find parameters that yield good behavior, whether or not  $Q^\theta \approx Q_{\theta}$  resembles  $Q^*$ .

So good policy performance does not require accurate value estimation.

Example:

A Q-function scaled by 1/100 may still induce the same optimal policy, even though it is numerically far from the true  $Q^*$ .

## 41. Discontinuity Problem with Deterministic Policies

A deterministic policy based on  $\arg\max$  can be discontinuous in the parameters.

A tiny change in parameters may suddenly switch the chosen action.

This makes gradient-based optimization difficult because the policy value may also change discontinuously.

## 42. Stochastic Policies

To fix this, policy search often uses **stochastic policies**, where:

$$\pi_\theta(s,a)$$

is the probability of choosing action  $a$  in state  $s$ .

A common representation is **softmax**:

$$\pi_\theta(s,a) = \frac{e^{\beta Q_\theta(s,a)}}{\sum_{a'} e^{\beta Q_\theta(s,a')}} \quad \pi_\theta(s,a) = \frac{e^{\beta Q_\theta(s,a)}}{\sum_{a'} e^{\beta Q_\theta(s,a)'}}$$

where  $\beta > 0$  controls how sharp the distribution is.

- large  $\beta$ : almost greedy choice
- small  $\beta$ : almost uniform random

Because softmax is differentiable, it is better suited for gradient-based search.

## 43. Policy Value

The **policy value**  $\rho(\theta)$  is the expected reward-to-go obtained by executing policy  $\pi_\theta$ .

Policy search aims to maximize  $\rho(\theta)$ .

If  $\rho(\theta)$  is differentiable, then one can use the **policy gradient**:

$$\nabla_{\theta} \rho(\theta)$$

to improve the policy.

#### 44. Simple Case: Deterministic Environment

If the environment and policy are deterministic and  $\rho(\theta)$  can be computed or estimated reliably, policy search reduces to ordinary optimization.

One can:

- compute the gradient directly if available,
- or estimate improvement with hill climbing.

This will converge to a local optimum under the usual assumptions.

#### 45. Difficulty in Stochastic Environments

In stochastic environments, evaluating a policy is noisy because total reward varies a lot from trial to trial.

This makes it hard to compare:

- $\rho(\theta)$
- and  $\rho(\theta + \Delta\theta)$

because differences may be drowned out by randomness.

A brute-force solution is to run many trials, but this can be expensive or dangerous.

#### 46. Policy Gradient Estimation

For stochastic policies, it is possible to derive an **unbiased estimate of the policy gradient directly from trial data**.

In the simplest episodic setting, the gradient can be estimated from observed rewards and the gradient of action probabilities.

In the sequential case, the approximate gradient becomes:

$$\nabla_{\theta} p(\theta) \approx \frac{1}{N} \sum_{j=1}^N \nabla_{\theta} \pi_{\theta}(s, a_j) \pi_{\theta}(s, a_j) \frac{u_j(s) - \bar{u}(s)}{\pi_{\theta}(s, a_j)}$$

where:

- $a_{j,j}$  is the action taken on trial  $j$ ,
- $u_j(s)$  is the reward from state  $s$  onward.

This gives a way to improve policies directly from sampled experience.

## 47. REINFORCE

This method is the basis of the **REINFORCE** algorithm.

REINFORCE:

- uses sampled trajectories,
- estimates the policy gradient,
- updates policy parameters in the direction of improvement.

It is usually much more effective than simple hill climbing.

## 48. Correlated Sampling

Comparing two policies by running each on different random trials can require a huge number of samples because randomness adds noise.

A better solution is **correlated sampling**:

- generate the same random scenarios for both policies,
- compare them under identical randomness.

Example:

In blackjack, if both policies play the exact same pre-generated hands, most of the noise from card luck disappears.

This makes comparison far more efficient.

## 49. PEGASUS

Correlated sampling was used in the **PEGASUS** policy-search algorithm.

PEGASUS achieved major success, including early stable autonomous helicopter flight.

Key theoretical point:

the number of random scenarios needed depends on the complexity of the policy space, not on the complexity of the environment itself.

That makes correlated sampling especially powerful for difficult domains.

## **Big-Picture Takeaways**

### **50. Why Generalization Matters**

Tabular RL does not scale to realistic state spaces. Function approximation allows the agent to learn compact value or Q representations and generalize to unseen states.

### **51. Function Approximation Creates Both Power and Risk**

It speeds up learning and makes large problems tractable, but can introduce instability, divergence, and catastrophic forgetting.

### **52. Deep RL Automates Feature Discovery**

Deep neural networks can learn useful features directly from raw input, but deep RL remains fragile and difficult to deploy robustly.

### **53. Reward Shaping Helps With Sparse Rewards**

Pseudorewards can dramatically speed learning, but naive shaping may distort the objective. Potential-based shaping is safe because it preserves the optimal policy.

### **54. HRL Uses Structure to Learn Long Behaviors**

Hierarchical RL breaks difficult tasks into manageable subtasks, allowing learning to exploit high-level structure and internal machine state.

### **55. Policy Search Learns Behavior Directly**

Rather than learning values and deriving a policy, policy search optimizes the policy itself, often through stochastic differentiable representations and policy gradients like REINFORCE.

**Q-learning** is a model-free, off-policy reinforcement learning algorithm that enables an agent to learn the value of actions (known as Q-values) through trial and error. Its primary goal is to find an optimal policy—a strategy that tells the agent which action to take in each state to maximize long-term rewards.

#### **How Q-Learning Works**

The algorithm operates by maintaining a "cheat sheet" called a Q-table.

- Q-Table: A lookup table where **rows represent environment states** and **columns represent possible actions**. Each cell contains a Q-value which estimates the "quality" or expected future reward of taking action in state.
- The Update Rule: After performing an action, the agent updates its Q-table using the Bellman Equation:
- - (Learning Rate): Determines how much new information overrides old knowledge.
  - (Discount Factor): Balances immediate rewards against future rewards.
  - $Q_{max}$  : The maximum expected reward from the next state, representing the best future the agent can envision.

## Key Characteristics

- Model-Free: The agent does not need to know the environment's rules or transition probabilities beforehand; it learns solely through interaction.
- Off-Policy: It learns the optimal policy independently of the agent's current actions, which may involve random exploration.
- Exploration vs. Exploitation: Agents typically use an
- -greedy policy. They mostly "exploit" the best-known actions but occasionally "explore" random actions to discover better strategies.

**RL Practice worksheet** answers explained (first question/answer found in my second lecture notebook):

### Question 2.

Consider a *two-armed bandit problem*, wherein we model an agent that can choose to exploit one of two core actions:  $A$  and  $B$ . Selecting  $A$  leads to a reward of  $r_A$  with probability  $p_A$  and a reward of 0 with probability  $1 - p_A$ . Selecting  $B$  leads to a reward of  $r_B$  with probability  $p_B$  and a reward of 0 with probability  $1 - p_B$ .

In our course, we have always modeled MDPs and Reinforcement Learning Problems as having deterministic rewards; that is, for a given state-action-state triple of  $(s, a, s')$ ,  $R(s, a, s')$  is a *constant* and not drawn from a probability distribution. Can you come up with a set of states, actions, a transition function, and a **constant valued** reward function that models this *two-armed bandit problem* in a repeatable setting with  $\gamma = 1$ ?

#### Solution:

There are two states: LOSE and WIN. Start in LOSE. From each state, there are two actions: PULL-A and PULL-B.

|              | LOSE      | WIN   |
|--------------|-----------|-------|
| LOSE, PULL-A | $1 - p_A$ | $p_A$ |
| LOSE, PULL-B | $1 - p_B$ | $p_B$ |
| WIN, PULL-A  | $1 - p_A$ | $p_A$ |
| WIN, PULL-B  | $1 - p_B$ | $p_B$ |

Table 1:  $T(s, a, s')$

|              | LOSE | WIN   |
|--------------|------|-------|
| LOSE, PULL-A | 0    | $r_A$ |
| LOSE, PULL-B | 0    | $r_B$ |
| WIN, PULL-A  | 0    | $r_A$ |
| WIN, PULL-B  | 0    | $r_B$ |

Table 2:  $R(s, a, s')$

### Question 3.

Consider a *two-armed bandit problem*, wherein we model an agent that can choose to exploit one of two core actions:  $A$  and  $B$ . Selecting  $A$  leads to a reward of  $r_A$  with probability  $p_A$  and a reward of 0 with probability  $1 - p_A$ . Selecting  $B$  leads to a reward of  $r_B$  with probability  $p_B$  and a reward of 0 with probability  $1 - p_B$ .

From Wikipedia: The *regret*,  $\rho$ , for a two-armed bandit problem after  $T$  rounds can be expressed using the following formula:

$$\rho = T\mu^* - \sum_{t=1}^T \hat{r}_t$$

Here,  $\mu^*$  refers to the optimal average rewards accrued by an optimal policy over  $T$  turns.  $\hat{r}_t$  refers to the reward actually earned on round  $t$ .

We can't compute  $\sum_{t=1}^T \hat{r}_t$  without actually running the simulation for some number of simulations.

A simple reinforcement learning strategy in this setting is to prefer one action for  $T$  consecutive rounds, but to choose the other action with (small) probability  $\varepsilon$ .

1. What is  $\mu_A$ , the expected utility derived assuming that we choose action  $A$  on a given round? What about  $\mu_B$ ?
2. What is the expected utility that we will accrue over  $T$  rounds if we use the strategy described above, preferring action  $A$ ?
3. What is the expected regret if we employ the strategy above for  $T$  rounds preferring action  $A$  if the optimal policy is to always choose action  $B$ ?
4. What is the expected regret if we employ the strategy above for  $T$  rounds preferring action  $A$  if the optimal policy is to always choose action  $A$ ?
5. With reference to #3, #4 and the definition of regret, explain why we say in the slides that a problem with choosing random actions forever is that *you do eventually explore the space, but keep thrashing around once learning is done*.

Refer to solution to above problem in my chat gpt chat on Saturday slowly and nicely called:

**“AI course textbook summary readings”**

**In Q-learning, the learning rate (alpha,**



) is a hyperparameter that determines how much newly acquired information overrides existing knowledge. It typically ranges from 0 to 1.

#### Role in the Q-Update Formula

The learning rate is used in the Temporal Difference (TD) update rule to adjust the stored "quality" values (Q-values) for state-action pairs:

- : The agent learns nothing and only uses its existing knowledge.
- : The agent ignores all prior knowledge and considers only the most recent experience.

#### Strategic Selection

Choosing the correct learning rate is critical for balancing the speed and stability of training:

- **High Learning Rate (e.g., 0.8):** Useful in deterministic environments where rewards are predictable. It allows for fast learning but may cause [Q-values to oscillate](#) or fail to converge in stochastic (random) environments.
- **Low Learning Rate (e.g., 0.1):** Preferred in stochastic environments. It ensures that the agent averages rewards over many experiences, leading to more stable [convergence to an optimal policy](#).
- **Decaying Learning Rate:** A common practice where
- starts high to allow rapid early exploration and [decreases over time](#) as the agent gains more knowledge. This provides a balance between learning speed and long-term stability

In Temporal Difference (TD) learning, the learning rate determines how much new information overrides old information, typically ranging from 0 to 1. A high

speeds up learning but may cause instability, while a low

ensures stable convergence at the cost of speed. The formula

updates values iteratively, with

scaling the prediction error.

### Key Aspects of TD Learning Rate (

):

- **Role:** The learning rate dictates the step size for updating value functions ( or ) based on the TD error, which is the difference between predicted and actual rewards.
- **Impact on Learning:** If , the agent learns nothing and keeps old estimates; if , the agent entirely forgets past estimates in favor of the new sample.
- **Convergence:** Using a constant
- (e.g., ) can be effective, but for guaranteed convergence in stochastic environments, the learning rate should decrease over time, usually following the Robbins-Monro conditions.
- In **TD** works alongside the trace decay parameter to update states based on a combination of immediate rewards and eligibility traces.

## Chapter 12.1 – Acting Under Uncertainty (Detailed Study Guide Summary)

### 1. Why Agents Must Handle Uncertainty

In real-world environments, intelligent agents must make decisions **without perfect information**. Uncertainty arises due to several factors:

- **Partial observability** – The agent cannot fully observe the environment.
- **Nondeterminism** – Actions may not always produce the same outcome.
- **Adversaries** – Other agents may influence the environment unpredictably.

Because of this, an agent often **cannot know its exact current state or the exact consequences of its actions**.

Example:

An **automated taxi** trying to reach the airport cannot be certain that its plan will succeed because unexpected events may occur:

- Traffic accidents
- Car breakdown
- Road closures
- Random events (e.g., extreme weather)

Therefore, agents must **reason and act under uncertainty** rather than relying on perfect logical certainty

## 2. Limitations of Pure Logical Reasoning Under Uncertainty

Earlier AI approaches used **belief states** and **contingency planning** to deal with uncertainty.

### Belief State

A **belief state** is the set of all possible world states consistent with the agent's knowledge.

Example:

If a robot cannot see clearly, it might consider multiple possible positions it could be in.

### Contingency Plans

Agents can create plans that include **branches for every possible observation** during execution.

However, this approach has serious limitations:

#### Problem 1: Explosion of Possible States

The agent must consider **every possible explanation** for its observations, including extremely unlikely ones.

Result:

- Very large belief states
- High computational complexity

#### Problem 2: Contingency Plans Become Huge

To account for all possibilities, contingency plans may grow extremely large.

Example:

A plan must account for every possible sensor reading or event.

#### Problem 3: No Guaranteed Success

Sometimes **no plan guarantees success**.

Example:

Even the best travel plan cannot guarantee arriving on time because random events might occur.

Thus, agents must be able to **compare plans that are not guaranteed to succeed**.

### 3. Rational Decisions Under Uncertainty

Instead of guaranteeing success, agents choose actions that **maximize expected performance**.

Example: Airport travel plans

| Plan  | Description             | Pros                      | Cons                          |
|-------|-------------------------|---------------------------|-------------------------------|
| A90   | Leave 90 minutes early  | Efficient                 | Slight risk of missing flight |
| A180  | Leave 180 minutes early | Lower risk                | Long wait at airport          |
| A1440 | Leave 24 hours early    | Almost guaranteed arrival | Extremely inefficient         |

A logical agent cannot prove any plan will succeed with certainty.

Instead, agents choose the **best plan based on probabilities and preferences**.

### 4. The Qualification Problem

The **qualification problem** refers to the difficulty of listing all conditions required for a rule or plan to succeed.

Example:

Logical rule:

If we leave 90 minutes early → we will reach the airport on time.

But many conditions could invalidate the rule:

- Car breakdown
- Traffic accident
- Road closure
- Weather delays
- Random disasters

Since **it is impossible to enumerate all possible exceptions**, purely logical reasoning fails in uncertain environments.

## 5. Example of Uncertainty: Medical Diagnosis

Consider diagnosing a patient with a **toothache**.

### Attempting Logical Rules

Rule 1:

Toothache  $\rightarrow$  Cavity

Problem:

Not all toothaches are caused by cavities.

Other causes include:

- Gum disease
- Abscess
- Infection
- Other dental issues

So the rule becomes:

Toothache  $\rightarrow$  Cavity OR GumDisease OR Abscess OR ...

But the list of possibilities can grow indefinitely.

### Reverse Rule Problem

Trying the reverse rule:

Cavity  $\rightarrow$  Toothache

This is also incorrect because:

- Many cavities do **not cause pain**.

Thus, logic cannot easily capture the uncertain relationship between symptoms and causes.

## 6. Why Logic Fails in Many Real Domains

Using strict logical rules fails because of three key limitations.

### 1. Laziness

It is impractical to list **all possible conditions** required for a rule to be true.

Example:

Listing every condition for when a cavity causes pain.

## 2. Theoretical Ignorance

Science does not have a **complete theory of many domains**.

Example:

Medicine still does not fully understand all causes of symptoms.

## 3. Practical Ignorance

Even if rules exist, we may **not have enough data** about a specific case.

Example:

A doctor may not have run all diagnostic tests.

Because of these issues, strict logical reasoning is insufficient.

Instead, we must represent **degrees of belief**.

## 7. Degrees of Belief and Probability

Instead of saying a statement is strictly **true or false**, we assign a **probability** representing our confidence.

Example:

If a patient has a toothache:

$$P(\text{Cavity} \mid \text{Toothache}) = 0.8$$

Meaning:

There is an **80% probability** the patient has a cavity.

This probability might come from:

- Statistical data
- Medical knowledge
- Previous observations

## 8. Probability Depends on Knowledge

An important idea:

**Probability reflects the agent's knowledge, not the true state of the world.**

In reality:

- The patient either **has a cavity or does not**.

But the agent **does not know which**, so it assigns probabilities.

### Example of Updating Beliefs

Initial knowledge:

$$P(\text{Cavity} \mid \text{Toothache}) = 0.8$$

After learning the patient has gum disease:

$$P(\text{Cavity} \mid \text{Toothache AND GumDisease}) = 0.4$$

After further tests:

$$P(\text{Cavity} \mid \text{all evidence}) \approx 0$$

Each probability reflects the **current knowledge state**.

These statements do **not contradict each other** because they are based on different information.

## 9. Rational Decision Making

Agents must choose actions based on:

1. **Probability of outcomes**
2. **Preferences for outcomes**

To represent preferences, AI uses **utility theory**.

## 10. Utility Theory

Utility theory assigns a **numerical value (utility)** to outcomes based on how desirable they are to the agent.

Key idea:

Higher utility = more preferred outcome.

Example: Airport travel

| Outcome                | Utility  |
|------------------------|----------|
| Catch flight on time   | High     |
| Arrive extremely early | Moderate |
| Miss flight            | Very low |

Utilities depend on the **agent's preferences**.

Example:

In chess:

- Winning has high utility
- Losing has low utility
- Drawing might be good or bad depending on the opponent.

Different agents may assign **different utilities** to the same outcome.

## 11. Decision Theory

Decision theory combines:

Decision Theory = Probability Theory + Utility Theory

This framework allows agents to choose actions under uncertainty.

## 12. Maximum Expected Utility (MEU)

The central principle of decision theory is:

### Maximum Expected Utility (MEU)

A rational agent should choose the action that **maximizes expected utility**.

Expected utility is calculated as:

Expected Utility =  $\Sigma$  (Probability of outcome  $\times$  Utility of outcome)

Thus the agent selects the action whose **average expected benefit is highest**.

## 13. Decision-Theoretic Agent

A **decision-theoretic agent** follows these steps:

1. Maintain a **belief state** about the world.
2. Update beliefs based on:
  - Percepts
  - Previous actions
3. Predict possible outcomes of each action.



4. Compute the probability of each outcome.
5. Calculate expected utility for each action.
6. Select the action with the **highest expected utility**.

### **Decision-Theoretic Agent Algorithm (Conceptual)**

1. Update belief state from percepts.
2. Determine outcome probabilities for each possible action.
3. Compute expected utility of each action.
4. Choose the action with the highest expected utility.

## **14. Importance of Decision Theory in AI**

Decision theory provides a **general framework for rational action under uncertainty**.

It is widely used in AI for:

- Medical diagnosis
- Autonomous vehicles
- Robotics
- Game playing
- Planning under uncertainty
- Economic decision making

### **Textbook readings 12.2 - 12.4**

## **12.2 Basic Probability Notation**

### **Main Idea**

To let an AI agent **represent uncertainty formally and reason with it**, we need a precise language for probability. Probability theory plays a role similar to logic, but instead of only saying whether a world is possible or impossible, it says **how likely each possible world is**.

### **12.2.1 What Probabilities Are About**

#### **Probability and Possible Worlds**

Like logic, probability theory is about **possible worlds**.

- In **logic**, a statement rules out worlds where it is false.
- In **probability theory**, we assign numbers to worlds to indicate **how likely** they are.

The set of all possible worlds is called the **sample space**.

### Sample Space

- Denoted by  $\Omega$  (uppercase omega)
- A single possible world is denoted by  $\omega$  (lowercase omega)

Properties of possible worlds in a sample space:

- **Mutually exclusive**: only one can be the true world
- **Exhaustive**: one of them must be true

### Example: Two Dice

If two distinguishable dice are rolled, there are **36 possible worlds**:

- (1,1), (1,2), ..., (6,6)

So the sample space contains all 36 outcomes.

### Probability Model

A **probability model** assigns a probability to each possible world.

This assignment must satisfy two **basic axioms**:

1. Each world's probability must be between 0 and 1  
 $0 \leq P(\omega) \leq 1$
2. The probabilities of all worlds in the sample space must add up to 1  
 $\sum_{\omega \in \Omega} P(\omega) = 1$

### Example

For two fair dice:

- Each world has probability **1/36**
- All 36 probabilities sum to 1

If the dice are loaded:

- Some worlds get larger probabilities
- Some get smaller probabilities
- But the total still must be 1

## Events

In probability, we usually ask about **sets of worlds**, not single worlds.

These sets are called **events**.

An **event** is essentially the same as a **proposition** in logic:

- It corresponds to all worlds where the proposition is true

## Example

Event: “the total is 11”

This includes the worlds:

- (5,6)
- (6,5)

## Key Definition

- The probability of a proposition is the sum of the probabilities of all worlds where it is true.

## Unconditional / Prior Probability

An **unconditional probability** (also called a **prior probability**) is the probability of a proposition **before considering any evidence**.

Examples:

- $P(\text{cavity}) = 0.2$

These reflect beliefs with no additional information.

## Conditional / Posterior Probability

A **conditional probability** tells us the probability of something **given evidence**.

Notation:

$$P(a|b)$$

read as:

- “the probability of a given b”

This is also called a **posterior probability**.

### Example

- $P(\text{doubles} \mid \text{Die1}=5) = P(\text{doubles} \mid \text{Die1}=5)$
- $P(\text{cavity} \mid \text{toothache}) = 0.6$   
 $P(\text{cavity} \mid \text{toothache}) = 0.6$

This is usually more useful than a prior, because in real decisions agents often already have evidence.

### Important Interpretation of Conditional Probability

A statement like:

$$P(\text{cavity} \mid \text{toothache}) = 0.6$$

does **not** mean:

- “Whenever toothache is true, conclude cavity with probability 0.6 no matter what.”

It means:

- “If toothache is true, and we have no additional relevant information, then our degree of belief in cavity is 0.6.”

If new evidence appears, the probability may change.

### Example

If the dentist later confirms no cavity, then:

$$P(\text{cavity} \mid \text{toothache} \wedge \neg \text{cavity}) = 0$$

So conditional probabilities depend on the **current knowledge state**.

### Definition of Conditional Probability

Conditional probability is defined by:

$$P(a \mid b) = \frac{P(a \wedge b)}{P(b)} \text{ when } P(b) > 0$$

Meaning:

- First restrict attention to worlds where bbb is true
- Then ask what fraction of those worlds also satisfy aaa

### Product Rule

Rearranging the conditional probability formula gives the **product rule**:

$$P(a \wedge b) = P(a | b)P(b) \quad P(a \wedge b) = P(a \wedge b)P(b | a \wedge b) = P(a \wedge b)P(b)P(a | b)$$

Interpretation:

For both aaa and bbb to happen:

- bbb must happen
- and aaa must happen given that bbb happened

This rule is used constantly in probability derivations.

## 12.2.2 The Language of Propositions in Probability Assertions

### Random Variables

In AI probability notation, propositions are represented using **random variables**.

A **random variable**:

- Has a name starting with a capital letter
- Maps each possible world to a value in its range

Examples:

- **Total**
- **Die1**
- **Weather**
- **Cavity**

### Range

The **range** of a variable is the set of values it can take.

Examples:

- Die1: {1, 2, 3, 4, 5, 6}
- Total: {2, 3, ..., 12}
- Weather: {sun, rain, cloud, snow}

## Boolean Random Variables

A Boolean random variable has the range:

- {true, false}

Example:

- **Doubles = true or false**

A Boolean variable may also use {0,1}; then it is said to have a **Bernoulli distribution**.

## Abbreviation Convention

- $A = \text{true}$   $A = \text{true}$  is abbreviated as **a**
- $A = \text{false}$   $A = \text{false}$  is abbreviated as  **$\neg a$**

So:

- cavity means  $\text{Cavity} = \text{true}$   $\text{Cavity} = \text{true}$
- $\neg \text{cavity}$  means  $\text{Cavity} = \text{false}$   $\text{Cavity} = \text{false}$

## Other Types of Values

Ranges do not have to be numeric.

Examples:

- Age = {juvenile, teen, adult}
- Weather = {sun, rain, cloud, snow}

Sometimes the value alone stands for the proposition:

- “sun” may stand for  $\text{Weather} = \text{sun}$   $\text{Weather} = \text{sun}$

Variables can also have:

- **finite ranges**
- **infinite discrete ranges** (like integers)
- **continuous ranges** (like real numbers)

If the range is ordered, inequalities can be used:

## Combining Propositions

Elementary propositions can be combined using logic:

- $\neg$  (not)
- $\wedge$  (and)
- $\vee$  (or)

## Probability Distributions

Sometimes we want the probabilities for **all values** of a variable.

Instead of listing each separately, we use vector notation.

Example:

$$P(\text{Weather}) = \langle 0.6, 0.1, 0.29, 0.01 \rangle$$

Assuming the value order is:

- sun, rain, cloud, snow

This is called a **probability distribution** for the variable.

If the variable has a finite discrete set of categories, this is a **categorical distribution**.

## Conditional Distributions

We can also write:

$$P(X|Y)$$

to mean the collection of all probabilities  $P(X=x_i | Y=y_j)$ .

So it represents an entire conditional distribution, not just one value.

## Continuous Variables and Density Functions

For continuous variables, we cannot list probabilities for every exact value, because there are infinitely many.

Instead, we use a **probability density function (pdf)**.

Example:

$$P(\text{NoonTemp}=x)=\text{Uniform}(x;18\text{C},26\text{C})P(\text{NoonTemp} = x) = \text{Uniform}(x; 18\text{C}, 26\text{C})P(\text{NoonTemp}=x)=\text{Uniform}(x;18\text{C},26\text{C})$$

This means the noon temperature is uniformly distributed between 18°C and 26°C.

## Important Difference

For continuous variables:

- The probability of an exact single value is **0**
- What matters is the probability over an **interval**

So:

- $P(\text{NoonTemp}=20.18\text{C})P(\text{NoonTemp} = 20.18\text{C})P(\text{NoonTemp}=20.18\text{C})$  is not a literal probability of that exact temperature
- It is a **density value**

A density is not the same thing as a probability.

## Why?

A single point has width 0, so its probability is 0.  
Probabilities only come from ranges/intervals.

## Joint Probability Distributions

We also need probabilities over **multiple variables together**.

Notation:

$$P(\text{Weather,Cavity})P(\text{Weather, Cavity})P(\text{Weather,Cavity})$$

This is the **joint probability distribution** over Weather and Cavity.

If Weather has 4 values and Cavity has 2 values, then the joint distribution is a **4 × 2 table**.

We can also mix values and variables:

- $P(\text{sun,Cavity})P(\text{sun, Cavity})P(\text{sun,Cavity})$  gives probabilities for cavity and no cavity, assuming sunny weather

## Full Joint Distribution

A **full joint probability distribution** gives the probabilities for **every combination of values of all random variables** in the domain.



Example:

If the variables are:

- Cavity
- Toothache
- Weather

then:

$P(\text{Cavity}, \text{Toothache}, \text{Weather})$

is the full joint distribution.

If the variables have sizes:

- $2 \times 2 \times 4$

then the joint table has:

- 16 entries

### **Why It Matters**

A full joint distribution completely determines the probability model.

Because every proposition corresponds to a set of worlds, and every world has a probability, the full joint distribution lets us compute the probability of **any proposition**.

### **Possible Worlds in Terms of Random Variables**

A possible world can be defined as:

- An assignment of values to all random variables in the domain

Example:

If the variables are Cavity, Toothache, and Weather:

- Cavity = true
- Toothache = false
- Weather = rain

That is one possible world.

This connects probability semantics directly to propositional logic.

### **Redundant Variables**

Sometimes one variable can be derived from others.

Example:

- Doubles is true exactly when  $\text{Die1} = \text{Die2}$

If we include Doubles in addition to Die1 and Die2:

- it seems to add more worlds
- but many of those combinations are logically impossible and therefore get probability 0

### 12.2.3 Probability Axioms and Their Reasonableness

#### Complement Rule

From the probability axioms we can derive:

$$P(\neg a) = 1 - P(a)$$

Meaning:

- The probability that  $a$  is false equals 1 minus the probability that  $a$  is true

This is intuitive because either  $a$  happens or it does not.

#### Inclusion–Exclusion Principle

For a disjunction:

$$P(a \vee b) = P(a) + P(b) - P(a \wedge b)$$

Why subtract the intersection?

Because if we just add  $P(a)$  and  $P(b)$ , then the cases where both happen get counted twice.

#### Kolmogorov's Axioms

These basic principles form the foundation of modern probability theory and are often called **Kolmogorov's axioms**.

They constrain rational degrees of belief.

#### Why Must Rational Beliefs Obey Probability Rules?

One key argument is due to **Bruno de Finetti**.

#### De Finetti's Betting Argument

If an agent states probabilities that violate the axioms, another agent can construct a set of bets that guarantees the first agent loses money no matter what happens.

This is called a **Dutch book**-style argument.

### Main Point

If your beliefs are inconsistent with probability theory, then your decisions can be exploited systematically.

So a rational agent should maintain probability assignments that obey the axioms.

### Bigger Philosophical Point

Probability is not just mathematically elegant—it is argued to be the **only coherent system** for numerical degrees of belief.

Other thinkers such as:

- Cox
- Carnap
- Jaynes

also argued that rational belief must behave like probability.

The book's practical point is:

- Beyond philosophy, probabilistic reasoning systems work well in real applications.

## 12.3 Inference Using Full Joint Distributions

### Main Idea

Probabilistic inference means:

- computing posterior probabilities for query variables
- given observed evidence

In this section, the **full joint distribution** is treated like a knowledge base from which any probability question can be answered.

### Example Domain

Three Boolean variables:

- Toothache

- Cavity
- Catch

The full joint distribution is a  $2 \times 2 \times 2$  table.

Each entry gives the probability of one complete world.

Because the entries sum to 1, they define a valid probability model.

### Computing the Probability of Any Proposition

To find the probability of any proposition:

1. Identify all worlds where the proposition is true
2. Add their probabilities

Example:

$P(\text{cavity} \vee \text{toothache})$

Add the probabilities of all worlds where cavity is true or toothache is true.

This directly uses the definition of probability as summing over worlds.

### Marginal Probability

A **marginal probability** is the probability of a subset of variables, ignoring others.

Example:

$P(\text{cavity})$

To compute it:

- sum over all values of Toothache and Catch while keeping Cavity fixed

This process is called **marginalization** or **summing out**.

### Marginalization Rule

For sets of variables  $YYY$  and  $ZZZ$ :

Meaning:

- To get the probability of  $YYY$ , sum over all possible values of the other variables  $ZZZ$

### Example

To compute  $P(\text{Cavity})$ :

- Add all joint entries where Cavity has the same value
- Ignore Toothache and Catch by summing over them

## Conditioning Rule

Using the product rule inside marginalization gives:

$$P(Y) = \sum_z P(Y|z)P(z) \quad P(Y) = \sum_z P(Y \mid z)P(z)$$

This is called **conditioning**.

It says:

- the probability of YYY can be computed as an average over conditional probabilities, weighted by the probability of each condition zzz

## Computing Conditional Probabilities from a Full Joint

To compute:

$$P(\text{cavity} \mid \text{toothache})$$

Use:

$$P(\text{cavity} \mid \text{toothache}) = \frac{P(\text{cavity} \wedge \text{toothache})}{P(\text{toothache})}$$

Then:

- find the numerator by summing all joint entries where both cavity and toothache are true
- find the denominator by summing all entries where toothache is true

In the example:

$$P(\text{cavity} \mid \text{toothache}) = 0.6$$

Similarly:

$$P(\neg \text{cavity} \mid \text{toothache}) = 0.4$$

These add to 1, as they should.

## Normalization

Often, when computing a conditional distribution, the denominator is just a **normalization constant**.

This constant ensures the final probabilities sum to 1 and values are in the range from [0-1].

The book uses  $\alpha$  to represent this normalization factor.

### Example Idea

Suppose after summing relevant terms you get:

- 0.12 for cavity
- 0.08 for  $\neg$ cavity

These are proportional to the correct probabilities, but they do not sum to 1.

Normalize by dividing each by:

$$0.12 + 0.08 = 0.20$$

Result:

- $0.12 / 0.20 = 0.6$
- $0.08 / 0.20 = 0.4$

So:

$$P(\text{Cavity} \mid \text{toothache}) = \langle 0.6, 0.4 \rangle$$

### Why Normalization Is Useful

It lets us:

- ignore the denominator until the end
- compute relative probabilities first
- turn them into a proper probability distribution afterward

### General Inference Formula

Suppose:

- XXX = query variable
- EEE = evidence variables
- eee = observed values of evidence
- YYY = remaining hidden/unobserved variables

Then:

$$P(X|e) = \alpha \sum_y P(X, e, y) \quad P(X \mid e) = \frac{1}{\alpha} \sum_y P(X, e, y)$$

Meaning:

1. Fix the query and evidence
2. Sum over all hidden variable assignments
3. Normalize

This is the basic full-joint inference procedure.

### Why Full Joint Inference Does Not Scale

This approach is theoretically correct but practically terrible for large domains.

If there are  $n$  Boolean variables:

- the full joint table has  $2^n$  entries
- inference takes  $O(2^n)$  time

For  $n=100$ :

- the table would have about  $2^{100} \approx 10^{30}$  entries

That is completely impractical.

### Main Limitation

The problem is not just storage and runtime.

It is also that we would need an enormous amount of data to estimate every probability separately.

### Key Conclusion

The full joint distribution is mainly a **theoretical foundation**, not a practical representation for large AI systems.

Later methods improve on this by exploiting structure, especially **independence**.

## 12.4 Independence

### Main Idea

Independence is one of the most important ideas in probability because it lets us **factor large joint distributions into smaller pieces**.

This dramatically reduces:

- representation size
- inference complexity

### Example: Adding Weather

Suppose we add a fourth variable, Weather, to the Toothache/Cavity/Catch domain.

Then the joint distribution becomes:

$P(\text{Toothache}, \text{Catch}, \text{Cavity}, \text{Weather})$   
 $P(\text{Toothache}, \text{Catch}, \text{Cavity}, \text{Weather})$

If Weather has 4 values, the table grows from:

- $2 \times 2 \times 2 = 8$
- $2 \times 2 \times 2 \times 4 = 32$ . That is already much larger.

### Independence Intuition

The book argues that dental problems and weather are unrelated.

So it is reasonable to say:

$P(\text{cloudy} | \text{toothache}, \text{catch}, \text{cavity}) = P(\text{cloud})$   
 $P(\text{cloud} | \text{toothache}, \text{catch}, \text{cavity}) = P(\text{cloud})$

Meaning:

Knowing the dental variables tells us nothing about the weather.

This is **independence**.

### Definition of Independence

Two propositions  $a$  and  $b$  are independent if any of these equivalent conditions hold:

$$P(a | b) = P(a) \quad P(a \wedge b) = P(a)P(b) \quad P(a \vee b) = P(a) + P(b) - P(a)P(b)$$

or

$$P(b | a) = P(b) \quad P(a \wedge b) = P(a)P(b) \quad P(a \vee b) = P(a) + P(b) - P(a)P(b)$$

or

$$P(a \wedge b) = P(a)P(b) \quad P(a \wedge b) = P(a)P(b) \quad P(a \vee b) = P(a) + P(b) - P(a)P(b)$$

These all mean:

- learning one does not change belief about the other



## Independence of Variables

For variables XXX and YYY:

$$P(X | Y) = P(X)P(X \text{ \textit{mid} } Y) = P(X)P(X | Y) = P(X)$$

or equivalently

$$P(X, Y) = P(X)P(Y)P(X, Y) = P(X)P(Y)P(X, Y) = P(X)P(Y)$$

## Factorization from Independence

If Weather is independent of the dental variables, then:

$$P(\text{Toothache, Catch, Cavity, Weather}) = P(\text{Toothache, Catch, Cavity}) P(\text{Weather}) \\ P(\text{Toothache, Catch, Cavity, Weather}) = P(\text{Toothache, Catch, Cavity}) \backslash, P(\text{Weather}) \\ P(\text{Toothache, Catch, Cavity, Weather}) = P(\text{Toothache, Catch, Cavity}) P(\text{Weather})$$

This means:

- instead of storing one 32-entry table
- we can store:
  - one 8-entry table for the dental variables
  - one 4-entry table for Weather

This is a major reduction.

## Another Example: Independent Coin Flips

If  $C_1, C_2, \dots, C_n$  are independent coin flips, then:

$$P(C_1, C_2, \dots, C_n) = \prod_i P(C_i) \\ P(C_1, C_2, \dots, C_n) = \prod_i P(C_i)$$

Without independence:

- joint distribution needs  $2^{2n}$  entries

With independence:

- only  $n$  small single-variable distributions are needed

This is a huge simplification.

## Why Independence Matters

Independence assertions are usually based on domain knowledge.

They are powerful because they:

- shrink the amount of information needed
- simplify inference
- allow large domains to be represented compactly

### **Limitation of Absolute Independence**

Unfortunately, full or absolute independence is rare in realistic domains.

Why?

Because if variables are connected even indirectly, strict independence often fails.

Example:

Within dentistry:

- diseases and symptoms are often highly interrelated

So even if Weather and Dentistry are independent, the dental variables themselves may not be.

Also, even one “independent subset” may still be large and complicated.

### **Important Conclusion**

Independence is useful, but not enough by itself for most realistic AI problems.

We need more subtle representations than just splitting domains into totally independent blocks.

This motivates later methods such as structured probabilistic models.

### **Key Concepts to Know for Exams**

#### **Vocabulary**

- **Sample space ( $\Omega$ ):** set of all possible worlds
- **Possible world ( $\omega$ ):** one complete outcome/state
- **Probability model:** assigns probabilities to possible worlds
- **Event:** set of worlds where a proposition is true
- **Prior / unconditional probability:** probability without evidence

- **Posterior / conditional probability:** probability given evidence
- **Random variable:** variable whose value depends on the world
- **Range:** set of values a variable can take
- **Bernoulli variable:** Boolean-valued random variable
- **Categorical distribution:** discrete distribution over categories
- **Probability density function (pdf):** continuous analogue of a discrete distribution
- **Joint distribution:** probabilities over combinations of variables
- **Full joint distribution:** probabilities for all combinations of all variables
- **Marginalization / summing out:** adding over hidden variables
- **Normalization:** rescaling numbers so they sum to 1
- **Independence:** one variable/proposition gives no information about another

## 12.5 Bayes' Rule and Its Use

### Main Idea

Bayes' rule is one of the most important formulas in probability and AI. It allows an agent to compute the probability of a **cause given an observed effect**, using information about the **effect given the cause**, together with prior probabilities.

This is extremely useful because, in real-world domains, we often know how causes produce effects, but what we really want is to reason backward from observed evidence to hidden causes.

### 12.5 Bayes' Rule

Starting from the product rule, we can write:

$$P(a \wedge b) = P(a | b)P(b)P(a \setminus \text{and } b) = P(a \setminus \text{mid } b)P(b)P(a \wedge b) = P(a | b)P(b)$$

and also:

$$P(a \wedge b) = P(b | a)P(a)P(a \setminus \text{and } b) = P(b \setminus \text{mid } a)P(a)P(a \wedge b) = P(b | a)P(a)$$

Setting the right-hand sides equal and dividing by  $P(a)P(a)$  gives **Bayes' rule**:

$$P(b | a) = P(a | b)P(b)P(a)P(b \setminus \text{mid } a) = \frac{P(a \setminus \text{mid } b)P(b)}{P(a)} P(b | a) = P(a)P(a | b)P(b)$$

This is the basic form of Bayes' rule.

## General Multivariable Form

For random variables:

$$P(Y|X) = \frac{P(X|Y)P(Y)P(X)}{P(X)P(Y)}$$

And with additional background evidence  $e$ :

$$P(Y|X,e) = \frac{P(X|Y,e)P(Y|e)P(X|e)}{P(X|e)P(Y|e)}$$

## Why Bayes' Rule Is Useful

At first, Bayes' rule may not seem helpful, because it computes one probability using three others. But it becomes useful in cases where:

- we **do know** the prior probability of a cause,
- we **do know** how likely an effect is if the cause is present,
- and we want to infer the hidden cause from observed evidence.

This is common in diagnosis.

## Cause-to-Effect vs. Effect-to-Cause

Bayes' rule is especially helpful when we know probabilities in the **causal direction** but want probabilities in the **diagnostic direction**.

### Causal Direction

$$P(\text{effect}|\text{cause})$$

This describes how likely an effect is when the cause is present.

### Diagnostic Direction

$$P(\text{cause}|\text{effect})$$

This describes how likely a hidden cause is after observing an effect.

In medicine, doctors often know:

- how diseases cause symptoms

but need to infer:

- how likely a disease is given symptoms

Bayes' rule connects these two directions.

## Example: Meningitis and Stiff Neck

Suppose:

- $mmm$  = patient has meningitis
- $sss$  = patient has a stiff neck

Given:

- $P(s|m)=0.7$   
 $P(s \mid m)=0.7$   
 $P(s|m)=0.7$
- $P(m)=1/50000$   
 $P(m)=1/50000$   
 $P(m)=1/50000$
- $P(s)=0.01$   
 $P(s)=0.01$   
 $P(s)=0.01$

Using Bayes' rule:

$$P(m|s)=\frac{P(s|m)P(m)}{P(s)} \quad P(m \mid s)=\frac{P(s \mid m)P(m)}{P(s)}$$

Substituting:

$$P(m|s)=0.7 \cdot (1/50000) / 0.01 = 0.0014$$
$$P(m \mid s)=\frac{0.7 \cdot (1/50000)}{0.01} = 0.0014$$

So:

- Even though meningitis causes stiff neck 70% of the time,
- the probability that a patient with a stiff neck has meningitis is only **0.14%**

### Why So Small?

Because stiff necks are much more common from other causes than meningitis is.

This demonstrates one of the most important lessons of Bayes' rule:

### Important Lesson

A strong causal connection does **not** necessarily imply a high diagnostic probability.

Even if:

- $P(\text{effect}|\text{cause})$  is high,

it may still be that:

- $P(\text{cause}|\text{effect})$  is low,

if the cause itself is very rare.

This is the **base-rate effect** in action.

### Bayes' Rule with Normalization

Instead of explicitly computing  $P(X)P(X)P(X)$  in the denominator, we can compute unnormalized values and then normalize them.

General form:

$$P(Y | X) = \alpha P(X | Y)P(Y)P(Y \mid X) = \alpha P(X \mid Y)P(Y)P(Y | X) = \alpha P(X | Y)P(Y)$$

where:

- $\alpha$  is the normalization constant that makes the probabilities sum to 1.

This is often easier in practice.

### Example Idea

To compute posterior probabilities over all possible causes:

1. For each cause value, compute:
  - likelihood  $\times$  prior
2. Normalize across all cause values

### Why Use Causal Knowledge Instead of Direct Diagnostic Statistics?

The text emphasizes that **causal knowledge is often more robust** than directly estimated diagnostic probabilities.

### Example

Suppose there is suddenly an epidemic of meningitis.

Then:

- $P(m)P(m)P(m)$  goes up

A doctor using Bayes' rule can immediately update:

- $P(m | s)P(m \mid s)P(m | s)$

because:

- $P(s|m)P(s \mid m)P(s|m)$  remains the same
- the disease still causes stiff neck the same way as before

But a doctor who memorized only:

- $P(m|s)P(m \mid s)P(m|s)$

from old statistics may not know how to update it.

### Key Point

Causal probabilities are often more stable and reusable than diagnostic probabilities.

That is one major reason Bayesian reasoning is so valuable.

## 12.5.2 Using Bayes' Rule with Multiple Pieces of Evidence

### Problem

Bayes' rule works nicely with one piece of evidence, but what if we have several pieces of evidence?

Example:

- toothache
- the dentist's probe catches in the tooth

We want:

$$P(\text{Cavity} | \text{toothache} \wedge \text{catch})P(\text{Cavity} \mid \text{toothache} \wedge \text{catch})P(\text{Cavity} | \text{toothache} \wedge \text{catch})$$

From the full joint distribution, this can be computed directly. But that does not scale.

Using Bayes' rule:

$$P(\text{Cavity} | \text{toothache} \wedge \text{catch}) = \alpha P(\text{toothache} \wedge \text{catch} | \text{Cavity})P(\text{Cavity})P(\text{Cavity} \mid \text{toothache} \wedge \text{catch}) = \alpha P(\text{toothache} \wedge \text{catch} \mid \text{Cavity})P(\text{Cavity})P(\text{Cavity} | \text{toothache} \wedge \text{catch}) = \alpha P(\text{toothache} \wedge \text{catch} | \text{Cavity})P(\text{Cavity})$$

The problem is that this requires knowing:

$$P(\text{toothache} \wedge \text{catch} | \text{Cavity})P(\text{toothache} \wedge \text{catch} \mid \text{Cavity})P(\text{toothache} \wedge \text{catch} | \text{Cavity})$$

And with many evidence variables, we would need probabilities for exponentially many combinations of symptoms. That becomes infeasible.

So Bayes' rule alone is not enough to solve the scaling problem.

## Solution: Conditional Independence

The key observation is:

- Toothache and Catch are **not** independent outright
- but they **are conditionally independent given Cavity**

Why?

Because:

- Both Toothaches and Catches are caused by the cavity
- but neither directly affects the other

So once we know whether there is a cavity, learning about toothache gives no extra information about catch, and vice versa.

This is written as:

$$\begin{aligned} P(\text{toothache} \wedge \text{catch} \mid \text{Cavity}) &= P(\text{toothache} \mid \text{Cavity})P(\text{catch} \mid \text{Cavity})P(\text{toothache} \wedge \text{catch} \mid \text{Cavity}) \\ &= P(\text{toothache} \mid \text{Cavity})P(\text{catch} \mid \text{Cavity})P(\text{toothache} \wedge \text{catch} \mid \text{Cavity}) \\ P(\text{toothache} \wedge \text{catch} \mid \text{Cavity}) &= P(\text{toothache} \mid \text{Cavity})P(\text{catch} \mid \text{Cavity}) \end{aligned}$$

This is **conditional independence**.

Substituting into Bayes' rule gives:

$$\begin{aligned} P(\text{Cavity} \mid \text{toothache} \wedge \text{catch}) &= \alpha P(\text{toothache} \mid \text{Cavity})P(\text{catch} \mid \text{Cavity})P(\text{Cavity})P(\text{Cavity} \mid \text{toothache} \wedge \text{catch}) \\ &= \alpha P(\text{toothache} \mid \text{Cavity})P(\text{catch} \mid \text{Cavity})P(\text{Cavity})P(\text{Cavity} \mid \text{toothache} \wedge \text{catch}) \\ P(\text{Cavity} \mid \text{toothache} \wedge \text{catch}) &= \alpha P(\text{toothache} \mid \text{Cavity})P(\text{catch} \mid \text{Cavity})P(\text{Cavity}) \end{aligned}$$

Now we only need:

- the prior  $P(\text{Cavity})$
- each symptom likelihood separately

This is much more scalable.

## Conditional Independence: Definition

Two variables  $X$  and  $Y$  are conditionally independent given  $Z$  if:

$$P(X, Y \mid Z) = P(X \mid Z)P(Y \mid Z)P(X, Y \mid Z) = P(X \mid Z)P(Y \mid Z)P(X, Y \mid Z) = P(X \mid Z)P(Y \mid Z)$$

Equivalent forms:

$$P(X \mid Y, Z) = P(X \mid Z)P(X \mid Y, Z) = P(X \mid Z)P(X \mid Y, Z) = P(X \mid Z)$$



and

$$P(Y | X, Z) = P(Y | Z)P(Y \mid X, Z) = P(Y \mid Z)P(Y | X, Z) = P(Y | Z)$$

Meaning:

- once Z is known, X tells us nothing more about Y, and vice versa

## Why Conditional Independence Matters

Conditional independence allows us to **factor a large joint distribution into smaller pieces**.

Example:

$$\begin{aligned} P(\text{Toothache}, \text{Catch}, \text{Cavity}) &= P(\text{Toothache} | \text{Cavity})P(\text{Catch} | \text{Cavity})P(\text{Cavity}) \\ P(\text{Toothache}, \text{Catch}, \text{Cavity}) &= P(\text{Toothache} \mid \text{Cavity})P(\text{Catch} \mid \text{Cavity})P(\text{Cavity}) \end{aligned}$$

Instead of storing one large joint table, we can store several smaller distributions.

## Efficiency Benefit

For just 3 variables, the reduction is modest.

But if there are many symptoms, the gain becomes enormous.

If  $n$  symptoms are all conditionally independent given a cause:

- full joint representation grows like  $O(2^n)$
- factored representation grows like  $O(n)$

This is a huge improvement.

## Separation Intuition

The book says that **Cavity separates Toothache and Catch**.

Meaning:

- cavity is the common direct cause
- once cavity is known, toothache and catch become unrelated

This idea of variables “separating” others is central later in Bayesian networks.

## 12.6 Naive Bayes Models

### Main Idea

A **naive Bayes model** is a probabilistic model where:

- there is one main **cause** variable
- many **effect** variables
- all the effect variables are assumed to be conditionally independent given the cause

This is called “naive” because the independence assumption is often not perfectly true in real life.

### **Naive Bayes Joint Distribution**

The full joint distribution in a naive Bayes model is:

$$P(\text{Cause}, \text{Effect}_1, \dots, \text{Effect}_n) = P(\text{Cause}) \prod_i P(\text{Effect}_i | \text{Cause})$$
$$P(\text{Cause}) \prod_i P(\text{Effect}_i | \text{Cause}) = P(\text{Cause}) \prod_i P(\text{Effect}_i | \text{Cause})$$

So instead of a huge joint table, the model only needs:

- one prior over the cause
- one conditional distribution for each effect

This makes the model compact and efficient.

### **Inference in Naive Bayes**

Suppose some effects are observed, written as  $E = e$ .

### **Interpretation in Words**

For each possible cause:

1. Start with the prior probability of the cause
2. Multiply by the probability of each observed effect given that cause
3. Normalize the results

### **Why Naive Bayes Is Efficient**

The runtime is:

- linear in the number of observed effects

It does **not** depend on the number of unobserved effects.

This is powerful in large domains such as medicine or language classification.

## 12.6.1 Text Classification with Naive Bayes

### Main Idea

Naive Bayes is often used in **text classification**.

Task:

- given a document or sentence
- decide which category it belongs to

Examples of categories:

- news
- sports
- business
- weather
- entertainment

### Variables in Text Classification

- **Cause variable:** Category
- **Effect variables:** whether certain words appear in the document

For each keyword  $w_i$ , we may define:

- $\text{HasWord}_i$

So the model assumes:

- the category causes certain words to be more or less likely to appear

### Training the Model

The probabilities are estimated from previously seen labeled documents.

### Prior Probability of a Category

$P(\text{Category}=c)$

is estimated as:

- fraction of documents in category  $c$

Example:

- if 9% of documents are weather articles, then  
 $P(\text{Category}=\text{weather})=0.09$

## Word Likelihoods

$P(\text{HasWord}_i=\text{true} \mid \text{Category}=c)$

is estimated as:

- fraction of category-ccc documents containing word iii

Example:

- if 37% of business articles contain the word “stocks,” then  
 $P(\text{HasWord}=\text{stocks} \mid \text{Category}=\text{business})=0.37$

## Classifying a New Document

To classify a new document:

1. Observe which words appear
2. For each possible category:
  - compute prior  $\times$  product of word likelihoods
3. Normalize
4. Choose the category with highest posterior probability

## Independence Assumption in Text

Naive Bayes assumes words occur independently given the category.

This is clearly not literally true.

Example:

- “first” and “quarter” are not independent in business articles
- the phrase “first quarter” occurs together more often than the model predicts

So the model’s independence assumption is violated.

## Consequence of the Wrong Independence Assumption

Because the independence assumption is not fully true:

- posterior probabilities are often too extreme
- the model tends to be **overconfident**

That means:

- probabilities may be closer to 0 or 1 than they should be

However:

### **Important Practical Point**

Even if the probability values are imperfect, the **ranking of categories** is often very good.

This is why naive Bayes works surprisingly well in practice.

### **Applications of Naive Bayes**

Naive Bayes is widely used in:

- document classification
- language identification
- document retrieval
- spam filtering
- other classification problems

For high-stakes decisions where the exact posterior probabilities matter a lot, more sophisticated models may be preferred.

## **12.7 The Wumpus World Revisited**

### **Main Idea**

Probability can improve decision making in the Wumpus world beyond what logic alone can do.

A purely logical agent may know only that several squares **might** contain pits, without knowing which one is more dangerous. A probabilistic agent can compute which squares are **more likely** to contain pits and choose more intelligently.

### **Why Uncertainty Exists in Wumpus World**

The Wumpus world is partially observable.

The agent does not directly see pits. Instead, it perceives indirect evidence:

- breezes near pits

So the agent must infer hidden dangers from incomplete sensor data.

### Example Situation

Suppose the agent has discovered breezes in:

- [1,2]
- [2,1]

Then squares such as:

- [1,3]
- [2,2]
- [3,1]

may contain pits.

Logic alone may conclude:

- none of these can be ruled out with certainty

So the logical agent may be forced to choose randomly.

Probability allows the agent to do better.

### Random Variables in the Wumpus Model

We define Boolean pit variables:

- $P_{ij}$ : true iff square  $[i,j]$  contains a pit

We also define Boolean breeze variables:

- $B_{ij}$ : true iff square  $[i,j]$  is breezy

Only observed breeze squares need breeze variables.

### Full Joint Distribution for Wumpus

The full joint distribution includes:

- all pit variables
- all observed breeze variables

Using the product rule:

$$P(\text{Pits}, \text{Breezes}) = P(\text{Breezes} \mid \text{Pits})P(\text{Pits})P(\text{Pits}, \text{Breezes}) = P(\text{Breezes} \mid \text{Pits})P(\text{Pits})P(\text{Pits}, \text{Breezes}) = P(\text{Breezes} \mid \text{Pits})P(\text{Pits})$$

## Meaning of Each Part

### 1. $P(\text{Breezes} \mid \text{Pits})P(\text{Breezes} \mid \text{Pits})P(\text{Breezes} \mid \text{Pits})$

This is determined by the rules of the environment:

- breezes occur exactly in squares adjacent to pits

So:

- probability is 1 if the breeze pattern matches the pit pattern
- probability is 0 otherwise

### 2. $P(\text{Pits})P(\text{Pits})P(\text{Pits})$

Each square independently contains a pit with probability 0.2.

So:

$$P(P_{1,1}, \dots, P_{4,4}) = \prod_{i,j} P(P_{ij})P(P_{1,1}, \dots, P_{4,4}) = \prod_{i,j} P(P_{ij})P(P_{1,1}, \dots, P_{4,4}) = \prod_{i,j} P(P_{ij})$$

If a configuration has  $n$  pits, its probability is:

$$0.2^n \cdot 0.8^{16-n}$$

## Query of Interest

We want probabilities like:

$$P(P_{1,3} \mid \text{known}, b)P(P_{1,3} \mid \text{known}, b)P(P_{1,3} \mid \text{known}, b)$$

Meaning:

- probability that square [1,3] has a pit
- given known safe squares and observed breeze facts

## Direct Inference Is Exponential

Using the standard full-joint approach, we must sum over all unknown pit configurations.

In the example, there are 12 unknown squares, so:

- $2^{12} = 4096$  terms

This grows exponentially and becomes infeasible.

## **Frontier vs. Other Squares**

To simplify, the book divides the unknown pit variables into two groups:

### **Frontier**

Unknown squares adjacent to visited squares

These directly affect the observed breezes.

### **Other**

Unknown squares far away from the visited area

These do not directly affect the observed breezes.

This distinction is crucial.

## **Key Conditional Independence Insight**

The observed breezes are conditionally independent of the distant “Other” pit variables once we know:

- the known visited squares
- the query square
- the frontier variables

In simpler terms:

- distant pits do not matter for current breeze observations once the nearby relevant pit variables are fixed

This lets the “Other” variables disappear from the computation.

## **Computational Benefit**

After applying conditional independence and algebraic simplification:

- the large sum over all unknown squares collapses to a sum over only the frontier variables

In the example:

- instead of summing over 4096 terms



- we only sum over 4 combinations of the frontier variables

This makes the computation tractable.

### **Result of the Wumpus Example**

The final computed probabilities are approximately:

- $[1,3][1,3][1,3]$  contains a pit with probability **0.31**
- by symmetry,  $[3,1][3,1][3,1]$  also has probability **0.31**
- $[2,2][2,2][2,2]$  contains a pit with probability **0.86**

### **Decision Consequence**

The agent should strongly avoid square  $[2,2]$ .

This is something logic alone could not tell us.

Logic could only say:

- it is unknown whether  $[2,2]$  contains a pit

Probability tells us:

- $[2,2]$  is much more dangerous than the other candidate squares

This is the key advantage of probabilistic reasoning.

### **Overall Meaning of the Wumpus Example**

The Wumpus example shows that:

1. complex uncertain domains can be represented formally with probability
2. full-joint inference is conceptually straightforward
3. conditional independence is what makes the computation practical
4. probabilistic agents can make better decisions than purely logical agents when certainty is impossible

\*\*\*\*\*Refer to ED discussion post “N-gram Language Models: Theory and Foundations” and “How Your N-Gram Language Model Learns and Generates Text”

For textbook readings on ngrams as well as the **solutions to the ngrams practice problems** found in my lecture notes before preceding to module 10.\*\*\*\*\* Other solutions to practice exam problems for exam 2 thread found in CHAT GPT chat called: “AI course textbook readings summarization” \*\*\*

## **Textbook readings 13.1**

### **1. Why We Need Something Better Than the Full Joint Distribution**

#### **Problem with Full Joint Distributions**

The full joint probability distribution is theoretically powerful because it can answer **any question** about the domain.

But it has major practical limitations:

#### **1. Size Explosion**

As the number of random variables grows, the number of possible worlds grows exponentially.

That means the full joint distribution quickly becomes too large to store or reason with efficiently.

#### **2. Unnatural Knowledge Representation**

Even if it were small enough, specifying probabilities for all possible worlds individually is:

- awkward
- unnatural
- tedious
- hard for humans to do directly

Domain experts usually do **not** think in terms of:

- “Let me assign a probability to every complete world.”

Instead, they think in terms of:

- which variables directly influence others
- what local causal or dependency relationships exist

This motivates the need for a more structured representation.

### **2. Role of Independence and Conditional Independence**

In Chapter 12, the book showed that **independence** and **conditional independence** can drastically reduce the number of probabilities that need to be specified.

This is the key idea behind Bayesian networks.

Instead of representing every world directly, we represent:

- which variables influence which others
- and which variables are independent once certain other variables are known

This allows the full joint distribution to be expressed in a factored, much smaller form.

### 3. What a Bayesian Network Is

A **Bayesian network** is a **directed graph** with probability information attached to each node.

It is designed to represent uncertain knowledge in a domain in a compact way.

A Bayesian network has three main components:

#### 1. Nodes

Each node corresponds to a **random variable**.

These variables may be:

- **discrete**
- or **continuous**

#### 2. Directed Arrows (Links)

If there is an arrow from node XXX to node YYY, then:

- XXX is a **parent** of YYY

The graph must have **no directed cycles**, so it is a:

#### Directed Acyclic Graph (DAG)

This means:

- you can follow arrows forward
- but you can never come back to the same node by following arrow directions

#### 3. Local Probability Information

Each node  $X_i$  has associated probability information of the form:

$$\theta(X_i | \text{Parents}(X_i))$$

This means:

- the probability distribution of the node
- conditioned on the values of its parent nodes

So each node stores only **local conditional probabilities**, not a full giant table over all variables.

## 4. What the Network Topology Means

The **topology** of the network means:

- the set of nodes
- and the arrows between them

This structure encodes **conditional independence relationships** in the domain.

So the graph is not just a picture — it has meaning:

- arrows indicate direct probabilistic influence
- absence of arrows often indicates conditional independence

### Intuitive Interpretation of an Arrow

The book says the intuitive meaning of an arrow is usually:

- $XX$  has a **direct influence** on  $YY$

This often suggests a **causal interpretation**:

- causes are parents
- effects are children

Although later sections discuss causality more carefully, for now the important intuition is:

### Typical design principle:

Put **causes as parents** of their **effects**

This usually makes the network easier to understand and easier to specify.

## 5. Why Bayesian Networks Are Natural for Domain Experts

The book emphasizes that it is often much easier for a domain expert to answer:

- “What directly affects what?”

than to answer:

- “What is the probability of every possible combination of all variables?”

This is one of the major advantages of Bayesian networks.

A domain expert can often identify:

- direct causes
- direct effects
- missing direct relationships

Once the structure is chosen, only the local conditional probabilities need to be specified.

This is usually far less work than building a full joint distribution.

## 6. Bayesian Networks Define the Full Joint Distribution

A very important idea:

Even though a Bayesian network is compact, it still defines a **complete full joint distribution** over all variables.

So Bayesian networks are not an approximation in the sense of “throwing away” the full probabilistic meaning.

Instead, they are a **structured factorization** of the same probabilistic information.

The full joint distribution is determined by:

- the network topology
- the local probability information at each node

So a Bayesian network is a compact representation of a full probability model.

## 7. First Example: Toothache, Cavity, Catch, and Weather

The section revisits the earlier example involving these four variables:

- **Toothache**
- **Cavity**
- **Catch**
- **Weather**

### 1. Weather is independent of the other three variables

This means:

- Weather has nothing to do with dental conditions in this example

### 2. Toothache and Catch are conditionally independent given Cavity

This means:

- once we know whether there is a cavity,
- knowing about Toothache gives no extra information about Catch, and vice versa

## How the Bayesian Network Represents These Relationships

The graph includes:

- a node for **Cavity**
- arrows from **Cavity** to **Toothache**
- arrows from **Cavity** to **Catch**

- **Weather** is disconnected from the dental variables

This represents:

### **Weather Independence**

Because Weather is not connected to the other variables, it is independent of them.

### **Conditional Independence of Toothache and Catch**

There is **no arrow directly between Toothache and Catch**.

That absence of a direct link reflects the fact that:

- there is no direct causal relationship between them
- their connection comes only through Cavity

So:

- Cavity directly influences Toothache
- Cavity directly influences Catch
- Toothache and Catch do not directly influence each other

### **Important Interpretation**

This example shows that in a Bayesian network:

- **presence of arrows** suggests direct dependency
- **absence of arrows** can represent conditional independence

That is a central idea of Bayesian networks.

## **8. Second Example: Burglary Alarm Network**

The section then introduces a more realistic and famous example:

Variables:

- **Burglary**
- **Earthquake**
- **Alarm**
- **John Calls**
- **Mary Calls**

This is one of the classic introductory Bayesian network examples.

### **Story of the Domain**

You have a burglar alarm at home.

- It usually detects burglaries

- but sometimes an earthquake also triggers it

Two neighbors may call when they hear the alarm:

### **John**

- almost always calls when he hears the alarm
- but sometimes mistakenly calls because he confuses another sound (such as a phone ringing) with the alarm

### **Mary**

- likes loud music
- sometimes misses the alarm entirely

The goal is:

- given whether John or Mary called,
- estimate the probability that a burglary occurred

## **9. Structure of the Burglary Bayesian Network**

The Bayesian network contains these arrows:

- **Burglary** → **Alarm**
- **Earthquake** → **Alarm**
- **Alarm** → **John Calls**
- **Alarm** → **Mary Calls**

This structure expresses the following assumptions:

### **Burglary and Earthquake directly affect Alarm**

Both can cause the alarm to go off.

### **John and Mary depend only on the Alarm**

Whether John calls depends only on whether the alarm sounded.

Whether Mary calls also depends only on whether the alarm sounded.

### **What the Network Assumes**

The network encodes several assumptions:

- John and Mary do **not** directly observe burglaries
- John and Mary do **not** directly notice minor earthquakes
- John and Mary do **not** consult each other before calling

Thus, the only thing influencing their calls is the alarm itself.

This is a good example of how the graph structure represents independence assumptions.

## 10. Conditional Probability Tables (CPTs)

Each node in the Bayesian network is annotated with local probability information.

For discrete variables, this information is represented using a:

### Conditional Probability Table (CPT)

A CPT lists the conditional probabilities for a variable given each possible combination of its parents' values.

#### What a CPT Row Represents

Each row in a CPT corresponds to a:

#### Conditioning case

A conditioning case is:

- one specific possible assignment of values to the parent nodes

So if a node has parents  $BB$  and  $EE$ , then the conditioning cases are all combinations of values of  $BB$  and  $EE$ .

Each row specifies the probability distribution of the child node under that specific parent assignment.

#### Why Each Row Must Sum to 1

For a given conditioning case, the child variable must take one of its possible values.

So each row must sum to 1 because it represents a complete probability distribution for the node in that parent context.

#### Boolean Variable Simplification

For Boolean variables:

- if  $P(X=\text{true})=p$ ,  $P(X=\text{true})=p$ ,
- then automatically  $P(X=\text{false})=1-p$ ,  $P(X=\text{false})=1-p$

So often only the probability of **true** is shown in the CPT.

This saves space.

## 11. Number of Probabilities Needed in a CPT



For a Boolean variable with  $k$  Boolean parents:

- there are  $2^k$  possible conditioning cases
- each conditioning case needs one independently specifiable number (because false is just  $1-p$ )

So such a node requires:

$2^k$

independent probability values.

### Node with No Parents

If a node has no parents, then its CPT has only one row.

That row simply gives the **prior probability distribution** of the variable.

Example:

- Burglary has no parents, so its CPT gives  $P(\text{Burglary})$
- Earthquake has no parents, so its CPT gives  $P(\text{Earthquake})$

## 12. Example Probabilities in the Burglary Network

The network includes prior probabilities such as:

- $P(\text{Burglary}=\text{true})=0.001$
- $P(\text{Earthquake}=\text{true})=0.002$

This means:

- burglaries are rare
- earthquakes are also rare

The Alarm node depends on both Burglary and Earthquake, so its CPT has rows for each combination of:

- burglary true/false
- earthquake true/false

JohnCalls depends only on Alarm, so its CPT is based only on:

- Alarm = true
- Alarm = false

MaryCalls also depends only on Alarm.

## 13. Missing Variables and Hidden Complexity

The book points out that the network does **not** explicitly include variables such as:

- whether Mary is currently listening to loud music
- whether a phone ringing confused John
- humidity
- power failures
- battery problems
- cut wires
- many other real-world details

These factors are **not modeled explicitly as separate nodes**.

Instead, their effects are **summarized probabilistically** in the CPTs.

### **Why This Matters**

This shows how Bayesian networks deal with complexity by summarizing many unknown or omitted factors into local probabilistic relationships.

The book connects this to two earlier ideas:

#### **Laziness**

It would take too much effort to explicitly model every possible factor.

#### **Ignorance**

We often do not know enough to model all those factors precisely anyway.

So Bayesian networks allow us to represent useful probabilistic knowledge without modeling every detail of reality.

## **14. Bayesian Networks as Compact Summaries of Complex Worlds**

The section emphasizes that a small Bayesian network can summarize an enormous number of possible real-world circumstances.

For example, the uncertainty in whether the alarm goes off may implicitly summarize countless possible causes of alarm failure, such as:

- dead battery
- power outage
- broken wiring
- mechanical failure
- unusual environmental conditions
- and so on

Similarly, the uncertainty in whether John or Mary call may summarize countless reasons they might fail to report the alarm, such as:

- not being home
- being distracted
- being on vacation
- helicopter noise
- lunch break
- etc.

## **Big Point**

A Bayesian network lets a **small agent cope with a very large world**, at least approximately.

This is one of the core reasons Bayesian networks are so useful in AI.

They do not require modeling every tiny detail directly; instead, they summarize uncertainty at the right level of abstraction.

## **15. Why Bayesian Networks Are Powerful**

This section introduces Bayesian networks as powerful because they combine:

- a **graphical structure**
- **independence assumptions**
- **local probability tables**

to produce a compact, interpretable, and efficient representation of uncertain knowledge.

They are powerful because:

### **1. They Can Represent Essentially Any Full Joint Distribution**

Bayesian networks are general enough to represent very rich probabilistic domains.

### **2. They Are Often Much More Compact**

Because they exploit conditional independence, they may require far fewer numbers than a full joint table.

### **3. They Match Human Intuition**

Humans tend to think in terms of:

- what affects what
- what causes what
- what is directly related vs indirectly related

Bayesian networks align naturally with that way of thinking.

## **High-Yield Concepts to Know**

## Bayesian Network

A **Bayesian network** is a directed acyclic graph where:

- nodes are random variables
- arrows represent direct probabilistic influence
- each node has a conditional probability distribution given its parents

### Directed Acyclic Graph (DAG)

A **DAG** is a directed graph with no directed cycles.

This is required for Bayesian networks.

### Parent

If there is an arrow from  $XX$  to  $YY$ , then:

- $XX$  is a **parent** of  $YY$

### Topology

The **topology** of a Bayesian network means:

- the structure of nodes and arrows

It encodes dependency and conditional independence relationships.

### Conditional Probability Table (CPT)

A **CPT** is a table giving:

- the probability distribution of a node
- for each possible assignment of values to its parents

### Conditioning Case

A **conditioning case** is:

- one possible combination of values of the parent variables

Each row in a CPT corresponds to one conditioning case.

## Most Important Main Points from This Section

### 1. Full joint distributions are too large and unnatural

Bayesian networks are introduced because full joint tables do not scale and are hard to specify directly.

## 2. Bayesian networks exploit conditional independence

They compactly represent domains by encoding dependency structure rather than every possible world explicitly.

## 3. Nodes are variables and arrows represent direct influence

A Bayesian network is a DAG whose structure encodes which variables directly affect which others.

## 4. Local probabilities are enough

Once the topology is specified, we only need local conditional probabilities for each node given its parents.

## 5. The full joint distribution is still defined

A Bayesian network compactly specifies a complete probability model over all variables.

## 6. Missing details are absorbed into uncertainty

We do not have to model every hidden factor explicitly; CPTs summarize many omitted influences.

## Textbook readings 13.2

### 1. Semantics of a Bayesian Network

#### Syntax vs. Semantics

A Bayes net's **syntax** is:

- a DAG
- with local probability information attached to nodes

Its **semantics** is:

- the meaning of that structure in terms of a full joint probability distribution

So the main semantic question is:

**How does a Bayes net determine the joint probability over all variables?**

#### Joint Distribution from Local Factors

Suppose the network contains variables:

$X_1, X_2, \dots, X_n$

A complete assignment is:

$$P(x_1, x_2, \dots, x_n)$$

The semantics of a Bayesian network says that each joint assignment is computed as:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n \theta(x_i | \text{parents}(X_i))$$

Meaning:

- take each variable in the assignment
- look at the value of that variable and the values of its parents
- extract the corresponding local conditional probability
- multiply all of them together

## Intuition

This formula says that the probability of a complete world can be built from:

- the probability of each node,
- conditioned only on its parents

So Bayesian networks replace one large full joint distribution with a product of many small local distributions.

This is the central semantic fact of Bayes nets.

## 2. Example: Burglary Network Joint Probability

The section uses the burglary network as an example.

Suppose we want the probability of the event:

- the alarm sounds
- there is no burglary
- there is no earthquake
- John calls
- Mary calls

That is:

$$P(j, m, a, \neg b, \neg e)$$

Using the Bayesian network semantics, we multiply the relevant local probabilities:

$$P(j|a)P(m|a)P(a|\neg b, \neg e)P(\neg b)P(\neg e)P(j|a)P(m|a)P(a|\neg b, \neg e)P(\neg b)P(\neg e)$$

This shows how a single entry in the full joint distribution is calculated from the local node tables.

## Why This Matters

Because every complete assignment can be computed in this way, a Bayesian network fully specifies the whole joint distribution.

And since the full joint distribution can answer **any query**, the Bayesian network can also answer any query in principle.

That said, computing answers by explicitly summing all joint entries is often too slow, so later sections introduce more efficient inference methods.

### 3. Meaning of the Local Parameters

The section asks an important question:

**What do the local parameters  $\theta(x_i|\text{parents}(X_i))$  actually mean?**

The answer is:

They are exactly the conditional probabilities:

$$P(x_i|\text{parents}(X_i)) = \theta(x_i|\text{parents}(X_i))$$

So the parameters stored in the network are not arbitrary numbers. They have a precise probabilistic interpretation.

#### Very Important Semantic Identity

The local parameter for a node equals the conditional probability of that node given its parents:

$$\theta(x_i|\text{parents}(X_i)) = P(x_i|\text{parents}(X_i))$$

Therefore the semantics can also be written as:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i|\text{parents}(X_i))$$

This is one of the most important equations for Bayesian networks.

#### Why This Is Important

Each number in the network has a **clear, local meaning**.

Example:

If we set:

$$\theta(\text{JohnCalls}=\text{true}|\text{Alarm}=\text{true}) = 0.90$$

this means:

- whenever the alarm sounds,
- John calls about 90% of the time

This local interpretability is a huge strength of Bayes nets because:

- it makes them easier to specify
- easier to understand
- more robust
- and more natural for domain experts

#### 4. Constructing a Bayesian Network

After defining what a Bayes net means, the section explains **how to build one**.

The key idea is to use the **chain rule** and conditional independence.

#### 5. Chain Rule

The **chain rule** says that any joint distribution can be written as:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | x_{i-1}, \dots, x_1)$$

This is always true for any ordering of variables.

So in the worst case, each variable may depend on all previous variables in the chosen ordering.

Bayesian networks become compact when we can replace:

$$P(x_i | x_{i-1}, \dots, x_1)$$

with a much smaller conditional probability:

$$P(x_i | \text{Parents}(x_i))$$

#### Key Independence Condition

A Bayes net is a correct representation if, for every variable  $x_i$ ,

$$P(x_i | x_{i-1}, \dots, x_1) = P(x_i | \text{Parents}(x_i))$$

provided the parents come from the earlier variables.

This means:

**Each variable must be conditionally independent of its other earlier variables, given its parents.**

This is the core condition that makes the Bayes net factorization valid.

#### 6. Topological Ordering

To make the above construction work, variables are numbered in a **topological ordering**.

A **topological ordering** is any order consistent with the direction of arrows in the DAG.



That means:

- parents always come before children

This guarantees the construction stays acyclic.

## 7. Method for Constructing a Bayesian Network

The book gives a practical construction method:

### Step 1: Choose the Variables

Determine the random variables needed to model the domain.

### Step 2: Order the Variables

Choose an ordering  $X_1, \dots, X_n$ .

Any order works in principle, but the network is often much more compact if:

- **causes come before effects**

### Step 3: Choose Parents for Each Variable

For each variable  $X_i$ :

- choose a **minimal set of parents** from earlier variables
- such that  $X_i$  is conditionally independent of all other earlier variables given those parents

### Step 4: Add Links

Draw arrows from each chosen parent to  $X_i$ .

### Step 5: Specify CPT

Write the conditional probability table:

$$P(X_i | \text{Parents}(X_i))$$

### Intuition for Choosing Parents

The parents should be the variables that **directly influence** the node.

Not every earlier variable that has some indirect relevance should become a parent.

Only the minimal set needed to make the conditional independence relation true should be chosen.

### Example: Choosing Parent for MaryCalls

In the burglary network, MaryCalls may be indirectly related to:

- Burglary
- Earthquake
- JohnCalls

But these are not direct influences.

The book argues that:

$$P(\text{MaryCalls} | \text{JohnCalls}, \text{Alarm}, \text{Earthquake}, \text{Burglary}) = P(\text{MaryCalls} | \text{Alarm}) P(\text{MaryCalls} | \text{JohnCalls}, \text{Alarm}, \text{Earthquake}, \text{Burglary}) = P(\text{MaryCalls} | \text{Alarm})$$

So the only parent MaryCalls needs is:

- Alarm

This illustrates the general idea that parents should be the direct influencers, not every related variable.

## 8. Why Bayes Nets Are Acyclic and Consistent

Because each node is only allowed to choose parents from earlier variables, cycles cannot arise.

So the construction automatically produces a DAG.

### No Redundant Probability Values

Another important point is that Bayes nets contain no redundant probabilities.

This means:

- the model does not overspecify the same thing twice
- so there is no chance of internal inconsistency

As a result:

- the user cannot accidentally specify probabilities that violate the probability axioms

This is a major practical advantage over less structured knowledge representations.

## 9. Compactness of Bayesian Networks

One of the most important benefits of Bayes nets is **compactness**.

Bayesian networks are often much smaller than full joint distributions because they exploit **local structure**.

## Locally Structured / Sparse Systems

A **locally structured** or **sparse** system is one in which each component directly interacts with only a limited number of other components.

In such systems:

- complexity often grows linearly rather than exponentially

Bayesian networks exploit this property.

### Size Comparison

Suppose:

- there are  $n$  Boolean variables
- each variable has at most  $k$  parents

Then:

- each CPT needs at most  $2^k$  numbers
- the entire network needs about  $2^k \cdot n$  numbers

But the full joint distribution needs:

$2^n$

numbers.

So if  $k$  is small and  $n$  is large, the savings are enormous.

### Example

If:

- $n=30$
- each node has at most  $k=5$  parents

then:

- Bayes net needs  $2^5 \cdot 30 = 960$  numbers
- full joint needs more than one billion numbers

This illustrates why Bayesian networks are feasible while full joint tables are usually not.

## 10. Tradeoff: Accuracy vs. Simplicity

If we fully connect the network so that every variable depends on all earlier ones, then the Bayes net is no smaller than the full joint distribution.

So to keep the representation compact, we often omit weak dependencies.

This means that sometimes:

- a slight dependency exists in reality
- but we ignore it because modeling it would add too much complexity

The modeler must balance:

- increased accuracy
- against increased complexity and harder probability assessment

## **11. Why Node Ordering Matters**

Even if the domain is locally structured, we get a compact Bayes net only if we choose a good node ordering.

This is a very important practical lesson.

### **Good Ordering**

Usually:

- causes first
- effects later

This tends to produce smaller, simpler CPTs.

### **Bad Ordering**

If we choose an unfortunate ordering, variables may require many more parents to preserve correctness.

This results in:

- more arrows
- larger CPTs
- more parameters
- less natural judgments

### **Burglary Example**

A bad ordering such as:

- MaryCalls, JohnCalls, Alarm, Burglary, Earthquake

forces the network to include more dependencies and more parameters than the original causal ordering.

Even worse, it requires unnatural judgments such as:

- probability of Earthquake given Burglary and Alarm

These are harder for experts to estimate.

### **Very Bad Ordering**

An even worse ordering can make the network just as large as the full joint distribution.

Important point:

### **Different Bayes nets can represent the same joint distribution**

But some do so compactly, while others fail to capture all conditional independence and therefore require many unnecessary parameters.

### **Key Takeaway**

Causal orderings are usually best because:

- they often yield smaller networks
- and the required probabilities are easier for humans to specify

This links back to Chapter 12's distinction between **causal** and **diagnostic** probabilities.

## **12. Conditional Independence Relations in Bayesian Networks**

From the semantics of Bayes nets, several key conditional independence properties follow.

### **Non-Descendants Property**

A variable is conditionally independent of its **non-descendants**, given its parents.

### **Descendant**

A descendant of a node is a node reachable by following arrows forward from it.

So non-descendants are nodes that are not its descendants.

### **Meaning**

Once we know a variable's parents, it no longer depends on any node that is not one of its descendants.

This is a fundamental independence property built into the network semantics.

### **Example**

In the burglary network:

- JohnCalls is independent of Burglary, Earthquake, and MaryCalls
- once Alarm is known

This matches the intuition that once we know whether the alarm sounded, John's call gives no extra dependence on those other variables.

## Alternative View of Bayes Nets

The book points out that we can think of Bayes nets in two equivalent ways:

### View 1

They define the full joint distribution by the product formula.

### View 2

They define a set of conditional independence assumptions from which the full joint can be derived.

So Bayes nets are both:

- a factorized distribution
- and a structured set of conditional independence statements

## 13. Markov Blanket

Another extremely important concept is the **Markov blanket**.

A variable is conditionally independent of **all other nodes** in the network given its:

- parents
- children
- children's parents

This set is called the variable's **Markov blanket**.

### Why It Matters

If you know a node's Markov blanket, then no other variable in the network provides additional information about that node.

So the Markov blanket is the minimal local neighborhood needed to predict the node.

### Example

In the burglary network:

- Burglary is independent of JohnCalls and MaryCalls
- given Alarm and Earthquake

Those nodes form its relevant local context.

### Importance in Inference

The Markov blanket property is crucial for local inference methods and sampling algorithms.

It allows computation to focus on a small neighborhood rather than the whole network.

This becomes important later in approximate inference methods

## 14. D-Separation

The most general independence question is:

**Is set  $XX$  conditionally independent of set  $YY$ , given set  $ZZ$ ?**

In Bayesian networks, this is answered by **d-separation**.

### D-Separation Procedure

To check whether  $ZZ$  d-separates  $XX$  and  $YY$ :

#### Step 1: Build the Ancestral Subgraph

Keep only:

- $XX, YY, ZZ$
- and all their ancestors

#### Step 2: Moralize the Graph

Add links between any two nodes that share a child.

This produces the **moral graph**.

#### Step 3: Drop Direction

Replace all directed edges with undirected ones.

#### Step 4: Check Separation

If  $ZZ$  blocks all paths between  $XX$  and  $YY$  in this graph, then:

- $ZZ$  d-separates  $XX$  and  $YY$

Therefore:

- $XX$  is conditionally independent of  $YY$  given  $ZZ$

### Intuition

D-separation is the graphical test for whether the network structure implies a certain conditional independence statement.

It is one of the most important conceptual tools for Bayesian networks.

## Example in Burglary Network

Using d-separation, we can conclude:

- Burglary and Earthquake are independent unconditionally
- JohnCalls and MaryCalls are conditionally independent given Alarm
- Burglary and Earthquake are **not** necessarily independent given Alarm

This last point is subtle and important: conditioning can sometimes create dependence.

## 15. Efficient Representation of Conditional Distributions

Even if each node has only a small number of parents, a CPT may still be large.

For  $k$  Boolean parents:

- full CPT may require  $O(2^k)$  numbers

But often the dependency follows a regular pattern, so we can represent it more efficiently.

These are called **canonical distributions**.

## 16. Deterministic Nodes

The simplest special case is a **deterministic node**.

A deterministic node has no uncertainty:

- its value is completely determined by its parents

Examples:

- logical OR of parent values
- minimum of a set of values
- sum of inflows and outflows for a reservoir

Deterministic nodes can often be represented by functions rather than large CPTs.

This saves space and can simplify inference.

## 17. Context-Specific Independence (CSI)

A conditional distribution has **context-specific independence** if the child becomes independent of some parents in certain contexts.

### Example

Suppose Damage depends on:

- Ruggedness



- Accident

If:

- Accident = false

then Damage no longer depends on Ruggedness.

So:

- Damage is context-specifically independent of Ruggedness
- given Accident = false

### Why It Matters

CSI allows us to simplify CPTs because some parent variables matter only in certain cases.

This can be represented with if-then-else style conditional descriptions rather than full tables.

CSI can also speed up inference.

### 18. Noisy-OR

A very important canonical distribution is **noisy-OR**.

It generalizes logical OR to uncertain causal relationships.

#### Standard Logical OR

In logic, Fever might be true if:

- Cold
- Flu
- or Malaria

are true.

#### Noisy-OR Idea

In noisy-OR:

- each cause may fail to activate the effect
- even if the cause is present

So:

- having Flu may usually cause Fever
- but sometimes the causal influence is inhibited

#### Assumptions of Noisy-OR

1. All relevant causes are listed  
(or else include a leak node for miscellaneous unmodeled causes)
2. Inhibition of each cause is independent of inhibition of the others

### Leak Node

A **leak node** captures the possibility of “other causes not explicitly listed.”

This helps when the model does not include all possible causes.

### Benefit of Noisy-OR

For a variable with  $k$  parents:

- full CPT needs  $O(2^k)$  parameters
- noisy-OR needs only  $O(k)$  parameters

This is a huge savings.

It makes:

- assessment easier
- learning easier
- storage smaller

### Practical Importance

Large medical networks often rely on noisy-OR or related noisy logical models because they would otherwise need enormous CPTs.

## 19. Continuous Variables in Bayesian Networks

Real-world domains often include continuous quantities such as:

- height
- temperature
- money
- cost
- mass

Since continuous variables have infinitely many possible values, we cannot list all probabilities in a table.

So Bayesian networks need special methods for continuous variables.

## 20. Discretization

One simple approach is **discretization**.

This means:

- divide the continuous range into a fixed number of intervals
- treat each interval as a discrete category

Example:

Temperature might be split into:

- below 0°C
- 0–100°C
- above 100°C

## Tradeoff

Discretization involves a tradeoff:

- more intervals = more accuracy
- but also larger CPTs and slower inference

## 21. Parametric Continuous Distributions

Another approach is to use standard probability density functions with a few parameters.

Example:

A Gaussian distribution is specified by:

- mean  $\mu$
- variance  $\sigma^2$

This is often much more efficient than discretizing.

## 22. Nonparametric Representation

Another possibility is a **nonparametric representation**, where the distribution is represented implicitly using stored examples/instances.

This is more flexible and can represent complex distributions without assuming a fixed parametric form.

## 23. Hybrid Bayesian Networks

A **hybrid Bayesian network** contains both:

- discrete variables
- continuous variables

To define such a network, we need:

1. distributions for continuous variables with discrete/continuous parents

2. distributions for discrete variables with continuous parents

## 24. Linear–Gaussian Conditional Distributions

For a continuous child with continuous parents, a common choice is the **linear–Gaussian** model.

This means:

- the child has a Gaussian distribution
- the mean depends linearly on the parents
- the variance is fixed

Example:

Cost depends on Harvest and Subsidy.

The distribution over Cost is Gaussian, and its mean changes linearly with Harvest.

### Why Important

Linear–Gaussian models are mathematically convenient.

A network of continuous variables with linear–Gaussian relationships yields:

- a multivariate Gaussian joint distribution

and posterior distributions also remain Gaussian.

This makes inference much easier.

### Conditional Gaussian

If we add discrete parents to continuous variables, we get a **conditional Gaussian** model:

- once the discrete variables are fixed,
- the continuous variables follow a multivariate Gaussian

This is still tractable in important special cases.

## 25. Discrete Variable with Continuous Parent: Soft Thresholds

When a discrete variable depends on a continuous parent, we often want a **soft threshold**.

Example:

- customer buys fruit if cost is low
- does not buy if cost is high
- and changes gradually in between

Two common models are:

## **Probit**

Based on the Gaussian cumulative distribution

## **Logit / Expit / Inverse Logit**

Based on the logistic function

Both create smooth S-shaped probability curves.

## **Difference Between Probit and Logit**

They look similar, but:

- the logistic model has longer tails
- probit sometimes fits real data better
- logistic is often mathematically easier and widely used in machine learning

These can also be generalized to multiple parent variables using linear combinations.

## **26. Case Study: Car Insurance Bayesian Network**

The section ends with a large case study showing how Bayes nets can model a real decision problem.

### **Goal of the Model**

A car insurance company wants to decide what premium to charge an applicant.

To do that, it needs to estimate future claims, based on:

- information on the application form
- and hidden underlying risk factors

### **Output Variables**

The costs to the insurer include:

- **MedicalCost**
- **LiabilityCost**
- **PropertyCost**

These are the outputs the insurer ultimately cares about.

### **Input Variables**

Observed information from the application includes:

#### **About the applicant**

- Age
- YearsLicensed
- DrivingRecord
- GoodStudent

### About the vehicle

- MakeModel
- VehicleYear
- Airbag
- SafetyFeatures

### About the driving situation

- Mileage
- Garaged

### Hidden Variables

The model includes important **hidden variables** that are not directly observed but are needed to structure the domain.

Examples:

- Theft
- Accident
- SocioEcon
- RiskAversion
- DrivingSkill
- DrivingBehavior
- CarValue

These hidden variables are essential because they help explain dependencies compactly.

This is a major point:

**Hidden variables are often crucial for keeping the network sparse and meaningful.**

### Example Causal Reasoning in the Insurance Network

#### Theft

Theft depends on factors like:

- MakeModel
- CarValue
- Garaged
- AntiTheft

#### CarValue

CarValue depends on:

- MakeModel
- VehicleYear
- Mileage

### **RiskAversion**

RiskAversion affects:

- whether the applicant garages the car
- whether they buy safety features
- their driving behavior

It depends on things like:

- Age
- SocioEcon

### **DrivingSkill**

DrivingSkill depends on:

- Age
- YearsLicensed

### **DrivingBehavior**

DrivingBehavior depends on:

- RiskAversion
- DrivingSkill

### **Accident**

Accident depends on:

- DrivingBehavior
- Mileage
- SafetyFeatures

### **Costs**

If an accident occurs, cost variables depend on:

- Age
- Cushioning
- Ruggedness
- Airbag
- CarValue
- and other vehicle properties

This illustrates the reasoning process for constructing a realistic causal Bayes net.

## 27. Design Choices: Discrete vs. Continuous Variables

When building a practical Bayes net, an important choice is whether each variable should be:

- discrete
- or continuous

### Continuous Variables

Pros:

- more precise

Cons:

- exact inference is harder or impossible except in special cases

### Discrete Variables

Pros:

- simpler exact inference
- easier to handle with CPTs

Cons:

- large ranges can make CPTs huge and tedious

Example:

- MakeModel may have thousands of values
- this can create enormous tables for child nodes such as CarValue

But if MakeModel is always observed, that reduces inference complexity because only one row is relevant at runtime.



# Active / Inactive Paths

## Question: Are X and Y conditionally independent given evidence variables {Z}?

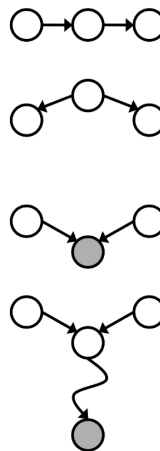
- Yes, if X and Y “d-separated” by Z
- Consider all (undirected) paths from X to Y
- No active paths = independence!

## A path is active if each triple is active:

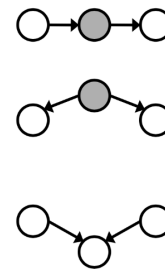
- Causal chain  $A \rightarrow B \rightarrow C$  where B is unobserved (either direction)
- Common cause  $A \leftarrow B \rightarrow C$  where B is unobserved
- Common effect (aka v-structure)  
 $A \rightarrow B \leftarrow C$  where B or one of its descendants is observed

## All it takes to block a path is a single inactive segment

Active Triples



Inactive Triples



## Textbook readings 13.3

### 1. Basic Task of Probabilistic Inference

The core task in probabilistic inference is:

**Given:**

- a **query variable** XXX
- some observed **evidence variables** EEE with values eee

**Compute:**

$$P(X|e)P(X \setminus e|e)P(X|e)$$

That is, compute the posterior distribution of the query variable after observing the evidence.

### Notation Used

The section reuses the notation from Chapter 12:

- XXX: the query variable
- EEE: the set of evidence variables
- eee: the observed values of the evidence variables
- YYY: the hidden variables  
(all variables that are neither evidence variables nor the query variable)

So the complete set of variables is:

$$\{X\} \cup E \cup Y \setminus \{X\} \cup E \cup Y \setminus \{X\} \cup E \cup Y$$

This notation is important because inference algorithms are usually described in terms of these three groups.

## 2. Example Query

In the burglary network, suppose we observe:

- JohnCalls=true
- MaryCalls=true

Then we might ask:

$P(\text{Burglary} \mid \text{JohnCalls}=\text{true}, \text{MaryCalls}=\text{true})$

This posterior distribution turns out to be:

$\langle 0.284, 0.716 \rangle$

Meaning:

- probability of burglary = 0.284
- probability of no burglary = 0.716

So even though both neighbors called, burglary is still not more likely than no burglary. This reflects the influence of the prior probabilities and the possibility of false alarms.

## 3. Exact Inference

This section focuses on **exact inference**.

That means:

- the algorithm computes the exact posterior probability
- not an approximation

This is important conceptually because it gives the correct mathematical answer.

However, the drawback is that exact inference is often computationally difficult.

The chapter later shows that exact inference is generally intractable in the worst case, which motivates approximate methods.

## 4. Starting Point: Conditional Probability from the Joint Distribution

From Chapter 12, the key formula is:

$$P(X \mid e) = \frac{P(X, e)}{\sum_y P(X, e, y)}$$

This is the standard formula for computing a posterior distribution by summing over all hidden variables.

### Interpretation

To compute  $P(X|e)P(X \setminus e)P(X|e)$ :

1. Fix the query variable  $XXX$  and evidence  $eee$
2. Sum over every possible assignment  $yyy$  of the hidden variables
3. Normalize using  $\alpha$

This is exactly what inference by enumeration does.

## 5. How the Bayesian Network Helps

A Bayesian network represents the full joint distribution in factored form:

$$P(x_1, \dots, x_n) = \prod_i P(x_i | \text{Parents}(X_i))$$

So instead of storing a giant joint table, we can compute each joint term  $P(X, e, y)$  as a product of local CPT entries from the network.

Thus, inference by enumeration in a Bayesian network means:

- sum over hidden variables
- but evaluate each joint term using the Bayes net factorization

## 6. Example: Burglary Query by Enumeration

Suppose we want:

$$P(B | j, m)P(B \setminus j, m)P(B | j, m)$$

where:

- $BBB = \text{Burglary}$
- $jjj = \text{JohnCalls} = \text{true}$
- $mmm = \text{MaryCalls} = \text{true}$

The hidden variables are:

- $EEE = \text{Earthquake}$
- $AAA = \text{Alarm}$

So:

$$P(B | j, m) = \alpha \sum_e \sum_a P(B, j, m, e, a)P(B \setminus j, m) = \alpha \sum_e \sum_a P(B, j, m, e, a)$$

This means:

- consider every possible combination of Earthquake and Alarm
- compute the joint probability of each complete assignment
- sum them

- normalize

## 7. Expanding the Joint Using the Bayes Net

Using the semantics of the Bayesian network, one joint term becomes:

$$P(b,j,m,e,a) = P(b)P(e)P(a|b,e)P(j|a)P(m|a) \\ P(b,j,m,e,a) = P(b)P(e)P(a \mid b,e)P(j \mid a)P(m \mid a)P(b,j,m,e,a) = P(b)P(e)P(a|b,e)P(j|a)P(m|a)$$

So:

$$P(b|j,m) = \alpha \sum_e \sum_a P(b)P(e)P(a|b,e)P(j|a)P(m|a) \\ P(b|j,m) = \alpha \sum_e \sum_a P(b)P(e)P(a \mid b,e)P(j \mid a)P(m \mid a)P(b|j,m) = \alpha \sum_e \sum_a P(b)P(e)P(a|b,e)P(j|a)P(m|a)$$

This is a sum of products of local conditional probabilities.

### Main Idea

Each complete assignment corresponds to:

- one path through the hidden-variable possibilities
- and each path contributes a product of CPT entries

The final answer is obtained by summing all such products.

## 8. Complexity of the Naive Enumeration Approach

If there are many hidden variables, this quickly becomes expensive.

For  $n$  Boolean hidden variables:

- there are  $2^n$  possible assignments

Each assignment requires multiplying together probabilities for all variables.

So a naive implementation has complexity:

$$O(n \cdot 2^n)$$

or, as the text phrases it in one step of analysis,  $O(n \cdot 2^n)$  /  $O(n \cdot 2^n)$ -style exponential cost depending on the implementation detail.

The key point is:

**Enumeration is exponential in the number of hidden variables.**

This makes it impractical for large networks.

## 9. Improving the Enumeration Expression

The text shows that we can make the computation more efficient by using the nested structure of the sum.

Instead of writing:

$$\sum_e \sum_a P(b)P(e)P(a|b,e)P(j|a)P(m|a) \sum_e \sum_a P(b)P(e)P(a|b,e)P(j|a)P(m|a)$$

we move constant factors outside the sums when possible:

$$P(b|j,m) = \alpha P(b) \sum_e P(e) \sum_a P(a|b,e)P(j|a)P(m|a) P(b|j,m) = \alpha P(b) \sum_e P(e) \sum_a P(a|b,e)P(j|a)P(m|a)$$

This is mathematically equivalent, but computationally better organized.

## Why This Helps

Not every factor depends on every variable.

For example:

- $P(b)P(b)P(b)$  does not depend on  $e$  or  $a$
- so it does not need to be recomputed inside the loops

This allows a more efficient recursive evaluation.

## 10. Tree Structure of the Computation

The book represents the evaluation as a tree.

**In that tree:**

- top levels correspond to choosing values of variables
- each branch multiplies probabilities along the way
- “+” nodes sum over alternative values of hidden variables

So the computation proceeds:

- top down
- multiplying on branches
- summing over branches where variables are hidden

This gives a nice visual interpretation of enumeration.

## 11. Example Result

Using the burglary network probabilities:

- the unnormalized value for burglary = true is  $\alpha \times 0.00059224$

- the unnormalized value for burglary = false is  $\alpha \times 0.0014919 \times 0.0014919$

After normalization:

$$P(B | j, m) = \langle 0.284, 0.716 \rangle$$

So the posterior probability of burglary is about 28%.

## 12. Role of Normalization

As in previous chapters,  $\alpha$  is the normalization constant.

Enumeration computes relative weights for each possible value of the query variable.

Then these values are normalized so they sum to 1.

This avoids having to explicitly compute the denominator in conditional probability.

## 13. ENUMERATION-ASK Algorithm

The section gives the **ENUMERATION-ASK** algorithm.

This is the standard exact inference algorithm by enumeration.

### High-Level Idea of ENUMERATION-ASK

For each possible value  $x_i$  of the query variable XXX:

1. Temporarily set  $X = x_i$
2. Extend the evidence with that value
3. Call a recursive procedure that sums over all remaining variables
4. Store the result as the unnormalized probability of  $x_i$

Finally:

5. Normalize the distribution over XXX

### Output

The result is a full posterior distribution over the query variable.

So if XXX is Boolean, we get two values:

- probability of true
- probability of false

## 14. ENUMERATE-ALL Recursive Procedure

The recursive helper function processes variables one by one.

It behaves differently depending on whether the current variable is already assigned in the evidence.

### **Case 1: Variable Is Evidence**

If the current variable VVV already has a value in the evidence, then:

- take the probability of that value given its parents
- multiply by the recursive result for the rest of the variables

So evidence variables contribute their probability factor directly.

### **Case 2: Variable Is Hidden**

If the current variable is not fixed by evidence, then:

- sum over all possible values vvv of that variable
- for each value:
  - multiply its probability given parents
  - recurse on the rest

So hidden variables are explicitly summed out.

### **Base Case**

If there are no variables left:

- return 1.0

This is the multiplicative identity, allowing recursion to build products correctly.

## **15. Why the Algorithm Is Similar to Backtracking / DPLL**

The text points out that the recursive structure is similar to algorithms from earlier chapters:

- **backtracking for CSPs**
- **DPLL for satisfiability**

That is because the algorithm explores combinations of variable assignments recursively.

But unlike logical search, here it:

- multiplies probabilities
- and sums over hidden choices

So it is a probabilistic analogue of recursive search over assignments.

## **16. Space Complexity**

One nice property of enumeration is that it does **not** explicitly construct the full joint distribution.

Instead, it recursively computes the needed sums.

So its **space complexity is only linear** in the number of variables.

This is much better than storing an exponential-size table.

### **Important Point**

Even though the time complexity is exponential, the memory use is relatively modest.

That is because recursion only needs to keep track of:

- the current assignment path
- the current stack of recursive calls

## **17. Time Complexity**

The bad news is that enumeration still has exponential time complexity.

For a network with  $n$  Boolean non-evidence variables:

- time complexity is  $O(2^n)$

This is because it must consider all possible assignments of the hidden variables.

So even though it avoids building the full joint explicitly, it still effectively traverses an exponentially large space of assignments.

### **Example from Insurance Network**

The text says that for the insurance network discussed earlier, exact inference using enumeration requires around:

- **227 million arithmetic operations**

for a typical query on the cost variables.

This shows how quickly exact enumeration becomes impractical even for moderately sized networks.

## **18. Main Inefficiency: Repeated Computation**

The section ends by pointing out a key inefficiency in the enumeration tree:

### **repeated subexpressions**

Some probability products are computed more than once.

Example:



- $P(j|a)P(m|a)P(j \mid a)P(m \mid a)P(j|a)P(m|a)$
- $P(j|\neg a)P(m|\neg a)P(j \mid \neg a)P(m \mid \neg a)P(j|\neg a)P(m|\neg a)$

These are repeated for each value of Earthquake.

So the algorithm wastes time recomputing identical pieces of the expression.

## Why This Matters

This repeated work is the main reason the basic enumeration approach is inefficient.

The next section introduces better inference methods that avoid this duplication.

This is the key transition from simple enumeration to more advanced exact inference algorithms.

## 19. Big Picture Meaning of Enumeration

Inference by enumeration is important because it shows:

1. Bayesian networks really can answer probabilistic queries
2. exact inference can be done by combining:
  - summation over hidden variables
  - products of local CPT factors
3. the naive exact method is conceptually simple but computationally expensive

So enumeration is mainly the **foundational exact inference algorithm**:

- easy to understand
- correct
- but not efficient enough for large-scale use

## High-Yield Concepts to Know

### Query Variable

The variable whose posterior probability distribution we want to compute.

### Evidence Variables

Variables whose values are observed.

### Hidden Variables

Variables that are neither query variables nor evidence variables; these must be summed out.

### Posterior Probability

$$P(X|e)P(X \mid e)P(X|e)$$

The distribution over the query variable after conditioning on evidence.

## Enumeration

The process of summing over all assignments of hidden variables.

## Exact Inference

Inference that computes the exact posterior probability, not an approximation.

## Normalization Constant ( $\alpha$ )

A factor used to rescale unnormalized values so they sum to 1.

## Most Important Formula

### Posterior by summing out hidden variables

$$P(X | e) = \alpha \sum_y P(X, e, y) \quad P(X \mid e) = \frac{\sum_y P(X, e, y)}{\sum_y P(X, e, y)}$$

This is the core inference formula.

## Bayes net factorization inside the sum

Each term  $P(X, e, y)$  is computed as a product of local CPT probabilities:

$$P(x_1, \dots, x_n) = \prod_i P(x_i \mid \text{Parents}(X_i)) \quad P(x_1, \dots, x_n) = \prod_i P(x_i \mid \text{Parents}(X_i))$$

So exact inference in a Bayes net becomes:

- sum over hidden-variable assignments
- of products of local factors

## Core Algorithm Idea

### ENUMERATION-ASK

For each possible value of the query variable:

- extend the evidence with that value
- call a recursive procedure to sum out the remaining variables
- collect the results
- normalize

### ENUMERATE-ALL

Recursively process variables in order:

- if assigned by evidence, multiply by its probability and continue
- if hidden, sum over all its possible values

### Strengths of Enumeration

- Conceptually simple
- Always correct
- Uses the Bayesian network directly
- Only requires linear space
- Good as a baseline exact inference method

### Weaknesses of Enumeration

- Exponential time complexity
- Repeats many identical subcomputations
- Not practical for large networks
- Motivates more advanced exact inference methods

### Textbook readings 19.1

#### 1. What It Means for an Agent to Learn

An agent is said to be **learning** if it improves its performance after making observations about the world.

This is a very broad definition. Learning can be:

- extremely simple, such as writing down a shopping list,
- or extremely deep and scientific, such as forming a new theory of the universe.

So learning is not limited to one kind of activity. The common idea is:

**Learning = using observations/experience to improve future behavior or understanding.**

#### 2. What Machine Learning Is

When the learning agent is a computer, this is called **machine learning**.

Machine learning means:

- a computer observes data,
- builds a model from that data,
- and then uses that model both:
  - as a hypothesis about how the world works, and
  - as a tool for solving problems.

So the learned model has a dual role:

**1. It represents knowledge about the world**

**2. It acts like software that can make predictions or decisions**

This is one of the most important conceptual points in machine learning.

**3. Why We Want Machines to Learn**

The text asks an important question:

**Why not just program the system correctly in the first place?**

It gives **two main reasons**.

**Reason 1: Designers Cannot Anticipate Every Future Situation**

Many environments are too variable or too large for programmers to hand-code responses for every possible case.

**Example: Maze robot**

A robot built to navigate mazes cannot be pre-programmed with the layout of every new maze. It must **learn the layout** as it explores.

**Main point**

Learning is necessary when the environment changes or contains new situations that the designer could not fully foresee.

**Reason 2: Designers May Not Know How to Program the Solution**

Some tasks are easy for humans or animals but very hard to describe explicitly.

**Example: Face recognition**

People can recognize family members' faces naturally and subconsciously, but they usually cannot explain exactly how they do it in a precise rule-based way.

So even expert programmers cannot simply write down all the right rules.

**Main point**

Sometimes we want machine learning because the required knowledge is too subtle, complex, or unconscious to program directly.

**5. Forms of Learning Depend on Several Factors**

**Factors that determine the learning setup:**

1. **Which component** of the agent is being improved
2. **What prior knowledge** the agent already has
3. **What data and feedback** are available

These three factors shape the kind of model the agent can build and the kind of learning algorithm that can be used.

## 6. Prior Knowledge

A very important concept is **prior knowledge**.

Prior knowledge means:

- what the agent already knows before it begins learning

This matters because it affects:

- the kinds of models the agent can consider
- how much data it needs
- how quickly it can learn

The chapter assumes relatively little prior knowledge:

- the agent starts mostly from scratch
- and learns mainly from the data

However, the designer still plays a role by choosing a **model framework** that makes good learning possible.

So even in “learning from scratch,” the designer is not doing nothing:

- the designer chooses the representation/model family
- the agent learns the actual parameters or rules from data

## 7. Components of an Agent That Can Be Learned

The text connects machine learning to the earlier discussion of agents. It says that **many different components** of an agent program can be learned.

These include:

### 1. Direct Mapping from Current Conditions to Actions

This means learning rules such as:

- “when should I brake?”
- “when should I turn left?”
- “when should I take a certain action?”

This is essentially learning a **policy** or condition-action mapping.

## 2. Means to Infer Relevant Properties of the World from Percepts

This means learning how to interpret sensory data.

Example:

- learning to recognize buses in images
- learning to detect pedestrians from camera input

This is about learning **perception** or **state estimation**.

## 3. Information About How the World Evolves and the Results of Actions

This means learning:

- what happens when actions are taken
- how the environment changes over time

Example:

- learning what happens when braking on a wet road

This is about learning a **world model** or action effects.

## 4. Utility Information

This means learning how desirable different outcomes are.

Example:

- learning that passengers dislike being shaken up
- so smoother driving should have higher utility

This is about learning the **utility function** or preferences.

## 5. Action-Value Information

This means learning how desirable particular actions are in particular situations.

This is closely connected to reinforcement learning and value estimation.

## 6. Goals

The agent may learn what states should count as desirable or goal states.

This is a higher-level form of learning about what it should try to achieve.

## 7. Problem Generator, Critic, and Learning Element

These are the internal pieces that help the system:

- explore,
- evaluate itself,
- and improve.

These too can be adapted or learned.

## **8. Self-Driving Car Example**

The section uses a self-driving car observing a human driver to show how multiple components can be learned.

### **Learning a Condition–Action Rule**

Whenever the human driver brakes, the system can learn:

- under what conditions braking is appropriate

This corresponds to learning component 1:

- direct mapping from state to action

### **Learning Object Recognition**

If shown many images labeled as containing buses, the system can learn:

- how to recognize buses

This corresponds to learning component 2:

- inferring world properties from percepts

### **Learning Action Effects**

If the system tries actions and watches what happens, it can learn:

- the effects of those actions

Example:

- what hard braking does on wet roads

This corresponds to learning component 3:

- knowledge of action outcomes

### **Learning Part of the Utility Function**

If passengers complain after being shaken around, the system can learn:

- that rough driving reduces desirability

This corresponds to learning component 4:

- utility information

### **Big Lesson from This Example**

Machine learning is not limited to just one module like image classification. It can improve:

- perception,
- decision making,
- prediction,
- preferences,
- and action choice.

So learning can affect almost every part of an intelligent agent.

## **9. Machine Learning as a Standard Part of Software Engineering**

The text makes the strong point that machine learning is no longer just a niche AI topic. It has become a **standard part of software engineering**.

That means:

- when building software systems, even systems not usually called “AI agents,”
- machine learning may still improve some components.

### **Examples Given**

The section gives examples where machine learning dramatically improved real systems:

- reducing energy use for cooling data centers by **40%**

These examples illustrate that machine learning is useful far beyond toy problems or academic demos.

### **Broader Message**

Machine learning has had such a large impact that it is reshaping computing itself, including software and hardware design.

So the field is presented as both:

- practically powerful
- and deeply important in modern computing

## **10. Learning Works with Many Kinds of Models**

The text reminds us that earlier chapters introduced different forms of knowledge representation, such as:



- atomic models
- factored models
- relational models
- logical models
- probabilistic models

The important point is:

**Learning algorithms have been developed for many of these model types.**

So machine learning is not tied to only one representation. It can be combined with many different AI frameworks.

## 11. Learning from Scratch vs. Transfer Learning

This chapter mainly assumes:

- the agent begins with little prior knowledge
- and learns from available data

This is the “start from scratch” setting.

However, the book notes that later it will discuss **transfer learning**, where:

- knowledge from one domain is reused in a new domain

This allows:

- faster learning
- less required data

So the chapter’s default assumption is minimal prior knowledge, but this is not the only possible setup.

## 12. Induction

### Definition

Induction is the process of going from:

- specific observations  
to a general rule or prediction

### Example

From observing that the sun has risen every day in the past, we induce:

- the sun will rise tomorrow

### Induction vs. Deduction

## Deduction

- studied earlier in logic
- if the premises are true and reasoning is correct, the conclusion must be true

## Induction

- conclusions are not guaranteed
- they may turn out to be wrong

So induction is fundamentally different from logical deduction.

## Key Point

Machine learning is mostly based on **induction**:

- generalizing from examples
- without certainty

This is why learned conclusions can be useful but not logically guaranteed.

## 13. Input Representation in This Chapter

The chapter mainly focuses on problems where the input is a **factored representation**.

That means:

- the input is represented as a vector of attribute values

Example:

- age, income, mileage, risk score, etc.
- or pixel-derived features, measurements, categorical values, etc.

The text also notes that learning can be done with other kinds of input structures too, including:

- atomic data
- relational data

But the main focus here is factored/vector-style inputs.

## 14. Classification

When the output belongs to a **finite set of possible values**, the problem is called **classification**.

## Examples

- sunny / cloudy / rainy
- true / false
- spam / not spam

In classification, the agent must assign each input to one of a limited set of labels or categories.

### **Main Point**

Classification = predicting a **discrete label**

This is one of the most common machine learning problem types.

## **15. Regression**

When the output is a **number**, the learning problem is called **regression**.

### **Examples**

- tomorrow's temperature
- house price
- stopping distance
- stock return
- amount of energy consumption

The text notes that the name “regression” is historically strange and somewhat obscure, but it refers to numeric prediction or function approximation.

### **Main Point**

Regression = predicting a **numeric value**

So the core distinction is:

- **classification** → finite/discrete output
- **regression** → numeric output

## **16. Feedback and the Three Main Types of Learning**

**feedback** as the information available to the learner about its data or behavior.

The kind of feedback determines the **type of learning**.

There are **three main types**:

1. supervised learning
2. unsupervised learning
3. reinforcement learning

## **17. Supervised Learning**

### **Definition**

In **supervised learning**, the agent observes **input-output pairs** and learns a function that maps from input to output.

That means the training data explicitly tells the learner:

- here is the input
- here is the correct answer/output

### Example

Camera images are given, each with a label such as:

- “bus”
- “pedestrian”

The system learns a function that predicts the **right label for a new image**.

The output in supervised learning is often called a **label**.

So in supervised learning:

- the training data is labeled
- and the agent tries to learn the mapping from input to label

### Supervised Learning in the Driving Example

If the car observes the current driving state and later observes how far it took to stop, then:

- the environment itself provides the correct output after the fact

So the environment can act as the teacher.

This is still supervised learning because the agent gets:

- input state
- correct output value

### Main Point

Supervised learning = learning from **examples with correct answers**

## 18. Unsupervised Learning

In **unsupervised learning**, the agent gets only the inputs, with **no explicit feedback or labels**.

So the learner must find a useful structure on its own.

### Most Common Task: Clustering

#### Clustering

Finding groups of similar examples in the data.

### Example

Given millions of internet images, a system may discover a cluster of visually similar images that humans would call:

- “cats”

But no one explicitly told it which images were cats.

So the learner is discovering patterns rather than mapping inputs to provided outputs.

### Main Point

Unsupervised learning = learning **patterns/structure** from unlabeled data

## 19. Reinforcement Learning

### Definition

In **reinforcement learning**, the agent learns from a sequence of **rewards and punishments**.

Instead of receiving the correct output for every input, the agent gets feedback about how good or bad its behavior was.

### Example

At the end of a chess game:

- win = reward
- loss = punishment

The agent is not told exactly which move was right or wrong.

It must figure out:

- which earlier actions contributed to the eventual outcome

This is much harder than supervised learning because the feedback is often:

- delayed
- sparse
- not attached directly to each individual action

### Main Point

Reinforcement learning = learning how to act in order to **maximize future rewards**

## 20. Comparison of the Three Learning Types

## Supervised Learning

- Data: input-output pairs
- Feedback: explicit correct answers
- Goal: learn mapping from input to output

## Unsupervised Learning

- Data: inputs only
- Feedback: none
- Goal: find hidden patterns or structure

## Reinforcement Learning

- Data: experience over time
- Feedback: rewards/punishments
- Goal: learn actions that lead to high long-term reward

## Textbook readings 19.2

**Supervised learning** is the problem of learning a function from **input-output examples**. The learner is given a set of training examples where the correct answer is already known, and its goal is to discover a hypothesis that approximates the unknown true function that generated the data.

### 1. Formal Definition of Supervised Learning

In supervised learning, we are given a **training set** of  $N$  example input-output pairs:

$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$

Each pair is assumed to have been generated by some unknown true function:

$$y = f(x)$$

The task is to discover a function  $h$  that approximates the true function  $f$ .

So the central supervised learning goal is:

**Learn a hypothesis  $h$  that predicts the output  $y$  from input  $x$ .**

### 2. Training Set

The **training set** is the collection of labeled examples the learner receives.

Each example contains:

- an **input**  $x_i$
- and the correct **output**  $y_i$

The training set is the learner's main evidence about how the world works.

## Main point

The learner does not know the true function directly. It only sees examples from it.

## 3. Hypothesis

The learned function  $h$  is called a **hypothesis**.

A hypothesis is:

- the learner's guess about the underlying relationship between inputs and outputs
- a model of the world based on the data

So in supervised learning:

- $f$  = true but unknown function
- $h$  = learned approximation to  $f$

## 4. Hypothesis Space / Model Class / Function Class

The hypothesis  $h$  is chosen from a **hypothesis space**  $H$ .

The **hypothesis space** is the set of all candidate functions the learner is allowed to consider.

Examples of hypothesis spaces:

- all degree-3 polynomials
- all JavaScript functions

The section says these terms are often used similarly:

- **hypothesis space**
- **model class**
- **function class**

## Main idea

The learner does not search over all imaginable functions. It searches only within a chosen hypothesis space.

This choice strongly affects what can be learned.

## 5. Ground Truth

The output value  $y_i$  in each training example is called the **ground truth**.

This means:

- it is the true answer we want the model to predict

So the training data provides the learner with examples of:

- what the correct outputs should be

## **6. Choosing a Hypothesis Space**

A major design decision in supervised learning is:

**What hypothesis space should we use?**

### **A. Use Prior Knowledge**

If we know something about how the data was generated, that knowledge can guide us toward an appropriate hypothesis space.

For example:

- if the data is expected to be approximately linear, a linear model may be reasonable
- if we expect periodic patterns, a sinusoidal model may be better

### **B. Exploratory Data Analysis**

If we do not know much in advance, we can examine the data itself.

This is called **exploratory data analysis**.

Exploratory data analysis includes:

- histograms
- scatter plots
- box plots
- statistical tests
- visualizations in general

The purpose is to get:

- a feel for the structure of the data
- clues about what kind of hypothesis space may fit

### **C. Try Several Hypothesis Spaces**

Another strategy is simply:

- try multiple model classes
- evaluate which one works best

This is often how machine learning is done in practice.



## 7. Choosing a Good Hypothesis Within the Hypothesis Space

Once the hypothesis space  $H$  is chosen, the learner still must decide:

**Which hypothesis  $h \in H$  is best?**

### Consistent Hypothesis

A **consistent hypothesis** is one that exactly matches the training data:

$$h(x_i) = y_i \quad \text{for every training example } (x_i, y_i)$$

for every training example.

If the outputs are discrete or noise-free, consistency may be possible.

### Best-Fit Hypothesis

When outputs are continuous, exact matching is often unrealistic.

In that case, we want a **best-fit function**:

- one where  $h(x_i)$  is close to  $y_i$

This means we want the model to minimize some notion of prediction error.

So in many regression problems, “best fit” replaces “consistency.”

## 8. Training Performance Is Not the Real Goal

A very important idea in supervised learning is:

**The true measure of a hypothesis is not how well it fits the training set.**

Instead, what matters is:

**How well it predicts outputs for inputs it has never seen before.**

This property is called **generalization**.

## 9. Test Set

To evaluate generalization, we use a **test set**.

A **test set** is:

- a second collection of input-output pairs
- separate from the training data
- used only to evaluate performance on unseen examples

The idea is:

- training set measures fit to known data
- test set measures how well the model generalizes

## 10. Generalization

A hypothesis **generalizes well** if it accurately predicts outputs on the test set.

This is one of the most important ideas in machine learning.

### Main point

A good model is not just one that memorizes training examples. It is one that predicts well on new data.

## 11. Figure 19.1: Same Data Source, Different Hypothesis Spaces

The section uses Figure 19.1 to show that the learned function depends on:

1. the hypothesis space
2. the particular training set

The figure shows:

- two different training sets, both sampled from the same true function  $f(x)f(x)f(x)$
- and four different hypothesis spaces

Each hypothesis space produces a different best-fit hypothesis.

This shows that the learner's result is shaped both by:

- the model class chosen
- and the specific sample of data observed

## 12. Four Example Hypothesis Spaces in the Figure

### Column 1: Straight Lines

Functions of the form:

$$h(x)=w_1x+w_0h(x)=w_1x+w_0h(x)=w_1x+w_0$$

This hypothesis space is very simple.

### Key property

No line can exactly fit all the data points, so this space cannot produce a consistent hypothesis here.

## Interpretation

This model may miss important patterns in the data.

## Column 2: Sinusoidal Functions

Functions of the form:

$$h(x) = w_1 x + \sin(w_0 x)$$

This space is more expressive than straight lines.

### Key property

It is not exactly consistent, but it fits both data sets well.

## Interpretation

This may be a good balance between flexibility and generalization for this problem.

## Column 3: Piecewise-Linear Functions

These connect the training points directly with line segments.

### Key property

They are always consistent because they pass through every training example exactly.

## Interpretation

They fit the given data perfectly, but consistency alone does not guarantee they are a good guess for unseen data.

## Column 4: Degree-12 Polynomials

Functions of the form:

$$h(x) = \sum_{i=0}^{12} w_i x^i$$

A degree-12 polynomial can always fit 13 distinct points exactly.

### Key property

These are consistent with the training data.

But:

- they can behave wildly between or beyond the data points
- and small changes in the training set can lead to very different curves

## Interpretation

Perfect fit does not mean good generalization.

### 13. Bias

Bias means, loosely:

- the tendency of a predictive hypothesis to systematically deviate from the expected value when averaged over different training sets

In simpler terms:

- bias comes from the restrictions built into the model class

#### Example of High Bias

A linear hypothesis space has strong bias because it can only represent straight lines.

If the true pattern contains:

- curvature
- cycles
- local structure

then the linear model cannot capture it well.

So the model is biased toward overly simple explanations.

#### Main Intuition

Bias reflects the limitations imposed by the hypothesis space itself.

### 14. Underfitting

A model is **underfitting** when it fails to find an important pattern in the data.

This usually happens when:

- the hypothesis space is too simple
- or too restrictive

Example:

- fitting a straight line to strongly nonlinear data

Underfitting is strongly associated with:

- **high bias and low variance**

### 15. Variance

The second major concept is **variance**.

Variance means:

- how much the learned hypothesis changes when the training data changes slightly

If small changes in the training set cause big changes in the learned function, then the model has high variance.

### Example of High Variance

The degree-12 polynomial examples change dramatically between the two rows of the figure, even though both data sets come from the same true function.

This means:

- the model is highly sensitive to the particular sample
- it is unstable across training sets

That is high variance.

### Low Variance

The first three model classes change less from one sample to another.

That means they have lower variance.

## 16. Overfitting

A model **overfits** when it pays too much attention to the particular training set and performs poorly on unseen data.

This typically happens when:

- the model is too flexible
- and starts fitting random fluctuations or noise rather than the real pattern

Overfitting is strongly associated with:

- **high variance and low bias**

## 17. Bias–Variance Tradeoff

Often there is a **bias–variance tradeoff**.

This means:

- simpler models usually have **higher bias** but **lower variance**
- more complex models usually have **lower bias** but **higher variance**

So there is a tension between:

- fitting the training data well
- and generalizing well to unseen data

### Example

- A straight line may be too simple and underfit
- A degree-12 polynomial may fit perfectly but overfit
- A moderate-complexity model may strike the right balance

This tradeoff is one of the most central ideas in supervised learning.

### Meaning in Machine Learning

Among hypotheses that fit the data reasonably well:

- prefer the simpler one

This does **not** mean “always choose the simplest possible model.”

It means:

- choose simplicity **without sacrificing necessary explanatory power**

## 19. Simplicity Is Hard to Define

It may seem obvious that:

- a polynomial with 2 parameters is simpler than one with 13 parameters
- very large and complex models, such as deep neural networks with billions of parameters, can still generalize well

So the number of parameters alone is **not always a perfect measure** of model quality.

### Better Idea: Appropriateness

The text suggests that maybe the goal should be:

- not simply “simplicity”
- but rather **appropriateness**

That means choosing a model class that is suitable for the actual structure of the problem.

## 20. Which Hypothesis Is Best?

The section says that from Figure 19.1 alone, we cannot be sure which hypothesis is best.

The answer depends on domain knowledge.

### If the data is cyclic

A sinusoidal model may make sense.

**If the data is not cyclic but noisy**

A linear model might be preferable.

So model evaluation depends on both:

- data
- and knowledge about the domain

## 21. Probabilistic View of Supervised Learning

In some cases, instead of asking whether a hypothesis is merely possible or impossible, we can ask:

**How probable is this hypothesis given the data?**

Then the goal becomes choosing:

$$h^* = \operatorname{argmax}_{h \in H} P(h | \text{data}) \quad h^* = \operatorname{argmax}_{h \in H} P(h | \text{data})$$

That is:

- choose the hypothesis that is most probable given the observed data

Using Bayes' rule, this is equivalent to:

$$h^* = \operatorname{argmax}_{h \in H} P(\text{data} | h) P(h) \quad h^* = \operatorname{argmax}_{h \in H} P(\text{data} | h) P(h)$$

This means the best hypothesis balances:

- how well it explains the data
- and how plausible it was beforehand

## Prior Probability of a Hypothesis

This framework allows us to encode a preference for smoother, less extreme functions.

For example:

- a smooth low-degree polynomial might have high prior probability
- a wildly oscillating degree-12 polynomial might have lower prior probability

So Bayesian model selection naturally expresses a preference for reasonable hypotheses, while still allowing complex ones if the data strongly demands them.

## 22. Expressiveness vs. Computational Complexity

The section asks:

### **Why not let the hypothesis space be all computer programs or all Turing machines?**

The answer is that there is a tradeoff between:

- **expressiveness**
- and **computational complexity**

### **More Expressive Spaces**

A very expressive hypothesis space can represent many complicated functions.

But this creates problems:

- searching it may be computationally hard or impossible
- evaluating the learned hypothesis may be slow or nonterminating

### **Simpler Spaces**

Simple spaces such as linear functions are:

- easier to search
- easier to compute with
- easier to use after learning

This is one reason machine learning often focuses on restricted model classes.

### **Important Point**

A more expressive language can sometimes let a simple hypothesis describe the data, whereas a less expressive language may require a more complicated hypothesis.

## **23. Restaurant Waiting Example**

The section introduces a sample supervised learning problem:

### **Task**

Predict whether we should **wait for a table at a restaurant**.

### **Output Variable**

The output `yyy` is a Boolean variable called:

### **WillWait**

- true if we decide to wait
- false otherwise



So this is a **classification problem**, because the output is one of a finite set of values:

- Yes / No

## 24. Input Attributes for the Restaurant Problem

The input `xxx` is a vector of **10 discrete attributes**.

These are:

1. **Alternate**  
Whether there is a suitable alternative restaurant nearby
2. **Bar**  
Whether the restaurant has a comfortable bar area to wait in
3. **Fri/Sat**  
Whether it is Friday or Saturday
4. **Hungry**  
Whether we are hungry now
5. **Patrons**  
Number of people in the restaurant  
Values: None, Some, Full
6. **Price**  
Restaurant's price range  
Values: \$, \$\$, \$\$\$
7. **Raining**  
Whether it is raining outside
8. **Reservation**  
Whether we made a reservation
9. **Type**  
Kind of restaurant  
Values: French, Italian, Thai, Burger
10. **WaitEstimate**  
Host's estimate of the wait time  
Values: 0–10, 10–30, 30–60, >60

So each example in this problem is a factored input vector of these 10 attributes.

## 25. Training Examples in the Restaurant Domain

The book gives **12 labeled examples** of restaurant situations.

Each example includes:

- values for the 10 attributes
- the correct output: WillWait = Yes or No

## 26. Why This Example Shows the Essence of Induction

The text emphasizes how little data we actually have.

There are:

- $2^6 \times 3^2 \times 4^2 = 9,216$  possible input combinations

But only:

- 12 examples are labeled

That means:

- for the remaining 9,204 possible cases,
- we do not know the correct output

This is exactly why learning is hard.

### **Main Point**

The learner must generalize from a tiny sample of labeled cases to a huge space of possible inputs.

That is the essence of **induction**:

- infer the unknown outputs from a small amount of evidence

### **Training Set**

A set of labeled input-output examples used to learn a hypothesis.

### **Hypothesis**

The learned function  $h$  that approximates the true function  $f$ .

### **Hypothesis Space**

The set of possible hypotheses the learner is allowed to choose from.

### **Ground Truth**

The true output label/value in the training data.

### **Exploratory Data Analysis**

Examining data with plots and statistics to understand it and help choose a model class.

### **Consistent Hypothesis**

A hypothesis that matches every training example exactly.

## Test Set

A separate sample of labeled examples used to measure performance on unseen data.

## Generalization

How well a hypothesis predicts outputs for new inputs not seen during training.

## Bias

Systematic tendency of a model class to miss certain patterns due to its restrictions.

## Variance

Sensitivity of the learned hypothesis to changes in the training data.

## Underfitting

Failure to capture real patterns in the data, usually due to overly simple models.

## Overfitting

Fitting the training data too closely, including noise, causing poor test performance.

## Bias–Variance Tradeoff

The balance between too-simple/high-bias models and too-flexible/high-variance models.

## Ockham's Razor

Prefer simpler hypotheses unless extra complexity is necessary.

## Textbook readings 19.3

A **decision tree** is a model for supervised learning that maps a vector of input attribute values to a single output value by asking a sequence of questions. It starts at the root, tests one attribute at a time, follows the corresponding branch, and eventually reaches a leaf that gives the final prediction.

It works by performing a sequence of tests:

1. start at the **root**
2. test the value of one attribute
3. follow the branch corresponding to that value
4. continue testing until a **leaf node** is reached
5. return the value stored in the leaf

## Structure of a Decision Tree

## Internal nodes

Each internal node tests one input attribute.

## Branches

Each branch is labeled with one of the possible values of that attribute.

## Leaf nodes

Each leaf gives the output value to return.

So a decision tree is essentially a structured sequence of conditional decisions.

## 2. Setting Used First in This Section

At first, the section focuses on a simpler case:

- input attributes are **discrete**
- output is **Boolean**

So the output is either:

- **true** = positive example
- **false** = negative example

This is called:

## Boolean classification

### Positive and Negative Examples

- A **positive example** is one where the output is true
- A **negative example** is one where the output is false

This terminology is very common in classification.

## 3. Notation

The section uses:

- $j$  to index examples
- $x_j$  = input vector for example  $j$
- $y_j$  = output for example  $j$
- $x_{j,i}$  = value of the  $i$ -th attribute in example  $j$

This notation is useful when describing how learning algorithms process data.

## 4. Restaurant Example in Tree Form

The restaurant waiting problem is used again here.

A decision tree for the restaurant problem is shown, where the tree classifies whether SR will wait for a table.

Example:

- if **Patrons = Full**
- and **WaitEstimate = 0–10**  
then the tree predicts:
- **Yes**, we will wait

This shows how a path from root to leaf represents a classification rule.

## 5. Expressiveness of Decision Trees

### Decision Trees as Logical Formulas

A Boolean decision tree is equivalent to a propositional logic statement of the form:

$$\text{Output} \Leftrightarrow (\text{Path}_1 \vee \text{Path}_2 \vee \dots) \text{Output} \rightarrow (\text{Path}_1 \vee \text{Path}_2 \vee \dots) \text{Output} \Leftrightarrow (\text{Path}_1 \vee \text{Path}_2 \vee \dots)$$

where each path is a conjunction of attribute-value tests:

corresponding to a root-to-leaf path ending in a true leaf.

So the whole tree corresponds to a **disjunctive normal form (DNF)** logical formula.

### Important Consequence

Because decision trees correspond to DNF, **any function in propositional logic can be represented as a decision tree.**

So decision trees are very expressive for Boolean logic functions.

## 6. Decision Trees Are Good for Some Functions, Bad for Others

Although decision trees can represent any propositional logic function, some functions require very large trees.

### Functions Hard to Represent Concisely

#### Majority function

Returns true if more than half the inputs are true.

#### Parity function

Returns true if an even number of inputs are true.

Both require **exponentially large decision trees**.

So while representation is possible, it may not be compact.

### Continuous Boundary Problem

With real-valued attributes, decision trees can struggle with functions whose decision boundary is diagonal or curved.

Example:

$$y > A_1 + A_2$$

This is hard for a decision tree because decision tree tests split space into **axis-aligned rectangular regions**, not diagonal boundaries.

So to approximate a diagonal line, the tree would need many little boxes.

### Main Lesson

Decision trees are:

- very good for some kinds of functions
- inefficient or awkward for others

## 8. Goal of Decision Tree Learning

We want to learn a decision tree that is:

1. **consistent** with the examples
2. as **small** as possible

Unfortunately:

- finding the guaranteed smallest consistent tree is **intractable**

So instead, we use heuristics to find a tree that is hopefully close to smallest.

## 9. Main Learning Strategy: Greedy Divide-and-Conquer

The algorithm **LEARN-DECISION-TREE** uses a:

**greedy divide-and-conquer strategy**

This means:

- choose what seems like the best attribute to test now
- split the examples based on that attribute

- recursively solve each smaller subproblem

It is greedy because:

- it makes the best local choice at each step
- without guaranteeing a globally optimal smallest tree

## 10. What Counts as the “Most Important” Attribute?

The algorithm picks the attribute that seems most useful for classification.

The “most important” attribute is:

- the one that makes the biggest difference to classifying the examples

The hope is:

- choose a very informative test early
- so paths are short
- and the overall tree is shallow

The exact measure of importance is developed later using **information gain**.

## 11. Example: Good vs. Bad Splits

### Type as a Poor Attribute

Splitting on **Type** is poor because:

- it creates four subgroups
- but each subgroup still has mixed positive and negative examples
- so it doesn’t move us much closer to a final decision

### Patrons as a Good Attribute

Splitting on **Patrons** is much better because:

- if Patrons = None → all examples are negative
- if Patrons = Some → all examples are positive
- only Patrons = Full remains mixed

So Patrons does a good job of separating the data.

## Lesson

A good attribute is one that creates “purer” subsets where the classification becomes more certain.

## 12. Four Cases in Recursive Decision Tree Learning

### Case 1: All Remaining Examples Have the Same Classification

If all examples are positive, return **Yes**.

If all are negative, return **No**.

Then we are done for that branch.

### Case 2: Mixed Positive and Negative Examples Remain

We must choose the best attribute to split on and continue recursively.

### Case 3: No Examples Left

This means the current combination of attribute values was not seen in the training data.

In that case, return the **most common output value** from the parent's examples.

This is a default guess for unseen cases.

### Case 4: No Attributes Left but Examples Still Conflict

This means:

- examples have the same description
- but different classifications

Possible reasons:

- noise in the data
- errors in labels
- nondeterminism in the domain
- missing/unobserved attributes that actually matter

Then the best we can do is return the **most common output value** among the remaining examples.

## 13. LEARN-DECISION-TREE Algorithm

The algorithm works as follows:

1. If no examples remain → return plurality value of parent examples
2. If all examples have same classification → return that classification
3. If no attributes remain → return plurality value of current examples
4. Otherwise:
  - choose the attribute with highest importance
  - create a tree with that attribute as the root test
  - for each value of the attribute:
    - collect matching examples
    - recursively learn a subtree



## PLURALITY-VALUE

This function returns the most common output value among a set of examples.

- If tied, break randomly

This is used when the data is insufficient or conflicting.

### 14. What the Learned Tree Tells Us

The learned tree from the 12 restaurant examples is different from the hand-crafted tree shown earlier.

This is **not** evidence that the algorithm failed.

Why not?

Because the learning algorithm only sees:

- the training examples

It does **not** know the original intended decision function.

So it tries to find:

- a tree consistent with the examples
- and preferably simple

The learned tree is:

- consistent with the data
- and actually simpler than the original hand-designed tree

This is a key lesson:

**The goal of learning is not to recover the teacher's exact internal reasoning, but to find a hypothesis that fits the observed examples and generalizes well.**

### 15. Interesting Effects of Learning from Data

The learner omits attributes such as:

- Raining
- Reservation

because they are not needed to classify the training examples.

It also finds a pattern such as:

- SR waits for Thai food on weekends

This may be a real pattern or an accidental one caused by limited data.

This shows how a learner can discover patterns, but also how it may form imperfect rules when data is scarce.

## **16. Limited Data Means Mistakes on Unseen Cases**

The section points out that the learner has never seen some combinations of attribute values.

So it may make mistakes there.

Example:

- it may misclassify a full restaurant with 0–10 minute wait if the relevant combination wasn't in the training data

More training data could correct such errors.

This reinforces the importance of:

- generalization
- sufficient data
- and learning curves

## **17. Learning Curves**

A **learning curve** plots performance versus training set size.

The example in Figure 19.7 shows:

- as the number of training examples increases
- test accuracy increases

This is why learning curves are sometimes called:

**“happy graphs”**

because they usually rise as more data is added.

### **Purpose of a Learning Curve**

A learning curve helps us understand:

- whether more data improves performance
- whether the algorithm is saturating
- how fast the learner improves with added experience

## **18. Choosing Attribute Tests with Information Gain**

The decision tree algorithm chooses the attribute with highest **IMPORTANCE**.

This importance is measured using:

### **information gain**

Information gain is based on:

### **entropy**

These are core ideas from information theory.

## **19. Entropy**

Entropy measures the **uncertainty** of a random variable.

### **Intuition**

- more uncertainty → higher entropy
- more certainty → lower entropy

### **Examples**

- A coin that always lands heads has entropy 0
- A fair coin has entropy 1 bit
- A fair 4-sided die has entropy 2 bits
- A very biased coin (99% heads) has entropy close to 0

### **Entropy Formula**

For a random variable  $V$  with values  $v_k$  and probabilities  $P(v_k)$ :

$$H(V) = -\sum_k P(v_k) \log_2 P(v_k)$$

This is the standard entropy formula.

### **Boolean Entropy Function**

For a Boolean variable true with probability  $q$ :

$$B(q) = -(q \log_2 q + (1-q) \log_2 (1-q))$$

This is just the entropy of a Boolean random variable.

## **20. Entropy in the Training Set**

If a training set has:

- ppp positive examples
- nnn negative examples

then the entropy of the output variable is:

$$H(\text{Output}) = -B(p) \log_2(p) - B(n) \log_2(n) = -B\left(\frac{p}{p+n}\right) \log_2\left(\frac{p}{p+n}\right) - B\left(\frac{n}{p+n}\right) \log_2\left(\frac{n}{p+n}\right)$$

In the restaurant training set:

- p=6, n=6
- n=6, p=6

So entropy is:

$$H(0.5) = -1 \log_2(0.5) - 1 \log_2(0.5) = 1$$

This means maximum uncertainty:

- before splitting, the class label is perfectly mixed

## 21. Remainder After Splitting on an Attribute

Suppose an attribute AAA has ddd distinct values.

Splitting the training set on AAA creates subsets:

$$E_1, E_2, \dots, E_d$$

Each subset has:

- p<sub>k</sub> positive examples
- n<sub>k</sub> negative examples

This is the weighted average entropy of the branches.

## Interpretation

- If the subsets are pure, remainder is low
- If the subsets remain mixed, remainder is high

So lower remainder means a better split.

## 22. Information Gain

Information gain is the reduction in entropy produced by splitting on an attribute:

$$\text{Gain}(A) = H(\text{Output}) - \text{Remainder}(A) = H(\text{Output}) - \sum_{k=1}^d \frac{|E_k|}{N} H(E_k)$$

So:

- start with uncertainty before the split
- subtract the expected uncertainty after the split

The bigger the gain, the more useful the attribute.

### Example

The section computes:

- **Gain(Patrons)**  $\approx 0.541$  bits
- **Gain(Type)** = 0 bits

So Patrons is much better than Type as the root split.

This matches intuition from the earlier figures.

## 23. Root Attribute Selection

The algorithm chooses the attribute with **maximum information gain** as the root.

In the restaurant example:

- **Patrons** has the highest gain
- so it becomes the root test

This is a key part of the greedy decision tree learning strategy.

## 24. Generalization and Overfitting in Decision Trees

As with other learning models, we want the tree to:

- fit the training data
- but also generalize well

Decision trees can overfit, especially when:

- they become too large
- training data is noisy
- there are many attributes
- the hypothesis space is large

### Capacity

Larger trees have more **capacity**:

- they can fit more patterns
- but also more noise

So capacity helps fit but also increases overfitting risk.

## 25. Decision Tree Pruning

To combat overfitting, we use:

### **decision tree pruning**

Pruning removes parts of the tree that do not seem genuinely useful.

### **How Pruning Works**

1. Start with the full learned tree
2. Look at a node whose descendants are all leaves
3. Decide whether that test is relevant or just modeling noise
4. If irrelevant, replace the whole test node with a leaf
5. Repeat until all such nodes are either kept or pruned

### **Goal of Pruning**

- remove unnecessary complexity
- improve generalization
- reduce sensitivity to noise
- create smaller, more understandable trees

## 26. How to Detect Irrelevant Splits

If an attribute is irrelevant, then after splitting:

- each branch should have about the same positive/negative proportion as the original node

That means:

- information gain should be close to zero

So low gain is a clue that the attribute may be irrelevant.

But we still need a principled rule for deciding how low is “too low.”

## 27. Statistical Significance Test

To decide whether a split is meaningful, we use a:

### **statistical significance test**

This starts with a:

### **null hypothesis**

The null hypothesis says:

- there is no real pattern
- the attribute is irrelevant

Then we check whether the observed deviation from irrelevance is statistically unlikely.

If the deviation would rarely occur by chance (commonly 5% or less), then we reject the null hypothesis and keep the split.

Otherwise, we prune it.

## 28. Expected Counts Under the Null Hypothesis

At a node with:

- ppp positive
- nnn negative

and a split into branches with totals  $p_k + n_k$  and  $p_{k+1} + n_{k+1}$ ,

the expected counts under irrelevance are:

$$\hat{p}_k = p \times \frac{p_k + n_k}{p + n} \quad \hat{p}_{k+1} = p \times \frac{p_{k+1} + n_{k+1}}{p + n}$$

$$\hat{n}_k = n \times \frac{p_k + n_k}{p + n} \quad \hat{n}_{k+1} = n \times \frac{p_{k+1} + n_{k+1}}{p + n}$$

These are what we would expect if the attribute had no real effect.

## 29. Chi-Squared Deviation Measure

The total deviation from expected counts is measured by:

$$\Delta = \sum_{k=1}^d \frac{(p_k - \hat{p}_k)^2}{\hat{p}_k} + \frac{(n_k - \hat{n}_k)^2}{\hat{n}_k} \quad \Delta = \sum_{k=1}^d \frac{(p_k - \hat{p}_k)^2}{\hat{p}_k} + \frac{(n_k - \hat{n}_k)^2}{\hat{n}_k}$$

Under the null hypothesis, this follows a:

### $\chi^2$ (chi-squared) distribution

with:

- $d - 1$  degrees of freedom

This lets us test whether the split is statistically significant.

## Chi-Squared Pruning

Using this significance test to prune is called:

### $\chi^2$ pruning

If the deviation is too small to reject irrelevance, prune the branch.

### 30. Why Pruning Helps

Pruning helps especially when there is noise in the data.

The section notes:

- pruned trees perform significantly better than unpruned trees when the data is noisy
- pruned trees are often smaller
- smaller trees are easier to understand
- smaller trees are also usually more efficient to execute

So pruning is both a statistical and practical improvement.

### 31. Why Early Stopping Is Dangerous

You might think:

- why not stop growing the tree early whenever no attribute looks useful?

This is called:

#### **early stopping**

But the book warns against relying on this instead of full grow-then-prune.

#### **Problem with Early Stopping**

Sometimes:

- no single attribute looks useful alone
- but combinations of attributes are informative

#### **Example: XOR**

For XOR of two binary attributes:

- neither attribute individually has high information gain
- but splitting on one and then the other works perfectly

Early stopping would miss this structure.

Generate-then-prune does better because it allows deeper combinations to appear before simplifying the tree.

### 32. Broadening Decision Trees to Real-World Problems

Real-world decision tree systems must handle more than the simple discrete-Boolean case.

The section lists several important complications.



### 33. Missing Data

Sometimes an example does not have all attribute values known.

This causes two problems:

1. How do we classify an example missing a tested attribute?
2. How do we compute information gain when some training examples are missing values?

Real-world systems must handle this explicitly.

### 34. Continuous and Multivalued Input Attributes

If an attribute is continuous, then every example may have a unique value.

If we split directly on each unique value, the tree becomes useless:

- many tiny branches
- no generalization to unseen values

#### Better Idea: Split Point Test

For continuous attributes, use an inequality test such as:

$Weight > 160$

This divides examples into two groups:

- above the split point
- below or equal to the split point

#### Finding Good Split Points

Efficient approach:

1. sort values of the attribute
2. only consider split points between consecutive examples with different classifications
3. keep running counts of positives and negatives on each side

This is a practical way to search for good continuous splits.

### 35. Multivalued Categorical Attributes

Some attributes are not continuous but have many possible values, like:

- Zipcode
- CreditCardNumber

Splitting directly on every distinct value can create many one-example branches.

To help avoid this:

- use **information gain ratio**
- or allow equality tests like:  
Zipcode=10002Zipcode = 10002Zipcode=10002  
vs “other”

This prevents exploding branching factors.

### **36. Continuous-Valued Output: Regression Trees**

If the output is numeric instead of categorical, we use a:

**regression tree**

Instead of a classification tree.

#### **Regression Tree Leaves**

A regression tree leaf contains:

- a linear function of some numerical attributes

instead of a single class label.

Example:

- for two-bedroom apartments, predict price using square footage and number of bathrooms

So regression trees combine:

- tree-based splitting
- local regression models at leaves

### **CART**

The term:

**CART = Classification And Regression Trees**

is used to cover both classification trees and regression trees.

### **37. Why Decision Trees Are Popular**

The section emphasizes several strengths of decision trees.

They have:

- ease of understanding
- scalability to large datasets
- versatility with discrete and continuous inputs
- applicability to both classification and regression

Because of these strengths, decision trees are often the **first method tried** in real-world classification problems.

### 38. Weaknesses of Decision Trees

Decision trees also have limitations.

#### 1. Suboptimal accuracy

Because learning uses a greedy search, it may not find the globally best tree.

#### 2. Expensive prediction for very deep trees

If a tree is deep, classifying a new example may take many tests.

#### 3. Instability

Decision trees are unstable:

- adding just one new training example can change the best root split
- which can change the entire tree structure

This sensitivity is one of their biggest drawbacks.

### LEARN-DECISION-TREE

At each node:

1. if examples are empty → return plurality of parent examples
2. if all examples same class → return that class
3. if no attributes left → return plurality of current examples
4. otherwise:
  - choose attribute with highest information gain
  - split examples on that attribute
  - recursively build subtrees

This is a greedy recursive algorithm.

### Exam-Focused Takeaways

- A decision tree classifies by asking a sequence of attribute-value tests from root to leaf.
- Boolean decision trees are equivalent to DNF logical formulas, so they can represent any propositional logic function.
- Some functions, such as majority and parity, require exponentially large trees.

- Decision tree learning seeks a small consistent tree, but finding the smallest one is intractable.
- The standard learning algorithm uses greedy divide-and-conquer, choosing the attribute with highest information gain at each step.
- Entropy measures uncertainty; information gain measures how much a split reduces uncertainty.
- Good attributes create purer subsets and therefore high information gain.
- Decision trees can overfit, especially when trees become large or data is noisy.
- Pruning reduces overfitting by removing statistically unhelpful tests.
- Chi-squared pruning tests whether a split is statistically significant under the null hypothesis of irrelevance.
- Early stopping is risky because combinations of weak attributes may still be informative.
- Real-world decision trees must handle missing data, continuous-valued inputs, many-valued categorical inputs, and continuous outputs.
- Regression trees and CART extend decision trees beyond simple Boolean classification.
- Decision trees are interpretable and practical, but they can be unstable and sometimes less accurate than other methods.

## **Textbook readings 19.4**

### **1. Main Goal of Machine Learning**

The ultimate goal of machine learning is:

**Select a hypothesis that will optimally fit future examples**

To formalize this, we must define:

1. What counts as “future examples”
2. What counts as “optimal fit”

### **2. Stationarity and i.i.d. Assumption**

#### **Stationarity Assumption**

We assume that:

- future data comes from the same distribution as past data

#### **Independence Assumption**

Each example is independent of previous ones:

#### **i.i.d. (Independent and Identically Distributed)**

Together, these assumptions mean:

- all examples are **independent and identically distributed**

## Why this matters:

Without i.i.d., learning from past data would not help predict future data.

### 3. Defining “Optimal Fit”

Initially, optimal fit is defined as:

#### Minimizing Error Rate

Error rate = proportion of times  $h(x) \neq y$   
Error rate = proportion of times  $h(x) = y$

This is the simplest performance measure.

### 4. Training, Validation, and Test Sets

To evaluate models properly, data must be split into **three distinct sets**:

#### 1. Training Set

- Used to **learn the model**
- Used to fit parameters

#### 2. Validation Set (Dev Set)

- Used to **compare models and tune hyperparameters**
- Helps choose the best model

#### 3. Test Set

- Used for **final unbiased evaluation**
- Must NOT be used during training or tuning

#### Key Rule:

Never use the test set until all decisions are finalized

Otherwise, you “leak” information and get overly optimistic results.

### 5. Hyperparameters

#### Definition

Hyperparameters are:

parameters of the model class, not the learned model itself

Examples:

- decision tree size
- polynomial degree
- pruning threshold

## Key Idea

Changing hyperparameters creates **different hypotheses**, so they must be evaluated carefully without contaminating the test set.

## 6. k-Fold Cross-Validation

When data is limited, we use:

### k-fold cross-validation

#### Procedure:

1. Split data into  $k$  equal parts
2. Repeat  $k$  times:
  - use 1 part as validation set
  - use remaining  $k-1$  parts as training set
3. average the validation errors

#### Common choices:

- $k=5$
- $k=10$

#### Special case:

### LOOCV (Leave-One-Out Cross-Validation)

- $k=n$
- each example is tested individually

#### Benefit:

- more efficient use of limited data
- gives a more reliable estimate of performance

## 7. Model Selection vs Optimization

The learning process has two key stages:

### 1. Model Selection

- choose the **hypothesis space**
- choose hyperparameters

Example:

- choose polynomial vs decision tree
- choose degree = 2 vs 10

## 2. Optimization (Training)

- find the **best hypothesis within that space**

**Key Idea:**

Model selection = “what kind of model?”

Optimization = “best model within that type”

## 8. MODEL-SELECTION Algorithm (Core Idea)

The algorithm:

1. try models of increasing complexity
2. evaluate each using cross-validation
3. pick the model with lowest validation error
4. retrain that model on full training data
5. evaluate once on test set

**Important:**

- validation set guides model choice
- test set is used only once at the end

## 9. Training vs Validation Error Behavior

Two common patterns appear:

### Pattern 1: U-Shaped Validation Curve

- training error decreases with complexity
- validation error decreases, then increases

 indicates **overfitting**

Optimal model = bottom of the “U”

### Pattern 2: Double Descent / Continued Improvement

- validation error decreases again at very high complexity

Seen in:

- deep neural networks
- random forests
- boosted models

### **Key Insight:**

More complexity doesn't always mean worse generalization — depends on model class.

## **10. Interpolation**

A model **interpolates the data** if:

- it fits all training examples perfectly

### **Important:**

- many models can interpolate (e.g., large trees, high-degree polynomials)
- interpolation often leads to overfitting
- but some modern models (deep learning) can still generalize well

## **11. Model Capacity**

**Capacity = flexibility of model class**

- low capacity → underfitting
- high capacity → risk of overfitting

### **Key Idea:**

Model selection balances:

- too simple (high bias)
- too complex (high variance)

## **12. Beyond Error Rate: Loss Functions**

Error rate is not always sufficient.

### **Example:**

Spam classification:

- misclassifying real email as spam is worse than the reverse

So we need:



## Loss Function

$$L(x, y, \hat{y})$$

Measures penalty for predicting  $\hat{y}$  instead of true  $y$

## Utility Perspective

Loss is defined as:

$$L = \text{utility of correct prediction} - \text{utility of predicted result}$$

## 13. Common Loss Functions

### 1. 0/1 Loss (classification)

$$L_{0/1}(y, \hat{y}) = \begin{cases} 0 & \text{if correct} \\ 1 & \text{if incorrect} \end{cases}$$

### 2. L1 Loss (absolute error)

$$L_1 = |y - \hat{y}|$$

### 3. L2 Loss (squared error)

$$L_2 = (y - \hat{y})^2$$

## Key Idea:

- small errors should cost less than large errors
- squared loss penalizes large errors more strongly

## 14. Generalization Loss

The true objective is:

### Expected Generalization Loss

$$\text{GenLoss}_L(h) = \sum_{(x,y)} L(y, h(x)) P(x, y)$$

## Best Hypothesis:

$$h^* = \arg\min_{h \in H} \text{GenLoss}_L(h)$$

## Problem:

We don't know  $P(x,y)P(x,y)P(x,y)$

## 15. Empirical Loss

So we approximate using training data:

$$\text{EmpLoss}(h) = \frac{1}{N} \sum_{(x,y) \in E} L(y, h(x))$$

### Learned Hypothesis:

$$\hat{h} = \underset{h}{\text{argmin}} \text{EmpLoss}(h)$$

### Key Idea:

We minimize empirical loss as an estimate of generalization loss.

## 16. Why Learned Model $\neq$ True Function

Even the best learned hypothesis may differ from the true function due to:

### 1. Unrealizability

- true function not in hypothesis space

Example:

- linear model trying to fit quadratic function

### 2. Variance

- different training sets  $\rightarrow$  different models
- decreases with more data

### 3. Noise

- same input may have different outputs
- cannot be predicted

### 4. Computational Limits

- cannot search entire hypothesis space
- settle for approximate solution

## 17. Small-Scale vs Large-Scale Learning

Small-scale learning:

- limited data
- main issues:
  - approximation error
  - estimation error (variance)

### Large-scale learning:

- massive datasets
- main issue:
  - computational constraints

## 18. Regularization

### Core Idea:

Add penalty for complexity:

$$\text{Cost}(h) = \text{EmpLoss}(h) + \lambda \cdot \text{Complexity}(h)$$

### Goal:

$$\hat{h}^* = \underset{h}{\text{argmin}} \text{Cost}(h) \quad \hat{h}^* = \underset{h}{\text{argmin}} \text{Cost}(h)$$

### $\lambda$ (lambda):

- controls tradeoff between:
  - accuracy
  - simplicity

### Interpretation:

- high  $\lambda \rightarrow$  simpler model
- low  $\lambda \rightarrow$  more complex model

### Purpose:

Prevent overfitting by discouraging overly complex hypotheses.

## 19. Regularization Functions

Depend on model type.

Example:

- for polynomials  $\rightarrow$  penalize sum of squared coefficients

 discourages “wiggly” functions

## 20. Feature Selection

Another way to simplify models:

### Feature selection

- remove irrelevant attributes

Example:

- $\chi^2$  pruning in decision trees

## 21. Minimum Description Length (MDL)

### Idea:

Choose hypothesis that minimizes total encoding length:

- bits to describe model
- bits to describe errors

### Interpretation:

- simpler models = shorter descriptions
- connects learning with information theory

## 22. Hyperparameter Tuning

Choosing hyperparameters is crucial.

### Methods:

#### 1. Hand-Tuning

- use intuition and experience
- iterative trial and error

#### 2. Grid Search

- try all combinations
- works when parameter space is small

#### 3. Random Search

- randomly sample parameter combinations
- works better for large or continuous spaces

#### 4. Bayesian Optimization

- treats tuning as a learning problem
- models function:  
 $y=f(x)$   
 where:
  - $xxx$  = hyperparameters
  - $yyy$  = validation loss
- balances:
  - exploration
  - exploitation

## 5. Population-Based Training (PBT)

- trains multiple models in parallel
- evolves hyperparameters over generations
- combines:
  - random search
  - learning from past runs

## 23. Key Tradeoffs

### Bias–Variance Tradeoff

- simple models → high bias
- complex models → high variance

### Expressiveness vs Computation

- more expressive models → harder to optimize

### Accuracy vs Complexity

- regularization balances these

## Textbook readings 19.5

### 1. Main Questions of Learning Theory

The section begins by listing foundational questions:

- How can we be confident that a learned hypothesis predicts well on unseen inputs?
- How many examples are needed?
- What hypothesis space should we choose?
- How do we avoid overfitting?

These are the central questions of learning theory.

## 2. Learning Curves Are Useful but Problem-Specific

But learning curves only describe:

- one specific algorithm
- on one specific problem

## 4. PAC Learning: Core Idea

The key theoretical idea is:

**A seriously wrong hypothesis will probably be exposed by enough examples.**

If a hypothesis makes many mistakes, then:

- it is unlikely to remain consistent with a large sample

Therefore:

- any hypothesis that remains consistent with enough examples is unlikely to be very wrong

This idea leads to:

## Probably Approximately Correct (PAC)

## 5. Meaning of PAC

A hypothesis is **PAC** if it is:

- **Probably** correct: with high probability
- **Approximately** correct: error is small, not necessarily zero

So PAC learning does not promise perfect recovery of the true function. Instead, it promises that with enough examples, the learner will likely find a hypothesis whose error is acceptably small.

## 6. PAC Learning Algorithm

A **PAC learning algorithm** is one that returns hypotheses that are probably approximately correct.

The usefulness of PAC theory is that it gives:

- performance guarantees
- bounds on required training set size

## 7. Assumptions Needed for PAC Theory

PAC results depend on assumptions. The “juice” that allows us to reason from past data to future data comes from these assumptions.

The section uses:

### A. Stationarity / i.i.d. Assumption

Future examples are drawn from the same fixed distribution as past examples.

This is the same assumption introduced earlier in model selection.

We do **not** need to know what the distribution is — only that it does not change.

### B. Deterministic True Function

The section assumes:

- the true target function  $f$  is deterministic

So the same input always has the same correct output.

### C. Realizability Assumption

The section also assumes:

- the true function  $f$  is actually in the hypothesis space  $H$

This simplifies the theory.

## 8. Error Rate in PAC Learning

For Boolean functions, the section uses **0/1 loss**, so the key quantity is the error rate of a hypothesis:

$$\text{error}(h) = \text{GenLoss}_{L_{0/1}}(h) = \sum_{x,y} L_{0/1}(y, h(x)) P(x,y) \quad \text{error}(h) = \text{GenLoss}_{\{L_{0/1}\}}(h) = \sum_{x,y} L_{\{0/1\}}(y, h(x)) P(x,y)$$

This means:

**error(h) = probability that h misclassifies a new example**

So PAC learning is mainly concerned with bounding the true generalization error.

## 9. Approximately Correct

A hypothesis  $h$  is called:

**approximately correct**

if:

$$\text{error}(h) \leq \epsilon$$

where:

- $\epsilon$  is a small positive constant

This means the error must be small, though not necessarily zero.

## 10. The $\epsilon$ -Ball Around the True Function

We can think of all approximately correct hypotheses as being close to the true function  $f$ .

This region is called the:

### $\epsilon$ -ball

around  $f$

It contains all hypotheses whose error is at most  $\epsilon$ .

### Outside the $\epsilon$ -Ball

The rest of hypothesis space is:

### $H_{\text{bad}}$

These are the “seriously wrong” hypotheses whose error is greater than  $\epsilon$ .

PAC analysis tries to show that, after enough examples, hypotheses in  $H_{\text{bad}}$  are unlikely to remain consistent.

## 11. Probability That a Bad Hypothesis Survives

Suppose  $h \in H_{\text{bad}}$ , so:

$$\text{error}(h) > \epsilon$$

Then the probability that  $h$  agrees with one random example is at most:

$$1 - \epsilon$$

Because the examples are independent, the probability that it agrees with **all  $N$  examples** is at most:

$$(1 - \epsilon)^N$$

This is the probability that a seriously wrong hypothesis “gets lucky” and looks consistent on all training examples.



## 12. Probability That Any Bad Hypothesis Survives

There may be many bad hypotheses in  $H_{\text{bad}}$ .

Using a union bound, the probability that **at least one** bad hypothesis remains consistent is:

$$P(H_{\text{bad}} \text{ contains a consistent hypothesis}) \leq |H_{\text{bad}}| (1-\epsilon)^N$$

$$P(H_{\text{bad}} \text{ contains a consistent hypothesis}) \leq |H_{\text{bad}}| (1-\epsilon)^N$$

Since  $H_{\text{bad}} \subseteq H$ ,

$$\leq |H| (1-\epsilon)^N \leq |H| (1-\epsilon)^N$$

So the chance of being fooled depends on:

- the size of the hypothesis space
- the number of examples
- the accuracy threshold  $\epsilon$

## 13. PAC Bound on Number of Examples

We want this bad event to happen with probability at most  $\delta$ , where  $\delta$  is another small constant:

$$|H| (1-\epsilon)^N \leq \delta \implies |H| (1-\epsilon)^N \leq \delta$$

Using the bound  $1-\epsilon \leq e^{-\epsilon}$ , we get:

$$N \geq \frac{1}{\epsilon} (\ln \delta + \ln |H|)$$

This is the central PAC sample complexity bound in this section.

## 14. Sample Complexity

The number of examples required is called the:

### sample complexity

It depends on:

- $\epsilon$ : desired accuracy
- $\delta$ : desired confidence
- $|H|$ : size of the hypothesis space

### Interpretation

We need more examples if:

- we want lower error ( $\epsilon$  smaller)
- we want higher confidence ( $\delta$  smaller)
- the hypothesis space is larger ( $|H|$  larger)

This is one of the most important takeaways of learning theory.

## 15. Why Very Large Hypothesis Spaces Are Hard

Consider  $H$  = all Boolean functions on  $n$  Boolean attributes.

Then:

$$|H| = 2^{2^n}$$

This is enormous.

Substituting into the PAC bound shows sample complexity grows like:

$$2n2^{2n}$$

That is essentially as large as the number of possible input examples.

### Main Consequence

If the hypothesis space includes **all** Boolean functions, then generalization is impossible unless you see almost all possible examples.

Why?

Because the hypothesis space is so expressive that for any finite set of examples, there are still many hypotheses that fit those examples but disagree arbitrarily on unseen inputs.

So a huge unrestricted hypothesis space destroys generalization.

## 16. Three Ways to Escape This Problem

The section gives three ways out of this dilemma.

### 1. Bring in Prior Knowledge

If we know something about the domain, we can restrict the hypothesis space in a meaningful way.

### 2. Prefer Simple Hypotheses

Rather than accept any consistent hypothesis, prefer a simple one.

This matches ideas from:

- decision tree learning

- regularization
- Ockham's razor

This often improves sample complexity.

### 3. Restrict the Hypothesis Space to a Learnable Subset

Instead of all Boolean functions, focus on a smaller, structured class of functions.

Then:

- search becomes easier
- generalization improves
- PAC sample complexity is much smaller

The section now explores this third approach through **decision lists**.

## 17. Decision Lists

A **decision list** is a sequence of tests, where each test is a conjunction of literals.

The list is processed in order:

1. apply the first test
2. if it succeeds, return the associated output
3. if it fails, move to the next test
4. continue until a test succeeds
5. if none succeed, use the default outcome

So a decision list is simpler in overall structure than a decision tree:

- it branches in one direction only

But each test itself can be more complex.

## 18. Example of a Decision List

For the restaurant domain, the section gives a decision list equivalent to:

$$\text{WillWait} \Leftrightarrow (\text{Patrons} = \text{Some}) \vee (\text{Patrons} = \text{Full} \wedge \text{Fri/Sat})$$

Meaning:

- if Patrons = Some  $\rightarrow$  Yes
- otherwise, if Patrons = Full and Fri/Sat  $\rightarrow$  Yes
- otherwise  $\rightarrow$  No

This shows how decision lists represent Boolean rules as an ordered sequence.

## 19. k-DL

If each test is restricted to be a conjunction of at most  $k$  literals, we get:

### k-DL

This is the class of decision lists whose tests contain at most  $k$  conjunctions/literals.

The restaurant example above is in:

- **2-DL**

because one test uses at most 2 literals.

## 20. Relationship to Decision Trees

The section notes that:

### k-DL includes k-DT

where:

- $k$ -DT = decision trees of depth at most  $k$

So decision lists are related to restricted-depth decision trees.

## 21. Goal: Show That k-DL Is Learnable

To prove PAC learnability, we must:

1. count how many hypotheses are in the hypothesis space
2. show that the number is manageable enough for PAC sample complexity to be polynomial

## 22. Counting the Number of Decision Lists

Let:

- $\text{Conj}(n, k)$  = set of conjunctions of at most  $k$  literals over  $n$  Boolean attributes

The number of such conjunctions is:

$$|\text{Conj}(n, k)| = \sum_{i=0}^k \binom{n}{i} = O(n^k) \quad |\text{Conj}(n, k)| = \sum_{i=0}^k \binom{n}{i} = O(n^k)$$

Then the total number of  $k$ -decision lists is bounded by:

$$|\text{k-DL}(n)| \leq 3^c \quad \text{where } c = |\text{Conj}(n, k)| \quad |\text{k-DL}(n)| \leq 3^c \quad \text{where } c = |\text{Conj}(n, k)|$$

After simplification:

$$|k\text{-DL}(n)| = 2^{O(nk \log_2(nk))} \quad |k\text{-DL}(n)| = 2^{O(n^k \log_2(n^k))}$$

This is much smaller than the space of all Boolean functions.

### 23. Sample Complexity for k-DL

Substituting this into the PAC bound gives:

$$N \geq \frac{1}{\epsilon} (\ln \frac{1}{\delta} + O(nk \log_2(nk))) \quad N \geq \frac{1}{\epsilon} (\ln \frac{1}{\delta} + O(n^k \log_2(n^k)))$$

So for small  $\epsilon$ , the number of examples needed is:

**polynomial in  $n$**

This is a major success:

- k-decision lists can be PAC-learned with a reasonable number of examples

### 24. Efficient Algorithm for Learning Decision Lists

The section gives a greedy learning algorithm:

#### DECISION-LIST-LEARNING

It repeatedly finds a test that matches a nonempty subset of examples where all examples have the same classification.

Then:

- add that test to the decision list
- remove the matched examples
- recurse on the remaining examples

Repeat until no examples remain.

### 25. DECISION-LIST-LEARNING Algorithm Structure

The algorithm works like this:

1. If examples are empty  $\rightarrow$  return default “No” decision list
2. Find a test  $t$  that matches a nonempty subset of examples and all matched examples have the same class
3. If the matched examples are positive  $\rightarrow$  attach output Yes
4. If matched examples are negative  $\rightarrow$  attach output No
5. Recursively learn the rest of the decision list on remaining examples

So it is a greedy covering algorithm:

- each step explains some examples with one rule
- then removes them

## 26. Choice of Test

The theory only needs:

- any consistent test

But in practice, it makes sense to prefer:

- smaller tests
- that match larger uniformly classified subsets

This tends to produce more compact and understandable decision lists.

The simplest strategy is:

- choose the smallest test that matches any uniformly classified subset

Even this works reasonably well.

## 27. Performance of Decision Lists vs Decision Trees

The section shows a learning curve comparing:

- DECISION-LIST-LEARNING
- LEARN-DECISION-TREE

For the restaurant problem:

- decision trees learn slightly faster
- decision lists have a bit less variation
- both reach over 90% accuracy after 100 trials

So decision lists are competitive, though decision trees may do slightly better on this domain.

## 28. Big Theoretical Message

The key lesson of this section is:

**Generalization becomes possible when the hypothesis space is restricted appropriately.**

If the hypothesis space is too large, PAC sample complexity becomes huge.

If we use a structured, learnable subset such as k-decision lists, then:

- the number of required examples becomes polynomial
- learning is feasible

- generalization is mathematically justified

This is one of the most important themes in learning theory.

## High-Yield Concepts to Know

### Computational Learning Theory

The mathematical study of when and how learning is possible.

### PAC Learning

Probably Approximately Correct learning:

- probably = with high probability
- approximately correct = low error

### PAC Learning Algorithm

An algorithm that returns a probably approximately correct hypothesis.

### Error Rate

Probability that a hypothesis misclassifies a new example.

### Approximately Correct

A hypothesis with error at most  $\epsilon$ .

### $\epsilon$ -Ball

Set of hypotheses whose error is at most  $\epsilon$ .

### $H_{\text{bad}}$

Set of seriously wrong hypotheses with error greater than  $\epsilon$ .

### Sample Complexity

Number of training examples required to achieve PAC guarantees.

### Decision List

Ordered sequence of conjunction tests with associated outputs.

### k-DL

Decision list where each test contains at most  $k$  literals.

## Most Important Formulas

### Error rate

$$\text{error}(h) = \sum_{x,y} L_{0/1}(y, h(x)) P(x, y) \quad \text{error}(h) = \sum_{x,y} L_{0/1}(y, h(x)) P(x, y)$$

### Probability bad hypothesis survives N examples

$$P(h \text{ agrees with } N \text{ examples}) \leq (1 - \epsilon)^N \quad P(h \text{ agrees with } N \text{ examples}) \leq (1 - \epsilon)^N$$

### Probability some bad hypothesis survives

$$P(\text{Hbad contains a consistent hypothesis}) \leq |H| (1 - \epsilon)^N \quad P(\text{Hbad contains a consistent hypothesis}) \leq |H| (1 - \epsilon)^N$$

### PAC sample complexity bound

$$N \geq \frac{1}{\epsilon} (\ln \delta + \ln |H|) \quad N \geq \frac{1}{\epsilon} (\ln \delta + \ln |H|)$$

### Number of conjunctions of size at most k

$$|\text{Conj}(n, k)| = \sum_{i=0}^k \binom{2n}{i} = O(n^k) \quad |\text{Conj}(n, k)| = \sum_{i=0}^k \binom{2n}{i} = O(n^k)$$

### Size of k-decision list hypothesis space

$$|k\text{-DL}(n)| = 2^{O(n^k \log_2(n^k))} \quad |k\text{-DL}(n)| = 2^{O(n^k \log_2(n^k))}$$

### Sample complexity for k-DL

$$N \geq \frac{1}{\epsilon} (\ln \delta + O(n^k \log_2(n^k))) \quad N \geq \frac{1}{\epsilon} (\ln \delta + O(n^k \log_2(n^k)))$$

## Textbook readings 19.6

This section introduces a new hypothesis space: **linear functions**. Unlike decision trees and decision lists, which split data into branches based on attribute tests, linear models predict outputs using weighted sums of input variables.

The section covers:

- **univariate linear regression** (fitting a straight line)
- **gradient descent** as a general optimization method
- **multivariable linear regression**
- **regularization** for controlling overfitting



- **linear classification with a hard threshold** (perceptron-style classification)
- **logistic regression**, which replaces the hard threshold with a soft one

This section is important because linear models are among the oldest, simplest, and most widely used tools in machine learning. They are mathematically elegant, computationally efficient, and often surprisingly effective.

## 1. Overall Shift from Decision Trees to Linear Models

Earlier sections focused on:

- decision trees
- decision lists

These are discrete, rule-based hypothesis spaces.

Now the text moves to:

- **linear functions of continuous-valued inputs**

This is a different kind of model class:

- instead of branching logic
- we compute a weighted sum of inputs

## 2. Linear Models Can Be Used for Both Regression and Classification

The section shows that linear functions can support two major tasks:

### Regression

Predict a numeric output

### Classification

Predict a category (often 0/1)

The same weighted-sum structure is used in both cases, but classification requires an additional thresholding or squashing step.

## 3. Univariate Linear Regression

### Definition

A **univariate linear function** has:

- one input  $x$
- one output  $y$

and takes the form:

$$y = w_1 x + w_0$$

where:

- $w_1$  = slope
- $w_0$  = intercept

## Weight Interpretation

The coefficients are called **weights** because they control how much each term contributes to the output.

The weight vector is:

$$\mathbf{w} = \langle w_0, w_1 \rangle$$

and the corresponding linear hypothesis is:

$$h_{\mathbf{w}}(x) = w_1 x + w_0$$

## 4. Linear Regression

The task of finding the best line for data is called:

### linear regression

Example:

- input  $x$ : house size in square feet
- output  $y$ : house price

The goal is to find the line that best fits the training points.

## 5. Loss Function for Linear Regression

The section uses the traditional **squared-error loss** (L2 loss), summed over all training examples:

$$\text{Loss}(\mathbf{w}) = \sum_{j=1}^N (y_j - h_{\mathbf{w}}(x_j))^2$$

Substituting  $h_{\mathbf{w}}(x_j) = w_1 x_j + w_0$ :

$$\text{Loss}(\mathbf{w}) = \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2$$

So linear regression chooses the weights that minimize the total squared error.

## 6. Optimal Weights for Univariate Linear Regression

We want to find:

$$w^* = \operatorname{argmin}_w \text{Loss}(h_w) \quad w^* = \arg\min_w \text{Loss}(h_w)$$

To do this analytically:

- take partial derivatives with respect to  $w_0$  and  $w_1$
- set them equal to zero
- solve the resulting equations

This yields a **unique solution**:

$$w_1 = \frac{N(\sum x_j y_j) - (\sum x_j)(\sum y_j)}{N(\sum x_j^2) - (\sum x_j)^2} \quad w_0 = \frac{(\sum y_j) - w_1(\sum x_j)}{N}$$

This is the standard closed-form solution for straight-line regression.

## 7. Weight Space

To understand optimization, the section introduces:

### weight space

For univariate linear regression, the weight space has two dimensions:

- one dimension for  $w_0$
- one dimension for  $w_1$

We can think of every possible line as one point in this 2D weight space.

The loss function can then be graphed over weight space, giving a 3D surface.

## 8. Convexity of the Linear Regression Loss

The loss surface for linear regression with L2 loss is:

### convex

This means:

- there is a single global minimum
- there are no local minima

This is a very important property.

**Why it matters:**

Optimization is easy because any descent method that works properly will eventually reach the global minimum.

## 9. Why Linear Regression Is Nice

For simple linear regression, things are especially clean:

- there is a unique best solution
- it can be found analytically
- the optimization problem is convex
- no local minima cause trouble

In that sense, fitting straight lines is one of the easiest and most elegant learning problems.

## 10. Gradient Descent

The section next introduces a much more general optimization method:

### gradient descent

This is important because many learning problems do not have easy closed-form solutions.

### Core Idea of Gradient Descent

1. Start at any point in weight space
2. Compute the gradient of the loss
3. Move a small step in the direction of steepest descent
4. Repeat until convergence

Gradient descent attempts to reach the **global minimum**—the absolute lowest point of a loss function—because this represents the parameters that produce the lowest possible error in a model. It acts as a search algorithm, iteratively moving in the direction of the steepest descent to minimize prediction errors.

The update rule is:

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(w)$$

where:

- $\alpha$  is the **learning rate**

## 11. Learning Rate

The parameter  $\alpha$  controls how big each update step is.

This is called the:

## learning rate

- Too large → may overshoot the minimum
- Too small → learning is slow

It can be:

- fixed
- or gradually decreased over time

## 12. Chain Rule Reminder

The derivation of gradient descent updates depends on the:

## 13. Gradient Descent for One Example in Linear Regression

For one training example  $(x, y)$ , the squared error is:

$$(y - h_w(x))^2$$

Taking derivatives yields:

$$\begin{aligned} \frac{\partial \text{Loss}}{\partial w_0} &= -2(y - h_w(x)) \\ \frac{\partial \text{Loss}}{\partial w_1} &= -2(y - h_w(x))x \end{aligned}$$

So the update rule becomes:

$$\begin{aligned} w_0 &\leftarrow w_0 + \alpha(y - h_w(x)) \\ w_1 &\leftarrow w_1 + \alpha(y - h_w(x))x \end{aligned}$$

## Intuition

If prediction is too high:

- reduce weights accordingly

If prediction is too low:

- increase weights accordingly

Each update nudges the line toward better agreement with the training point.

## 14. Batch Gradient Descent

For all NNN training examples, we sum the contributions from all examples:

$$\begin{aligned} w_0 &\leftarrow w_0 + \alpha \sum_j (y_j - h_w(x_j)) \\ w_1 &\leftarrow w_1 + \alpha \sum_j (y_j - h_w(x_j))x_j \end{aligned}$$

This is called:

### **batch gradient descent**

also called:

### **deterministic gradient descent**

because each update uses the entire training set.

#### **Advantage**

- guaranteed convergence for convex loss (if learning rate is reasonable)

#### **Disadvantage**

- slow when dataset is large
- each step requires processing all examples

## **15. Epoch**

A full pass over all training examples is called an:

### **epoch**

In batch gradient descent, each step effectively uses a full epoch.

This term is used heavily in later machine learning material.

## **16. Stochastic Gradient Descent (SGD)**

To speed things up, we can use:

### **stochastic gradient descent (SGD)**

Instead of computing the gradient from the whole training set at each step, SGD uses:

- one randomly chosen example  
or more commonly:
- a small random subset of examples

### **Minibatch SGD**

Modern SGD often uses a:

### **minibatch**

of size mmm

This means:

- sample mmm examples
- compute approximate gradient from those
- update weights

## **Why Minibatches Help**

Compared with full batch:

- much cheaper per step
- still gives a useful estimate of the gradient
- often faster overall

Also:

- works well with parallel hardware (CPU/GPU vectorization)

So minibatch size becomes a tunable hyperparameter.

## **17. Convergence of SGD**

Unlike batch gradient descent:

- minibatch SGD is not guaranteed to settle exactly at the minimum
- it may oscillate near the minimum

But if the learning rate decreases over time appropriately:

- convergence can be guaranteed in theory

This is similar in spirit to:

- simulated annealing

## **18. SGD as Online Gradient Descent**

SGD is especially useful in an:

### **online setting**

where new data arrive continuously.

Then the model can:

- update incrementally
- adapt to changes in the environment
- gradually forget some past information while learning from new information

This is also called:

## online gradient descent

### 19. SGD Beyond Linear Regression

The section notes that SGD is widely used not just for linear regression, but also for:

- neural networks
- other large-scale machine learning models

Even when the loss surface is not convex, SGD often works surprisingly well in practice.

This is one reason SGD is such a central algorithm in modern machine learning.

### 20. Multivariable Linear Regression

Now extend linear regression to inputs with multiple dimensions.

Suppose each example  $x_j$  is an  $n$ -dimensional vector.

The hypothesis becomes:

$$h_w(x_j) = w_0 + w_1 x_{j,1} + \dots + w_n x_{j,n} \\ h_w(x_j) = w_0 + w_1 x_{j,1} + \dots + w_n x_{j,n}$$

or more compactly:

$$h_w(x_j) = \sum_i w_i x_{j,i} \\ h_w(x_j) = \sum_i w_i x_{j,i}$$

with  $x_{j,0} = 1$  to absorb the intercept.

#### Dot Product Form

Using  $x_{j,0} = 1$ , we can write:

$$h_w(x_j) = w \cdot x_j = w^T x_j \\ h_w(x_j) = w \cdot x_j = w^T x_j$$

This is the standard compact form.

### 21. Gradient Descent for Multivariable Regression

The update rule becomes:

$$w_i \leftarrow w_i + \alpha \sum_j (y_j - h_w(x_j)) x_{j,i} \\ w_i \leftarrow w_i + \alpha \sum_j (y_j - h_w(x_j)) x_{j,i}$$

This is the multivariable version of the same gradient idea.

### 22. Matrix Form of Multivariable Regression



Let:

- $y$  = vector of training outputs
- $X$  = **data matrix**, with one example per row

Then:

- predicted outputs are  $\hat{y} = Xw$

The total squared error is:

$$L(w) = \|Xw - y\|^2$$

Setting the gradient to zero gives:

$$\nabla_w L(w) = 2X^T(Xw - y) = 0$$

which yields the solution:

$$w^* = (X^T X)^{-1} X^T y$$

This is the standard matrix solution.

## 23. Data Matrix

The matrix  $X$  containing one example per row is called the:

**data matrix**

This is a standard term in machine learning and statistics.

## 24. Pseudoinverse and Normal Equation

The expression:

$$(X^T X)^{-1} X^T$$

is called the:

**pseudoinverse**

And the full equation:

$$w^* = (X^T X)^{-1} X^T y$$

is called the:

**normal equation**

These are important formulas for linear regression.

## 25. Overfitting in Multivariable Regression

With one input dimension, overfitting is less of a concern.

But in high-dimensional multivariable regression:

- some dimensions may appear useful just by chance
- the model may fit irrelevant features
- this causes overfitting

So multivariable regression commonly uses:

### regularization

## 26. Regularization in Linear Models

We want to minimize total cost:

$$\text{Cost}(h) = \text{EmpLoss}(h) + \lambda \text{Complexity}(h) \quad \text{Cost}(h) = \text{EmpLoss}(h) + \lambda \text{Complexity}(h)$$

For linear models, complexity is expressed as a function of the weights.

### Lq Regularization Family

$$\text{Complexity}(h_w) = L_q(w) = \sum_i |w_i|^q \quad \text{Complexity}(h_w) = L_q(w) = \sum_i |w_i|^q$$

Important special cases:

#### L1 regularization

$$\sum_i |w_i| \quad \sum_i |w_i| \quad \sum_i |w_i|$$

#### L2 regularization

$$\sum_i w_i^2 \quad \sum_i w_i^2 \quad \sum_i w_i^2$$

## 27. L1 vs L2 Regularization

### L1 Regularization

- tends to produce **sparse models**
- often sets many weights exactly to zero
- effectively performs feature selection

### L2 Regularization

- shrinks weights toward zero

- but usually does not make them exactly zero

## Why L1 Produces Sparsity

Geometrically:

- L1 constraint region is diamond-shaped
- optimum often lands on a corner
- corners correspond to one or more zero weights

L2 constraint region is circular:

- no special preference for axes
- so exact zero weights are less likely

## Main Consequence

L1 regularization is especially good when many features are irrelevant, because it can zero them out entirely.

This makes models:

- simpler
- more interpretable
- often better at avoiding overfitting

## 28. Rotational Invariance Intuition

The text gives another way to understand L1 vs L2.

### L2

- rotationally invariant
- treats all directions equally

This makes sense when coordinate axes are arbitrary.

### L1

- not rotationally invariant
- treats axes specially

This makes sense when each feature has a meaningful identity, like:

- number of bathrooms
- lot size

In many real data problems, feature axes are meaningful, so L1 can be particularly useful.

## 29. Linear Classification with a Hard Threshold

Linear functions can also classify.

Suppose we compute:

$$w \cdot x = w_1 x_1 + w_2 x_2 + \dots + w_n x_n$$

Then classify by checking whether it is above or below zero.

The classifier is:

$$h_w(x) = \begin{cases} 1 & \text{if } w \cdot x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

This is equivalent to applying a:

### threshold function

$$\text{Threshold}(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

## 30. Decision Boundary and Linear Separator

The set of points where:

$$w \cdot x = 0$$

forms the:

### decision boundary

If this boundary is a straight line or hyperplane, it is called a:

### linear separator

### Linear Separability

Data are: **linearly separable**

if there exists a linear separator that perfectly separates the classes.

This is an important concept in classification. It allows simple, efficient algorithms—like logistic regression, linear SVMs, and perceptrons—to perfectly separate classes using a straight line or hyperplane. This simplicity provides high interpretability, lower computational costs, and acts as a foundational, "fruit-fly" benchmark for understanding classification models.

### Why Linear Separability Matters:

- **Algorithmic Efficiency and Simplicity:** When data is linearly separable, basic linear classifiers can find an optimal decision boundary easily. Examples include Linear Discriminant Analysis (LDA) and the Perceptron algorithm.
- **Model Selection:** Knowing if data is linearly separable guides the choice of model. If data is separable, linear models prevent overfitting, which is more common in complex, non-linear models.
- **Interpretability:** Linear models produce simple decision rules (e.g.,  $w_0 + w_1x_1 + \dots + w_nx_n > 0$ ), making it easier to interpret which features are driving classification decisions.
- **Optimal Boundaries (SVMs):** For linearly separable data, Support Vector Machines (SVMs) can find the maximum margin hyperplane—a line that offers the best separation safety margin between classes.
- **Foundation for Complex Problems:** While many real-world problems are not linearly separable, they are often transformed (using methods like kernel tricks) into higher-dimensional spaces where they *are* linearly separable.

If data is not linearly separable, complex models like kernel-based SVMs or deep neural networks are needed to handle nonlinear surfaces.

### 31. Perceptron Learning Rule

To learn a hard-threshold linear classifier, the section introduces the:

#### perceptron learning rule

For one example  $(x, y)$ :

$$w_i \leftarrow w_i + \alpha(y - h_w(x))x_i$$

This looks almost identical to the regression update rule.

#### Behavior of the Perceptron Rule

Since both  $y$  and  $h_w(x)$  are 0 or 1:

##### Case 1: Correct output

No weight change

##### Case 2: True label = 1, prediction = 0

Increase  $w \cdot x$  so future prediction is more likely to be 1

##### Case 3: True label = 0, prediction = 1

Decrease  $w \cdot xw \cdot x$  so future prediction is more likely to be 0

This is intuitive error-driven learning.

### 32. Perceptron Convergence

If the data are linearly separable:

- the perceptron rule is guaranteed to converge to a perfect separator

This is a classic result.

#### Limitation

If the data are **not** linearly separable:

- the perceptron may never converge
- it may keep changing weights forever
- it may oscillate even if it repeatedly visits good solutions

This is a major weakness of hard-threshold classification.

### 33. Learning Rate Schedules for Perceptron

If the learning rate decreases as:

$$\alpha(t) = O(1/t) \quad \alpha(t) = O(1/t)$$

then convergence to a minimum-error solution can be shown under random example presentation.

So decaying learning rates improve stability.

### 34. Problem with Hard Thresholds

The hard threshold classifier has several issues:

1. **nondifferentiable**
2. gradient is zero almost everywhere and undefined at the threshold
3. learning becomes unstable
4. outputs are always completely confident (0 or 1), even near the boundary

These motivate the move to:

#### logistic regression

### 35. Logistic Function

The logistic function is:

$$\text{Logistic}(z) = \frac{1}{1 + e^{-z}}$$

It is a smooth approximation to the threshold function.

It maps any real number into:

- a value between 0 and 1

### Advantages

- continuous
- differentiable
- interpretable as probability
- more stable for learning

## 36. Logistic Regression

Using the logistic function, the hypothesis becomes:

$$h_w(x) = \text{Logistic}(w \cdot x) = \frac{1}{1 + e^{-w \cdot x}}$$

This is called:

### logistic regression

Even though it is used for classification, the historical name is “regression.”

## 37. Interpretation of Logistic Regression Output

The output of logistic regression is between 0 and 1, so it can be interpreted as:

### probability of belonging to class 1

This gives a **soft boundary**:

- points far from the boundary have probabilities near 0 or 1
- points near the boundary have probabilities near 0.5

This is a major improvement over hard-threshold classification.

## 38. Gradient Descent for Logistic Regression

There is no easy closed-form solution like ordinary linear regression.

But gradient descent is straightforward.

For one example  $(x, y)$ , the update becomes:

$$w_i \leftarrow w_i + \alpha (y - h_w(x)) h_w(x) (1 - h_w(x)) x_i$$

$$w_i \leftarrow w_i + \alpha (y - h_w(x)) h_w(x) (1 - h_w(x)) x_i$$

This comes from:

- L2 loss
- chain rule
- derivative of logistic function

### 39. Why Logistic Regression Is Better Than Hard Thresholding

Compared with perceptron learning, logistic regression:

- converges more smoothly
- behaves more predictably
- works much better when data are noisy or not linearly separable
- provides probabilistic outputs rather than just 0/1

The section shows that:

- in linearly separable cases, logistic regression is a bit slower
- but in noisy, nonseparable cases it is much more stable and effective

This is why logistic regression has become extremely popular in practice.

### 40. Applications of Logistic Regression

The text says logistic regression is widely used in:

- medicine
- marketing
- survey analysis
- credit scoring
- public health
- and many other applied fields

This is one of the most widely used classification methods in real-world machine learning.

## High-Yield Concepts to Know

### Linear Regression

Learning a linear function that predicts a numeric output.

### Univariate Linear Regression

Linear regression with one input variable.



## **Weight Vector**

The vector of coefficients  $w$ .

## **Weight Space**

The space of all possible weight settings.

## **Convex Loss Surface**

Loss surface with one global minimum and no local minima.

## **Gradient Descent**

Optimization method that moves weights downhill along the gradient.

## **Learning Rate**

Step size parameter  $\alpha$ .

## **Batch Gradient Descent**

Uses all examples per update.

## **Epoch**

One full pass through all training examples.

## **Stochastic Gradient Descent (SGD)**

Uses one or a few random training examples per update.

## **Minibatch**

Small subset of training examples used for an SGD step.

## **Online Gradient Descent**

SGD used in an ongoing data stream setting.

## **Data Matrix**

Matrix containing input examples as rows.

## **Pseudoinverse**

$$(X^T X)^{-1} X^T (X^T X)^{-1} X^T$$

## **Normal Equation**

$$w^* = (X^T X)^{-1} X^T y \quad w^* = (X^T X)^{-1} X^T y$$

## **L1 Regularization**

Sum of absolute weights; encourages sparsity.

## **L2 Regularization**

Sum of squared weights; shrinks weights but usually not to zero.

## **Sparse Model**

A model with many zero weights.

## **Decision Boundary**

Set of points where classifier changes class.

## **Linear Separator**

Linear boundary separating two classes.

## **Linear Separability**

Existence of a perfect linear separator.

## **Threshold Function**

Hard 0/1 cutoff function.

## **Perceptron Learning Rule**

Update rule for hard-threshold linear classification.

## **Logistic Function**

$$\frac{1}{1 + e^{-z}}$$

## **Logistic Regression**

Linear classification using logistic function output.

## **Most Important Formulas**

### **Univariate linear hypothesis**

$$h_w(x) = w_1 x + w_0$$

## Squared-error loss

$$\text{Loss}(h_w) = \sum_j (y_j - h_w(x_j))^2 \quad \text{Loss}(h_w) = \sum_j (y_j - h_w(x_j))^2$$

## Closed-form univariate solution

$$w_1 = \frac{N(\sum x_j y_j) - (\sum x_j)(\sum y_j)}{N(\sum x_j^2) - (\sum x_j)^2} \quad w_0 = \frac{(\sum y_j) - w_1(\sum x_j)}{N}$$

## Gradient descent update

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(w) \quad w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(w)$$

## Multivariable linear hypothesis

$$h_w(x) = w^T x \quad h_w(x) = w^T x$$

## Multivariable regression solution

$$w^* = (X^T X)^{-1} X^T y \quad w^* = (X^T X)^{-1} X^T y$$

## Perceptron classifier

$$h_w(x) = \begin{cases} 1 & \text{if } w \cdot x \geq 0 \\ 0 & \text{otherwise} \end{cases} \quad h_w(x) = \begin{cases} 1 & \text{if } w \cdot x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

## Perceptron update

$$w_i \leftarrow w_i + \alpha (y - h_w(x)) x_i \quad w_i \leftarrow w_i + \alpha (y - h_w(x)) x_i$$

## Logistic regression hypothesis

$$h_w(x) = \frac{1}{1 + e^{-w \cdot x}} \quad h_w(x) = \frac{1}{1 + e^{-w \cdot x}}$$

## Logistic derivative

$$g'(z) = g(z)(1 - g(z)) \quad g'(z) = g(z)(1 - g(z))$$

## Logistic regression update

$$w_i \leftarrow w_i + \alpha (y - h_w(x)) h_w(x) (1 - h_w(x)) x_i \quad w_i \leftarrow w_i + \alpha (y - h_w(x)) h_w(x) (1 - h_w(x)) x_i$$

## Perceptron info:

<https://www.geeksforgeeks.org/deep-learning/what-is-perceptron-the-simplest-artificial-neural-network/>

Support Vector Machines (SVMs) are powerful, supervised machine learning algorithms used primarily for classification and regression tasks by finding the optimal hyperplane that separates data points into different categories. They are known for their high accuracy, efficiency on smaller datasets, and effectiveness in high-dimensional spaces, such as image or text recognition.

### How Support Vector Machines Work

SVMs identify the "best" hyperplane, maximizing the margin (gap) between the closest data points, known as Support Vectors. Key elements include the Margin and, for complex, non-linear data, the Kernel Trick, which maps data into higher-dimensional space.

- Kernels & Parameters: Kernels like Linear, Polynomial, and RBF (Gaussian) manage non-linear data, while Regularization (C) and Slack Variables ( ) balance the trade-off between maximizing the margin and allowing for misclassification.
- Types of SVM: Includes Linear/Non-linear SVMs, One-class SVM (for anomaly detection), and SVR for regression.
- Applications: Commonly applied in image classification, text categorization, bioinformatics, and fraud detection.

### Advantages and Disadvantages

- Advantages: High accuracy in high-dimensional spaces, memory efficiency (only uses support vectors), and versatile kernel choices.
- Disadvantages: Computationally expensive for large datasets, sensitive to noise/outliers, and challenging to interpret (black box).

A **single-layer perceptron** is primarily used for binary classification. It acts as a linear classifier, creating a decision boundary to separate data into two classes. While it can act as a linear regressor in some scenarios, its most common application is in binary classification problems.

Key Details:

- Purpose: It functions as a [binary classifier](#) (e.g., classifying data as a label with two outcome choices).

- Method: It computes a weighted sum of inputs and applies a threshold, creating a linear separation.
- Limitation: It is limited to linearly separable data.
- Extension: While a single perceptron is used for classification, multi-layer networks ([Multi-Layer Perceptrons - MLPs](#)) can perform complex classification and regression.

A [binary perceptron](#) is a fundamental, single-layer supervised learning algorithm for binary classification, acting as a linear classifier that separates data into two classes (e.g., +1/-1) using a linear decision boundary. It calculates a weighted sum of inputs plus a bias, applying a step function to predict labels, and updates weights only when misclassification occurs, making it ideal for linearly separable. It is designed to make "yes/no" or "0/1" decisions by determining if an input belongs to a specific class based on a set of features. Finding a Decision Boundary: The algorithm aims to find a [separating hyperplane](#) (a straight line in 2D) that perfectly divides the two classes in the feature space. Automated Learning: Through a supervised learning process, it iteratively adjusts internal weights and biases whenever it makes a misclassification, gradually improving its accuracy.

A **multiclass perceptron** generalizes the binary perceptron to classify data into multiple categories by maintaining separate weight vectors for each class. It predicts by choosing the class with the highest score (dot product) and updates weights by boosting the correct class vector and reducing the incorrect one upon errors.

### Key Concepts and Mechanisms

- Architecture: It typically uses one weight vector for each class. The prediction is made using an "argmax" function, selecting the class with the largest dot product between its weight vector and the input feature vector.
- Training/Update Rule: If a prediction is incorrect the weights are updated:
  - Subtract the input vector
  - from the weight vector of the mistakenly predicted class: .
  - Add the input vector
  - to the weight vector of the actual target class: .
- Terminating Condition: If the classes are linearly separable, the algorithm will eventually converge, meaning it will perfectly classify the training set.
- Approaches: Beyond training one vs. all, the multiclass perceptron often uses a holistic approach, simultaneously tuning all classifiers.

- Multilayer Perceptron (MLP): While a single-layer multiclass perceptron uses direct inputs, a multilayer perceptron (or neural network) adds hidden layers to learn non-linear patterns, using activation functions like ReLU.

### Advantages and Limitations

- Simplicity: The algorithm is simple, fast, and easy to implement.
- Linear Separation: It only works if the classes are linearly separable.
- No Probabilities: The standard multiclass perceptron does not produce class probabilities; it only outputs the winning class.

No, the standard perceptron activation function (the [binary step function](#)) is not differentiable. It is a discrete step function, meaning it has a discontinuity at 0 and its derivative is zero everywhere else, preventing the use of **gradient-based** optimization methods like backpropagation.

Key details regarding perceptron differentiability:

- Step Function Problem: The perceptron computes  $\sum w_i x_i$ , where  $\sigma$  is the Heaviside step function. This function has a sharp jump at 0, making it impossible to compute a meaningful derivative for optimization.
- Alternative Activation Functions: To enable **gradient-based learning**, modern neural networks (multilayer perceptrons) replace the step function with continuous, differentiable functions like [sigmoid](#), [tanh](#), or [ReLU](#).
- Learning without Differentiability: While modern deep learning requires differentiable activations, the classic single-layer perceptron learning algorithm manages weights through direct updates rather than calculating gradient slopes.

## Textbook readings Jurafsky and Martin 4.1

### 1. Why Classification Is Central to Language Processing

The section begins by stressing that:

**Classification lies at the heart of language processing and intelligence**

Many intelligent tasks can be viewed as deciding which category an input belongs to.

Examples given include:

- recognizing a letter

- recognizing a word
- recognizing a face
- sorting mail
- assigning grades to homeworks

All of these are examples of taking an input and assigning it to a class.

## 2. Borges Example: Why Classification Can Be Hard

The text mentions Jorge Luis Borges's famous imaginary encyclopedia that classified animals into bizarre categories such as:

- those that belong to the Emperor
- embalmed ones
- mermaids
- those that have just broken a flower vase
- those that resemble flies from a distance

The point of this example is:

**Classification is only useful when the classes themselves are coherent and meaningful**

In NLP, thankfully, our classes are usually much better defined than Borges's absurd categories.

So while classification is essential, choosing sensible categories matters too.

## 3. Text Categorization

The chapter focuses on: **text categorization**

This means:

- assigning a label or category to a text or document

Examples of text categorization tasks:

- sentiment analysis
- spam detection
- language identification
- authorship attribution

This is one of the central application areas of classification in NLP.

## 4. Sentiment Analysis

The section especially highlights:

**sentiment analysis**

Sentiment analysis is the task of deciding:

- whether a writer expresses a positive or negative attitude toward something

This “something” might be:

- a movie
- a restaurant
- a product
- a political candidate
- a policy or action

So sentiment analysis tries to determine the **orientation** or **stance** expressed in a text.

### Why Sentiment Analysis Matters

Sentiment is useful in many fields, including:

- marketing
- politics
- social analysis
- public opinion tracking

Because people frequently express their evaluations in text, sentiment analysis is a major real-world classification problem.

## 5. Words as Informative Features for Sentiment

The section notes that in binary sentiment classification, certain words can be highly informative.

Examples of positive-indicating words:

- awesome
- love

Examples of negative-indicating words:

- awful
- ridiculously

So texts like:

- “awesome caramel sauce and sweet toasty almonds. I love this place!”  
tend to signal positive sentiment

while texts like:

- “awful pizza and ridiculously overpriced...”  
tend to signal negative sentiment



This motivates the idea that classification can often be based on detecting useful features in the input.

## **6. Other Important Text Classification Tasks**

The section names several other common classification tasks.

### **Spam Detection**

Assign an email to one of two classes:

- spam
- not spam

This is a classic binary classification problem.

### **Language Identification**

Determine which language a text is written in.

So the classes are languages such as:

- English
- French
- Zulu
- Amharic
- etc.

This is a multiclass classification problem.

### **Authorship Attribution**

Determine who wrote a text.

This is useful in:

- literary analysis
- forensic analysis
- historical authorship questions

## **7. Language Modeling Can Also Be Viewed as Classification**

The idea is:

- each possible next word is like a class
- predicting the next word means classifying the current context into one of those word classes

So even language modeling can be reframed as classification.

## Why This Matters

This insight helps connect:

- text classification
- logistic regression
- neural networks
- large language models

The section says this idea underlies large language models.

So classification is not just one NLP task among many — it is deeply connected to the larger structure of language modeling itself.

## 8. Why Logistic Regression Is Important

The chapter introduces:

### logistic regression

as the main algorithm for classification in this chapter. Logistic regression is a supervised machine learning algorithm used for binary classification—predicting the probability of a categorical outcome (e.g., yes/no, true/false, pass/fail) based on input data. It fits an S-shaped [sigmoid curve](#) to map linear input data to probabilities between 0 and 1, usually applying a 0.5 threshold to classify results.

The text emphasizes that logistic regression is important in several ways.

### A. Connection to Neural Networks

A neural network can be viewed as:

- multiple logistic regression classifiers stacked together

So logistic regression is a conceptual building block for later neural network models.

### B. Introduces Core Functions

Logistic regression introduces important concepts used later in NLP and deep learning:

- sigmoid function
- softmax function
- logit

These are foundational tools.

### C. Introduces Gradient Descent

The chapter uses logistic regression to introduce:

- gradient descent
- learning by optimization

These are core ideas in machine learning.

## D. Important Beyond NLP

The text notes that logistic regression is also one of the most important analytical tools in:

- social sciences
- natural sciences
- many applied fields

So logistic regression is both theoretically central and practically widespread.

## 9. What Classification Is

The section formally defines the goal of classification as follows:

- take a single input observation
- extract useful features from it
- assign it to one of a fixed set of discrete classes

Each input is called an:

**observation**

### Notation

Let:

- input =  $x$
- output classes =  $Y = \{y_1, y_2, \dots, y_M\}$

The goal is:

- given  $x$ , return the correct class from  $Y$

Sometimes the set of classes is also written as:

- $C$

So:

- $Y$  or  $C$  = label set
- $x$  = input observation

## 10. Examples of Input and Output Classes

## Sentiment Analysis Example

Input:

- a review or some other text

Output classes:

$\{\text{positive}, \text{negative}\} \setminus \{\text{positive}, \text{negative}\} \setminus \{\text{positive}, \text{negative}\}$

or equivalently:

$\{0, 1\} \setminus \{0, 1\} \setminus \{0, 1\}$

So sentiment analysis is often treated as binary classification.

## Language ID Example

Input:

- some text whose language is unknown

Output classes:

- the set of possible languages

This is a much larger multiclass problem.

## 11. Different Ways to Do Classification

The section mentions several possible approaches to classification.

### A. Handwritten Rules

Example rule:

- if the word “love” appears
- and it is not preceded by “don’t”
- classify as positive

This illustrates a rule-based approach.

### Advantages of Rules

- interpretable
- can be useful as components of systems

### Weaknesses of Rules

- fragile
- may fail as language or situations change

- hard for humans to anticipate all interactions
- difficult to scale to complex tasks

So handwritten rules are useful but limited.

## B. Prompting a Large Language Model

Another possible approach is:

- ask a large language model to assign a class

This can be powerful, but the section notes important weaknesses:

- large language models may hallucinate
- they may not be able to explain their classifications clearly

So prompting is useful, but not always the best method if reliability and transparency matter.

## C. Supervised Machine Learning

The section says the most common way to do classification is **supervised machine learning**

## 12. Supervised Machine Learning for Classification

In supervised machine learning, we have:

1. the input observations
2. the possible output classes
3. a **labeled training set**
4. a **learning algorithm**

The model learns from labeled examples how to map new inputs to their correct class.

### Labeled Training Set

A training set is labeled when each input example is paired with the correct output class.

The training set has the form:

where:

- $x^{(i)}$  = input observation
- $y^{(i)}$  = correct label for that observation

The superscript in parentheses indexes examples in the training set.

The correct label  $y^{(i)}$  acts as the:

**supervision signal**

It tells the learning algorithm what the correct answer should be.

The learner's goal is to use the training set to build a: **classifier**

A classifier is a function that maps a new input  $x$  to a class  $y \in Y$  in  $Y$ .

In text classification, this often means learning which features of the text are predictive of the label.

Example:

- words like “awesome” or “awful” may become useful features for sentiment prediction

## 14. Probabilistic Classifiers

The section points out that logistic regression is a: **probabilistic classifier**

That means it does not just output a class label. It also outputs:

**the probability that the observation belongs to each class**

### Why Probabilities Matter

Probabilities are valuable because:

- they show confidence
- they preserve uncertainty information
- they are useful for downstream systems
- they avoid forcing a hard decision too early

So probabilistic classifiers provide richer information than simple yes/no outputs.

## 15. Four Components of Probabilistic Classifiers Like Logistic Regression

### 1. Feature Representation of the Input

Each input observation  $x^{(i)}$  is represented as a feature vector:

$[x_1, x_2, \dots, x_n]$

These are measurable or detectable properties of the input.

The section notes that feature notation may appear as:

- $x_i$
- $x^{(j)}$
- $f_i$
- $f_i(x)$
- or, for multiclass cases,  $f_i(c, x)$

So features are the basis for making predictions.

## 2. Classification Function

The classifier computes:

$$P(y=y_i | x)$$

for each possible class  $y_i$ .

This means:

- given the input, estimate the probability of each class
- the **sigmoid** function
- the **softmax** function

as tools for turning numeric scores into probabilities.

## 3. Objective Function

The model must define something to optimize during learning.

Usually this means:

- minimizing a **loss function**

The chapter will introduce:

### **cross-entropy loss**

which is one of the standard loss functions for classification.

## 4. Optimization Algorithm

Once we know the loss, we need a way to minimize it. **stochastic gradient descent**

This is the optimization algorithm used to train the classifier.

## 16. Two Phases of Logistic Regression

At the highest level, logistic regression has two main phases.

### **Training Phase**

During training:

- the model learns its parameters
- specifically, it learns weights  $w$  and bias  $b$
- it uses:
  - stochastic gradient descent

- cross-entropy loss

So training means adjusting the parameters to make the classifier perform well on the labeled training set.

## Test Phase

At test time:

- a new example  $x$  is given
- the model computes:  
 $P(y=y_i | x)$  for each output class
- then it selects the class with highest probability

So the model produces a probability distribution over classes and then chooses the most likely one.

## 17. Binary vs Multiclass Logistic Regression

The section explains that logistic regression can be used for:

### Binary classification

Two classes, such as:

- positive / negative
- spam / not spam

### Multiclass classification

More than two classes, such as:

- language identification across many languages

### Why Start With Binary Case?

The chapter first introduces the:

- two-class case

because:

- the mathematics is simpler

Then it will later extend to:

- multinomial logistic regression
- softmax



for the multiclass case.

## **18. Sigmoid, Softmax, and Logit Preview**

The chapter signals that logistic regression introduces three especially important ideas.

### **Sigmoid**

Used in binary logistic regression to map a score to a probability between 0 and 1.

### **Softmax**

Used in multiclass classification to convert a set of scores into a probability distribution over classes.

### **Logit**

A related transformation and concept central to logistic regression.

## **19. Big Conceptual Takeaways**

### **Classification is Fundamental to NLP**

Many language tasks are classification tasks, either directly or indirectly.

### **Text Categorization Is a Core NLP Task**

Tasks such as sentiment analysis, spam filtering, language ID, and authorship attribution are all examples of assigning labels to text.

### **Logistic Regression Is Foundational**

It is important not only as a useful classifier but also because it introduces:

- probabilistic classification
- sigmoid and softmax
- gradient descent
- connections to neural networks

### **Supervised Learning Is the Main Framework Here**

The chapter works in the supervised setting:

- labeled training data
- learning from examples
- mapping from features to class probabilities

### **Probabilities Matter**

A classifier that outputs probabilities gives richer, more useful information than one that only outputs a hard label.

## **Classification**

Assigning an input to one of a fixed set of discrete classes.

## **Observation**

A single input example to be classified.

## **Text Categorization**

Assigning a text or document to a category.

## **Sentiment Analysis**

Classifying a text according to positive or negative sentiment.

## **Spam Detection**

Classifying email as spam or not spam.

## **Language Identification**

Determining the language of a text.

## **Authorship Attribution**

Determining the author of a text.

## **Supervised Machine Learning**

Learning from labeled examples of input-output pairs.

## **Labeled Training Set**

A collection of training examples, each paired with its correct class label.

## **Supervision Signal**

The correct label that guides learning.

## **Probabilistic Classifier**

A classifier that outputs probabilities over classes, not just a label.

## **Feature Representation**

A vector of measurable properties extracted from the input.

## **Classification Function**

The function that computes class probabilities.

## **Objective Function**

The function the model tries to optimize during training.

## **Cross-Entropy Loss**

A common loss function for probabilistic classification.

## **Stochastic Gradient Descent**

An optimization method used to train classifiers.

## **Textbook readings Jurafsky and Martin 4.2, and 4.3**

The goal of binary logistic regression is to train a classifier that can decide between two classes for a new input observation.

Examples:

- positive vs negative sentiment
- spam vs not spam
- end-of-sentence vs not end-of-sentence

So the task is binary classification:

- output  $y=1$  means the observation belongs to the positive class
- output  $y=0$  means it does not

The model wants to compute:

$$P(y=1 \mid x)$$

the probability that the input  $x$  belongs to class 1.

## **2. Input Representation as a Feature Vector**

A single input observation  $x$  is represented as a vector of features:

$$[x_1, x_2, \dots, x_n]$$

Each feature captures some property of the input.

Example in sentiment analysis:

- one feature might count positive words

- another might count negative words
- another might indicate whether “no” occurs
- another might count pronouns

So logistic regression does not work directly on raw text. It works on a feature representation of the text.

### 3. Learned Parameters: Weights and Bias

Logistic regression learns:

- a vector of weights  $\mathbf{w}$
- a bias term  $b$

#### Weights

Each feature  $x_i$  has an associated weight  $w_i$ .

The weight tells us:

- how important that feature is for the classification decision
- whether it supports the positive or negative class

Interpretation:

- $w_i > 0$ : feature gives evidence for the positive class
- $w_i < 0$ : feature gives evidence for the negative class

Example:

- word “awesome” might get a high positive weight
- word “abysmal” might get a very negative weight

So weights encode how predictive each feature is.

#### Bias Term / Intercept

The model also includes a real-valued: bias term also called the: intercept

This is an extra constant added to the weighted sum. It allows the classifier to shift the decision boundary even when all features are zero.

### 4. Weighted Sum of Evidence

For a test input, the classifier first computes:

$$z = \left( \sum_{i=1}^n w_i x_i \right) + b$$

This value  $z$  is the weighted sum of the evidence for the class.

It combines:

- the values of the input features
- the importance of each feature
- and the bias term

## 5. Dot Product Notation

The section rewrites the weighted sum using dot product notation:

$$z = w \cdot x + bz$$

The dot product means:

- multiply corresponding components
- add them together

So this is just a compact way to write the same weighted sum.

## 6. Why z Is Not Yet a Probability

The score  $z$  can take any real value:

$-\infty < z < \infty$  Because:

- weights are real-valued
- the weighted sum can be negative or arbitrarily large

So  $z$  is not itself a legal probability.

A probability must be between 0 and 1.

Therefore we need a transformation that maps any real number into the interval (0,1).

## 7. The Sigmoid Function

The solution is the: sigmoid function also called the: logistic function

Its formula is:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + \exp(-z)}$$

This is the central function in binary logistic regression.

### Why It Is Useful

The sigmoid function has several key properties:

1. Maps any real number to (0,1)

This makes it suitable for probabilities.

## 2. Nearly linear around 0

So moderate input values are preserved relatively smoothly.

## 3. Flattens at the extremes

Very large positive values map near 1.

Very large negative values map near 0.

So outlier scores get “squashed” toward the ends.

## 4. Differentiable

This is crucial for gradient-based learning later.

## 8. Probability of the Positive and Negative Classes

After applying the sigmoid, we can interpret the output as the probability of the positive class:

$$P(y=1) = \sigma(w \cdot x + b)$$

Then the probability of the negative class is:

$$P(y=0) = 1 - \sigma(w \cdot x + b)$$

So the two probabilities automatically sum to 1.

This gives a full probability distribution over the two classes.

## 9. Useful Sigmoid Identity

The section highlights an important identity:

$$1 - \sigma(x) = \sigma(-x)$$

So the negative-class probability can also be written as:

$$P(y=0) = \sigma(-(w \cdot x + b))$$

This is a useful algebraic property of the sigmoid.

## 10. The Logit

The score

$z = w \cdot x + b$  is often called the: logit

Why It Is Called the Logit

The logit function is the inverse of the sigmoid.

It is:

$$\text{logit}(p) = \sigma^{-1}(p) = \ln \frac{p}{1-p} \quad \text{logit}(p) = \sigma^{-1}(p) = \ln \frac{p}{1-p}$$

This is the log of the odds ratio.

So when we use sigmoid to convert  $z$  into a probability, we are implicitly interpreting  $z$  as a log-odds score.

That is a key conceptual point in logistic regression.

## 11. $\hat{y}$ Notation

For binary classification, the section introduces the notation:

$$\hat{y}$$

Pronounced: “y hat”

This means:

$$\hat{y} = P(y=1 | x) \quad \hat{y} = P(y=1 | x)$$

So in binary logistic regression:

- $\hat{y}$  is the predicted probability of the positive class

The probability of the negative class is then:

$$1 - \hat{y}$$

Later, in multiclass classification,  $\hat{y}$  will become a vector of probabilities over all classes.

## 12. Turning Probability into a Decision

The sigmoid gives us a probability, but a classifier must often return a class label.

The rule is:

$$\text{decision}(x) = \begin{cases} 1 & \text{if } P(y=1 | x) > 0.5 \\ 0 & \text{otherwise} \end{cases} \quad \text{decision}(x) = \begin{cases} 1 & \text{if } P(y=1 | x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

So the classifier predicts class 1 if the positive-class probability exceeds 0.5.

### Decision Boundary

The threshold value 0.5 is called the: decision boundary

This is the point where the model switches from class 0 to class 1.

### 13. Sentiment Classification Example

The section gives a concrete sentiment classification example for movie reviews.

The input text is represented with 6 features:

#### Features Used

$x_1$

Count of positive lexicon words in the document

$x_2$

Count of negative lexicon words in the document

$x_3$

Whether the word “no” appears:

- 1 if yes
- 0 otherwise

$x_4$

Count of first- and second-person pronouns

$x_5$

Whether “!” appears:

- 1 if yes
- 0 otherwise

$x_6$

Natural log of document length in words + punctuation

#### Example Feature Values

For the sample review, the extracted feature values are:

- $x_1 = 3$
- $x_2 = 2$
- $x_3 = 1$
- $x_4 = 3$
- $x_5 = 0$
- $x_6 = \ln(66) = 4.19$



So the feature vector is:

$$x = [3, 2, 1, 3, 0, 4, 19] \quad x = [3, 2, 1, 3, 0, 4, 19] \quad x = [3, 2, 1, 3, 0, 4, 19]$$

#### 14. Example Learned Weights

Assume the model has learned these weights:

$$[2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \quad [2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \quad [2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$$

and bias:

$$b = 0.1 \quad b = 0.1 \quad b = 0.1$$

Interpretation of the Weights

$$w_1 = 2.5 \quad w_1 = 2.5 \quad w_1 = 2.5$$

Positive lexicon words strongly support positive sentiment.

$$w_2 = -5.0 \quad w_2 = -5.0 \quad w_2 = -5.0$$

Negative lexicon words strongly support negative sentiment.

And since  $-5.0$  has a larger magnitude than  $2.5$ , negative words are about twice as influential here as positive words.

This shows how logistic regression weights can be interpreted directly.

#### 15. Example Probability Computation

The section computes:

$$P(+ | x) = \sigma(w \cdot x + b) \quad P(+ | x) = \sigma(w \cdot x + b) \quad P(+ | x) = \sigma(w \cdot x + b)$$

Substituting values:

$$P(+ | x) = \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4, 19] + 0.1) \quad P(+ | x) = \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4, 19] + 0.1) \quad P(+ | x) = \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4, 19] + 0.1)$$

which yields:

$$P(+ | x) = \sigma(0.833) = 0.70 \quad P(+ | x) = \sigma(0.833) = 0.70 \quad P(+ | x) = \sigma(0.833) = 0.70$$

So the model predicts:

- probability of positive sentiment = 0.70
- probability of negative sentiment = 0.30

Since  $0.70 > 0.5$ , the classifier predicts:

- positive

## 16. Logistic Regression for Other NLP Tasks

The section emphasizes that logistic regression can be used for many NLP tasks, not just sentiment.

One example given is:

period disambiguation

The task is to classify whether a period marks:

- end of sentence (EOS)
- or not end of sentence

So each period is classified into one of two classes.

Example Features for Period Disambiguation

$x_1x_{-1x1}$

Whether the current word is lowercase

$x_2x_{-2x2}$

Whether the current word is in an acronym dictionary

$x_3x_{-3x3}$

Whether the current word is “St.” and the previous word is capitalized

These show that features can express:

- local properties
- dictionary membership
- combinations of properties

## 17. Feature Interactions

The section introduces:

feature interactions

These are features that combine multiple primitive properties.

Example:

- “word is St.” AND “previous word is capitalized”

This interaction is useful because:

- “St.” may behave differently depending on context

Feature interactions allow the model to capture more subtle patterns than simple individual features alone.

## 18. Designing Features by Hand

In classic NLP systems, features are often:

- designed manually
- based on linguistic intuition
- refined using error analysis

This process may also draw on prior literature and domain expertise.

So feature engineering used to be a major part of NLP system design.

## 19. Feature Templates

The section also mentions: feature templates

These are abstract descriptions of feature families.

Example:

- a bigram template for period disambiguation might create a feature for every bigram seen before a period in training

So instead of manually specifying every possible feature, a template defines how a whole class of features should be generated automatically.

## Sparse Feature Space

This often leads to a:

sparse feature space

because only features that actually occur in the training data need to be created.

Feature names may then be hashed into unique integer IDs for efficient implementation.

## 20. Representation Learning

The section contrasts older hand-designed features with more modern approaches:

representation learning

This means:

- learning features automatically from data
- often in an unsupervised or partially supervised way

Modern NLP systems rely much more on learned representations than on manual feature engineering.

## 21. Scaling Input Features

When input features have very different numeric ranges, it is common to:

scale the features

This helps because:

- some features may otherwise dominate due to large magnitude
- optimization becomes easier
- gradient descent converges faster

This is especially important in large neural networks.

## 22. Standardization (z-score)

One common scaling method is:

standardization

This transforms a feature so that it has:

- mean 0
- standard deviation 1

For feature  $x_i$ :

Mean:

$$\mu_i = \frac{1}{m} \sum_{j=1}^m x_i(j)$$

Standard deviation:

$$\sigma_i = \sqrt{\frac{1}{m} \sum_{j=1}^m (x_i(j) - \mu_i)^2}$$

Then standardized feature:

$$x'_i = \frac{x_i - \mu_i}{\sigma_i}$$

This is also called the:

z-score transformation

## 23. Normalization to [0,1]

Another common approach is:

normalization

Scale features to lie between 0 and 1:

$$x_i' = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$$

This is useful when we want features to share a common finite range.

## 24. Log Scaling

For NLP data, another very common scaling trick is:

take logs

This is useful when counts follow a:

Zipfian distribution

which is common for:

- word counts
- bigram counts
- frequency-based NLP features

Log scaling compresses large values and makes distributions more manageable.

## 25. Processing Many Examples at Once

So far, equations were written for one example at a time.

But in practice, we often want to process an entire set of  $m$  examples efficiently.

One approach is:

- loop over examples one by one

$$\hat{y}^{(i)} = \sigma(w \cdot x^{(i)} + b)$$

for each example  $i$

This works, but is not the most efficient.

## 26. Matrix-Based Computation

A much faster approach is to process all examples together using matrix operations.

Input Matrix XXX

Pack all input feature vectors into a single matrix:

$$X = \begin{bmatrix} x_1(1) & x_2(1) & \dots & x_f(1) & x_1(2) & x_2(2) & \dots & x_f(2) & x_1(3) & x_2(3) & \dots & x_f(3) \end{bmatrix}$$
$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_f^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_f^{(2)} \\ x_1^{(3)} & x_2^{(3)} & \dots & x_f^{(3)} \end{bmatrix}$$
$$X = \begin{bmatrix} x_1(1) & x_1(2) & x_1(3) & \dots & x_2(1) & x_2(2) & x_2(3) & \dots & x_f(1) & x_f(2) & x_f(3) \end{bmatrix}$$

Each row corresponds to one input example.

So if:

- $m$  = number of examples
- $f$  = number of features

then XXX has shape:

$$[m \times f]$$

Output Vector

Let:

$$\mathbf{y} = [y^{(1)}, y^{(2)}, \dots, y^{(m)}]$$
$$\hat{\mathbf{y}} = [\hat{y}^{(1)}, \hat{y}^{(2)}, \dots, \hat{y}^{(m)}]$$

where each entry is the probability of class 1 for one input.

Bias Vector

Let  $\mathbf{b}$  be repeated into a vector of length  $m$ .

Matrix Equation

Then all outputs can be computed at once as:

$$\mathbf{y} = \sigma(\mathbf{X}\mathbf{w} + \mathbf{b})$$

This is the matrix version of logistic regression prediction.

## 27. Why Matrix Computation Matters

This matrix formulation computes the same thing as looping over examples one at a time, but it is much faster because:

- modern hardware is optimized for matrix operations
- vectorized computation is highly efficient
- this becomes essential for large datasets

So matrix notation is not just mathematically elegant — it is computationally critical.

## 28. Shapes in the Matrix Equation

The section gives the shapes explicitly:

$$\hat{y} = \sigma(Xw + b)$$

with shapes:

- $\hat{y}$ :  $(m \times 1)$
- $X$ :  $(m \times f)$
- $w$ :  $(f \times 1)$
- $b$ :  $(m \times 1)$

So the matrix multiplication is dimensionally valid and returns one predicted probability per example.

## 29. Convention About Rows vs Columns

The text notes that:

- inputs are conventionally represented as rows of  $X$

This is why the multiplication is written as:

$$Xw$$

rather than  $wX$ .

The other convention could work too, but row-wise input examples are standard.

$$\text{logit}(p) = \ln \frac{p}{1-p}$$

## Textbook readings Jurafsky and Martin 4.4 - 4.6

### 1. Learning in Logistic Regression

In binary logistic regression, for each training example:

- input =  $x$
- true label =  $y$ , where  $y \in \{0, 1\}$

The model computes:

- $\hat{y}$ , the estimated probability that  $y=1$

So:

- if the true label is  $y=1$ , we want  $\hat{y}$  to be close to 1
- if the true label is  $y=0$ , we want  $\hat{y}$  to be close to 0

Thus, learning means finding weights and bias so that predicted probabilities match the true labels as closely as possible.

## 2. Two Things Needed for Learning

To learn the parameters, logistic regression needs two components:

### A. A loss function

This measures how far the prediction  $\hat{y}$  is from the true label  $y$ .

The text emphasizes that instead of measuring similarity, machine learning usually measures:

- **distance**
- **error**
- **cost**

This is called the: **loss function** or **cost function**

### B. An optimization algorithm

Once we know how bad the model is, we need a way to improve it.

That means:

- adjusting weights and bias to reduce loss

The standard optimization algorithm used here is: **gradient descent**

More specifically, later sections focus on: **stochastic gradient descent (SGD)**

## 3. Cross-Entropy Loss

The standard loss function for logistic regression is: **cross-entropy loss** It is also described as:

**negative log likelihood loss**

The idea is:

- the model should assign high probability to the correct label
- if it assigns low probability to the correct label, that should produce high loss

## 4. Conditional Maximum Likelihood Estimation



The text says logistic regression chooses parameters by: **conditional maximum likelihood estimation**

This means:

- choose  $w$  and  $b$  that maximize the probability of the true labels given the inputs

In other words:

- among all possible parameter settings, pick the one that makes the observed training labels as probable as possible

Because optimization is usually phrased as minimization, we instead minimize the **negative** log-likelihood, which gives cross-entropy loss.

## 5. Probability of One Training Example

For one example, logistic regression predicts  $\hat{y} = P(y=1 | x)$ .  $\hat{y} = P(y=1 | x)$ .

Since there are only two classes, the probability of the true label can be written compactly as:

$$p(y | x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

This works because:

- if  $y=1$ , it becomes  $\hat{y}$
- if  $y=0$ , it becomes  $1 - \hat{y}$

So it is a compact formula covering both binary cases.

## 6. Log Likelihood for One Example

Take the log of the above probability:

$$\log p(y | x) = y \log \hat{y} + (1-y) \log (1 - \hat{y})$$

This is the log-likelihood for one training example.

The log is useful because:

- it turns products into sums
- it simplifies derivatives
- maximizing probability and maximizing log-probability are equivalent

## 7. Cross-Entropy Loss Formula

To turn log-likelihood into a loss, flip the sign:

$$L_{CE}(\hat{y}, y) = -\log p(y | x)$$

So:

$$LCE(y^{\wedge}, y) = -[y \log y^{\wedge} + (1-y) \log(1-y^{\wedge})]$$
$$L_{\{CE\}}(\hat{y}, y) = -[y \log \hat{y} + (1-y) \log(1-\hat{y})]$$
$$LCE(y^{\wedge}, y) = -[y \log y^{\wedge} + (1-y) \log(1-y^{\wedge})]$$

This is the standard binary cross-entropy loss.

## 8. Cross-Entropy Loss with Sigmoid Plugged In

Since logistic regression predicts:

$$y^{\wedge} = \sigma(w \cdot x + b) \quad \hat{y} = \sigma(w \cdot x + b)$$

we can rewrite the loss as:

$$LCE(y^{\wedge}, y) = -[y \log \sigma(w \cdot x + b) + (1-y) \log(1 - \sigma(w \cdot x + b))]$$

This is the final loss function used for binary logistic regression.

## 9. Intuition Behind Cross-Entropy Loss

Cross-entropy loss behaves the way we want:

**If the model assigns high probability to the correct label:**

- loss is small

**If the model assigns low probability to the correct label:**

- loss is large

### Example: Correct Positive Prediction

Suppose:

- true label  $y=1$
- predicted probability  $y^{\wedge}=0.70$

Then:

$$LCE = -\log(0.70) \approx 0.36 \quad L_{\{CE\}} = -\log(0.70) \approx 0.36 \quad LCE = -\log(0.70) \approx 0.36$$

That is a relatively low loss.

### Example: Wrong Prediction

Suppose:

- true label  $y=0$
- but the model still predicts  $y^{\wedge}=0.70$

Then the relevant probability is  $1 - \hat{y} = 0.301 - \hat{y} = 0.301 - 0.30$ , and:

$$\text{LCE} = -\log(0.30) \approx 1.2$$

This is a much larger loss.

So the loss clearly penalizes being confidently wrong.

## 10. Why Cross-Entropy Is a Good Choice

Cross-entropy is convenient because:

- if the correct class gets probability 1, loss is 0
- if the correct class gets probability 0, loss goes to infinity
- maximizing probability of correct answer automatically reduces probability of incorrect answer, because the two must sum to 1

Thus cross-entropy directly encourages the model to make the correct class as probable as possible.

## 11. Gradient Descent: Goal

Once we have a loss function, we want to find the parameters that minimize it.

The goal is:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m \text{LCE}(f(x^{(i)}; \theta), y^{(i)})$$
$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(f(x^{(i)}; \theta), y^{(i)})$$

where:

- $\theta = \{w, b\}$
- $f(x; \theta)$  means the model's prediction using parameters  $\theta$

So gradient descent searches for the parameter values that minimize average loss over the training examples.

## 12. Gradient Descent Intuition

Gradient descent works by repeatedly:

1. computing the slope of the loss
2. moving parameters in the opposite direction

The text uses the analogy of:

- standing on a hill or canyon
- finding the direction of steepest upward slope
- then walking in the opposite direction to go downhill fastest

So gradient descent is a general minimization procedure.

### 13. Convexity of Logistic Regression Loss

For logistic regression, the cross-entropy loss surface is: **convex**

This is very important.

A convex function has:

- at most one minimum
- no local minima traps

**So for logistic regression:**

- gradient descent **is guaranteed to find the global minimum**, regardless of where it starts

This is a major advantage compared with deep neural networks, whose loss surfaces are generally non-convex.

### 14. One-Dimensional Gradient Descent Intuition

The section first explains gradient descent using a single parameter  $w$ .

If the slope at the current point is:

- negative, move right
- positive, move left

The update rule is:

$$w_{t+1} = w_t - \eta \frac{d}{dw} L(f(x; w), y)$$
$$w_{t+1} = w_t - \eta \frac{d}{dw} L(f(x; w), y)$$

where:

- $\eta$  is the learning rate

This is the 1D version of gradient descent.

### 15. Learning Rate

The parameter  $\eta$  is called the:

**learning rate**

It controls how large each step is.

**If  $\eta$  is too large:**

- overshoots the minimum
- may diverge or oscillate

**If  $\eta$  is too small:**

- learning is very slow

So choosing the learning rate well is important.

## 16. Gradient in Multiple Dimensions

For logistic regression, we usually have many weights and one bias, so the parameter vector has many dimensions.

The gradient is a vector of partial derivatives:

$$\nabla L(f(x;\theta), y) = \begin{bmatrix} \frac{\partial L}{\partial w_1} & \frac{\partial L}{\partial w_2} & \dots & \frac{\partial L}{\partial w_n} & \frac{\partial L}{\partial b} \end{bmatrix}$$

$$\nabla L(f(x;\theta), y) = \left[ \frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_n}, \frac{\partial L}{\partial b} \right]$$

Each component tells us:

- how much changing that parameter would affect the loss

So the gradient points in the direction of greatest increase in loss.

To reduce loss, we move in the opposite direction.

## 17. Full Parameter Update Rule

For all parameters together, the update is:

$$\theta^{t+1} = \theta^t - \eta \nabla L(f(x;\theta), y)$$

$$\theta^{t+1} = \theta^t - \eta \nabla L(f(x;\theta), y)$$

This is the general gradient descent update rule.

## 18. Gradient for Logistic Regression

For binary logistic regression with cross-entropy loss, the derivative with respect to a weight  $w_j$  is:

$$\frac{\partial L_{CE}(y^{\wedge}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

$$\frac{\partial L_{CE}(y^{\wedge}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

Since  $y^{\wedge} = \sigma(w \cdot x + b)$ , this is:

$$\frac{\partial L_{CE}(y^{\wedge}, y)}{\partial w_j} = (y^{\wedge} - y) x_j$$

$$\frac{\partial L_{CE}(y^{\wedge}, y)}{\partial w_j} = (y^{\wedge} - y) x_j$$

Equivalent form:

$$-(y - \hat{y})x_j = -(\hat{y} - y)x_j = (\hat{y} - y)x_j$$

This is one of the most important equations in logistic regression.

## Interpretation

The gradient for each weight is:

**prediction error  $\times$  corresponding input feature**

That is extremely intuitive.

- If prediction is too high compared to truth, adjust downward
- If prediction is too low, adjust upward
- The size of the change depends on the feature value  $x_j$

## 19. Gradient for the Bias

The derivative with respect to the bias is:

$$\frac{\partial \text{LCE}(\hat{y}, y)}{\partial b} = \sigma(w \cdot x + b) - y = \hat{y} - y \quad \frac{\partial \text{LCE}(\hat{y}, y)}{\partial b} = \sigma(w \cdot x + b) - y = \hat{y} - y$$

So the bias gradient is just the prediction error without any feature multiplier.

## 20. Stochastic Gradient Descent (SGD)

The section then introduces: **stochastic gradient descent**

SGD is called:

- **online**  
because it processes examples one at a time

and:

- **stochastic**  
because it typically uses examples in random order

### SGD Procedure

For each training example  $(x^{(i)}, y^{(i)})$ :

1. optionally compute current prediction and loss
2. compute gradient
3. update parameters using the gradient

Repeat until training is done.

So SGD updates the model after each example instead of waiting for the whole dataset.

## 21. SGD Algorithm Summary

The algorithm is:

1. initialize parameters  $\theta$
2. repeat until convergence or stopping condition
3. visit training examples in random order
4. compute gradient on current example
5. update parameters opposite the gradient

This is the standard online training procedure for logistic regression.

## 22. Hyperparameters vs Parameters

The section briefly distinguishes:

### Parameters

Learned from the training data:

- weights  $w$
- bias  $b$

### Hyperparameters

Chosen by the designer:

- learning rate  $\eta$
- number of epochs
- batch size
- stopping criteria

So the learning algorithm learns parameters, but hyperparameters must be selected externally.

## 23. Worked Example of One SGD Update

The section walks through one concrete update with:

### Features

- $x_1 = 3$ : count of positive lexicon words
- $x_2 = 2$ : count of negative lexicon words

### True label

- $y = 1$

## Initial parameters

- $w_1 = 0, w_2 = 0, b = 0$
- $w_1 = 0, w_2 = 0, b = 0$
- $b = 0, w_1 = 0, w_2 = 0$

## Learning rate

- $\eta = 0.1$

## Step 1: Compute Prediction

Since all weights and bias are zero:

$$w \cdot x + b = 0 \cdot x + 0 = 0$$

and:

$$\sigma(0) = 0.5$$

So the model initially predicts:

- probability of positive = 0.5

## Step 2: Compute Gradient

Using the derivative formulas:

$$\nabla_{w,b} L = [(\sigma(0) - 1)x_1, (\sigma(0) - 1)x_2, \sigma(0) - 1] = [-0.5 \cdot 3, -0.5 \cdot 2, -0.5] = [-1.5, -1.0, -0.5]$$
$$\begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5 \cdot 3 \\ -0.5 \cdot 2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$
$$\nabla_{w,b} L = (\sigma(0) - 1)x_1, (\sigma(0) - 1)x_2, \sigma(0) - 1 = -0.5 \cdot 3, -0.5 \cdot 2, -0.5 = -1.5, -1.0, -0.5$$

## Step 3: Update Parameters

Move opposite the gradient:

$$\theta^1 = [w_1, w_2, b] - 0.1[-1.5, -1.0, -0.5] = [0.15, 0.10, 0.05]$$
$$\begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - 0.1 \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.10 \\ 0.05 \end{bmatrix}$$
$$\theta^1 = w_1, w_2, b = 0.15, 0.10, 0.05$$

So after one update:

- $w_1 = 0.15, w_2 = 0.10, b = 0.05$
- $w_1 = 0.15, w_2 = 0.10, b = 0.05$
- $b = 0.05, w_1 = 0.15, w_2 = 0.10$

## Intuition

Because this was a positive example:



- the model increased the weights and bias in a direction that makes positive predictions more likely for similar examples

The section notes that later negative examples with many negative words would push  $w_2w_2$  negative.

## **24. Batch Training vs SGD vs Mini-Batch Training**

The section distinguishes three levels of gradient computation.

### **A. Stochastic Gradient Descent**

Use:

- one example at a time or instance (row) from the dataset at random

Pros:

- fast updates
- online learning
- good for large datasets

Cons:

- noisy, choppy movement

### **B. Batch Training**

Use:

- the entire training dataset per update

Pros:

- very accurate gradient estimate

Cons:

- expensive
- slow on large data

### **C. Mini-Batch Training**

Use:

- a small batch of examples, smaller than full dataset but larger than 1

This is the standard compromise in modern machine learning.

Pros:

- smoother than SGD
- faster than full batch
- easy to vectorize and parallelize

If:

- batch size = dataset size  $\rightarrow$  batch gradient descent
- batch size = 1  $\rightarrow$  stochastic gradient descent

## 25. Why Mini-Batches Are So Useful

Mini-batches are computationally attractive because:

- examples in the batch can be processed in parallel
- hardware such as GPUs is optimized for matrix operations
- they make training much more efficient

So mini-batch training combines:

- statistical efficiency
- hardware efficiency

This is why it is widely used in modern machine learning.

## 26. Mini-Batch Cross-Entropy Loss

For a mini-batch of size  $m$ , assuming examples are independent:

$$\log p(\text{training labels}) = \sum_{i=1}^m \log p(y(i)|x(i)) \quad \log p(\text{training labels}) = \sum_{i=1}^m \log p(y(i)|x(i))$$

Since each term is the negative of cross-entropy loss, the cost over the batch is the average loss:

$$\text{Cost}(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m \text{LCE}(\hat{y}(i), y(i)) \quad \text{Cost}(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(\hat{y}(i), y(i))$$

Expanded:

$$\begin{aligned} \text{Cost}(\hat{y}, y) &= -\frac{1}{m} \sum_{i=1}^m [y(i) \log \sigma(w \cdot x(i) + b) + (1 - y(i)) \log (1 - \sigma(w \cdot x(i) + b))] \\ \text{Cost}(\hat{y}, y) &= -\frac{1}{m} \sum_{i=1}^m [y(i) \log \sigma(w \cdot x(i) + b) + (1 - y(i)) \log (1 - \sigma(w \cdot x(i) + b))] \end{aligned}$$

This is the mini-batch cross-entropy objective.

## 27. Mini-Batch Gradient

The gradient for weight  $w_j$  becomes the average of the example-wise gradients:

$$\frac{\partial \text{Cost}(\hat{y}, y)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m [\sigma(w \cdot x(i) + b) - y(i)] x_j(i)$$

$$\frac{\partial \text{Cost}(\hat{y}, y)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m [\sigma(w \cdot x(i) + b) - y(i)] x_j(i)$$

So mini-batch training simply averages the per-example gradient contributions.

## 28. Matrix Form of Mini-Batch Gradient

Using the vectorized notation from the earlier section:

- $X$ : input matrix
- $y$ : vector of true labels
- $\hat{y} = \sigma(Xw + b)$ : vector of predicted probabilities

Then the gradient with respect to weights is:

$$\frac{\partial \text{Cost}(\hat{y}, y)}{\partial w} = \frac{1}{m} (\hat{y} - y)^T X$$

$$\frac{\partial \text{Cost}(\hat{y}, y)}{\partial w} = \frac{1}{m} (\hat{y} - y)^T X$$

The text's notation indicates the same vectorized idea: compute the difference between predictions and true labels, then combine with the input matrix.

This matrix form is what makes mini-batch training efficient in practice.

## High-Yield Concepts to Know

### Loss Function

Measures how far model output is from the true label.

### Cost Function

Often the average loss over a dataset or mini-batch.

### Cross-Entropy Loss

Standard loss for logistic regression and neural networks.

### Conditional Maximum Likelihood Estimation

Choose parameters that maximize the probability of the correct labels given the inputs.

### Negative Log Likelihood

Another name for cross-entropy loss in this setting.

### Gradient

Vector of partial derivatives showing steepest increase of the loss.

## Gradient Descent

Move parameters opposite the gradient to reduce loss.

## Learning Rate

Hyperparameter  $\eta$  controlling step size.

## Convex Function

Function with one global minimum and no local minima.

## Stochastic Gradient Descent (SGD)

Updates after one randomly selected example at a time.

## Batch Training

Compute gradient using all training examples.

## Mini-Batch Training

Compute gradient using a small batch of examples.

## Hyperparameter

Designer-chosen setting affecting the learning algorithm, such as learning rate.

## Most Important Formulas

### Probability of true label for one example

$$p(y | x) = y^y (1-y)^{1-y} \quad p(y \mid x) = \hat{y}^y (1-\hat{y})^{1-y} \quad p(y | x) = y^y (1-y)^{1-y}$$

### Log likelihood

$$\log p(y|x) = y \log y + (1-y) \log(1-y) \quad \log p(y \mid x) = y \log \hat{y} + (1-y) \log(1-\hat{y})$$

### Cross-entropy loss

$$LCE(y^{\wedge}, y) = -[y \log y + (1-y) \log(1-y)] \quad L_{CE}(\hat{y}, y) = -[y \log \hat{y} + (1-y) \log(1-\hat{y})]$$

### Logistic-regression cross-entropy

$$LCE(y^{\wedge}, y) = -[y \log \sigma(w \cdot x + b) + (1-y) \log(1 - \sigma(w \cdot x + b))] \quad L_{CE}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1-y) \log(1 - \sigma(w \cdot x + b))]$$

## General optimization goal

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{m} \sum_{i=1}^m \text{LCE}(f(x(i); \theta), y(i))$$
$$\frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(f(x^{(i)}; \theta), y^{(i)})$$

## Gradient descent update

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y)$$
$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y)$$

## Weight gradient for logistic regression

$$\frac{\partial \text{LCE}}{\partial w_j} = (y^{(i)} - y) x_j$$
$$\frac{\partial L_{\text{CE}}}{\partial w_j} = (\hat{y} - y) x_j$$

## Bias gradient

$$\frac{\partial \text{LCE}}{\partial b} = y^{(i)} - y$$
$$\frac{\partial L_{\text{CE}}}{\partial b} = \hat{y} - y$$

## Mini-batch cost

$$\text{Cost}(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m \text{LCE}(\hat{y}^{(i)}, y(i))$$
$$\text{Cost}(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(\hat{y}^{(i)}, y^{(i)})$$

## Mini-batch weight gradient

$$\frac{\partial \text{Cost}}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - y(i)) x_j(i)$$
$$\frac{\partial \text{Cost}}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j(i)$$

## Textbook readings chapter 6 for me to skim nicely!!

<https://web.stanford.edu/~jurafsky/slp3/6.pdf>

Extra slides at the end of module 12:

## Computing the gradient

Logistic regression can be thought of as a network with just one weight layer and a sigmoid output. In that case the gradient is:

$$\begin{aligned}\frac{\partial LCE(w, b)}{\partial w_j} &= (\hat{y} - y) x_j \\ &= (\sigma(w \cdot x + b) - y) x_j\end{aligned}$$

But these derivatives only give correct updates for the last weight layer!

For deeper networks, computing the gradients requires looking back through all the earlier layers in the network, even though the loss is only computed with respect to the output of the network.

Solution: **error backpropagation algorithm**

## Computation Graphs

Although backpropagation was invented for neural nets, it is related to general procedure called **backward differentiation**, which depends on the notion of **computation graphs**.

A computation graph represents the process of computing a mathematical expression. The computation is broken down into separate operations. Each operation is a node in a graph.

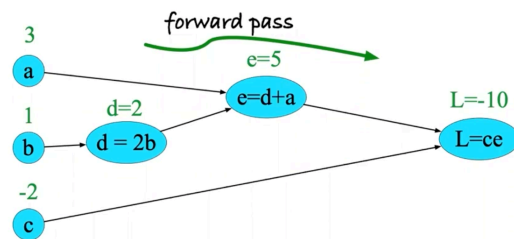
$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$

## Forward pass



$$L(a, b, c) = c(a + 2b)$$

inputs  $a = 3, b = 1, c = -2,$

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$

## Backward differentiation

The importance of the computation graph comes from the backward pass, which is used to compute the derivatives that we'll need for the weight update.

How do we compute the derivative of our output function  $L$  with respect to the input variables  $a, b,$  and  $c$ ?

Backwards differentiation uses the **chain rule** from calculus.

$$\frac{\partial L}{\partial a}, \frac{\partial L}{\partial b}, \text{ and } \frac{\partial L}{\partial c}$$

## Chain rule

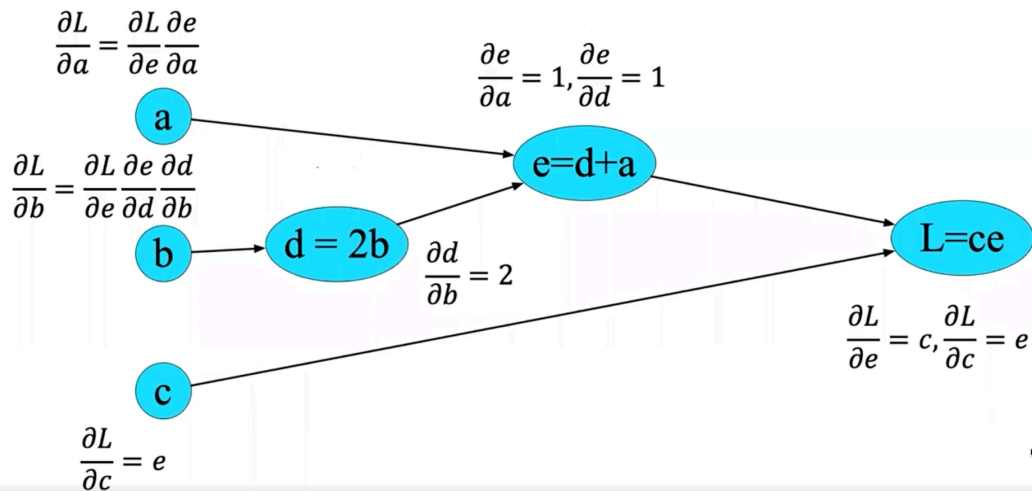
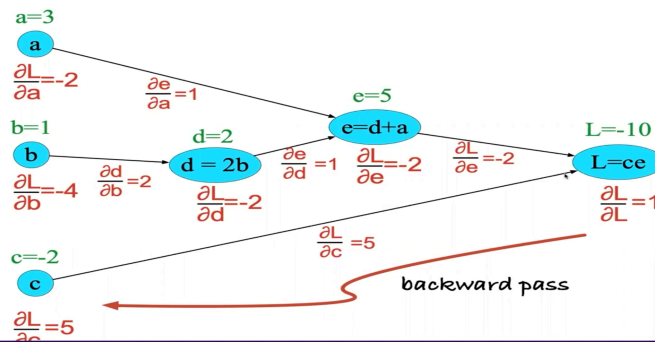
For a composite function  $f(x) = u(v(x))$ , the derivative of  $f(x)$  is:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$$

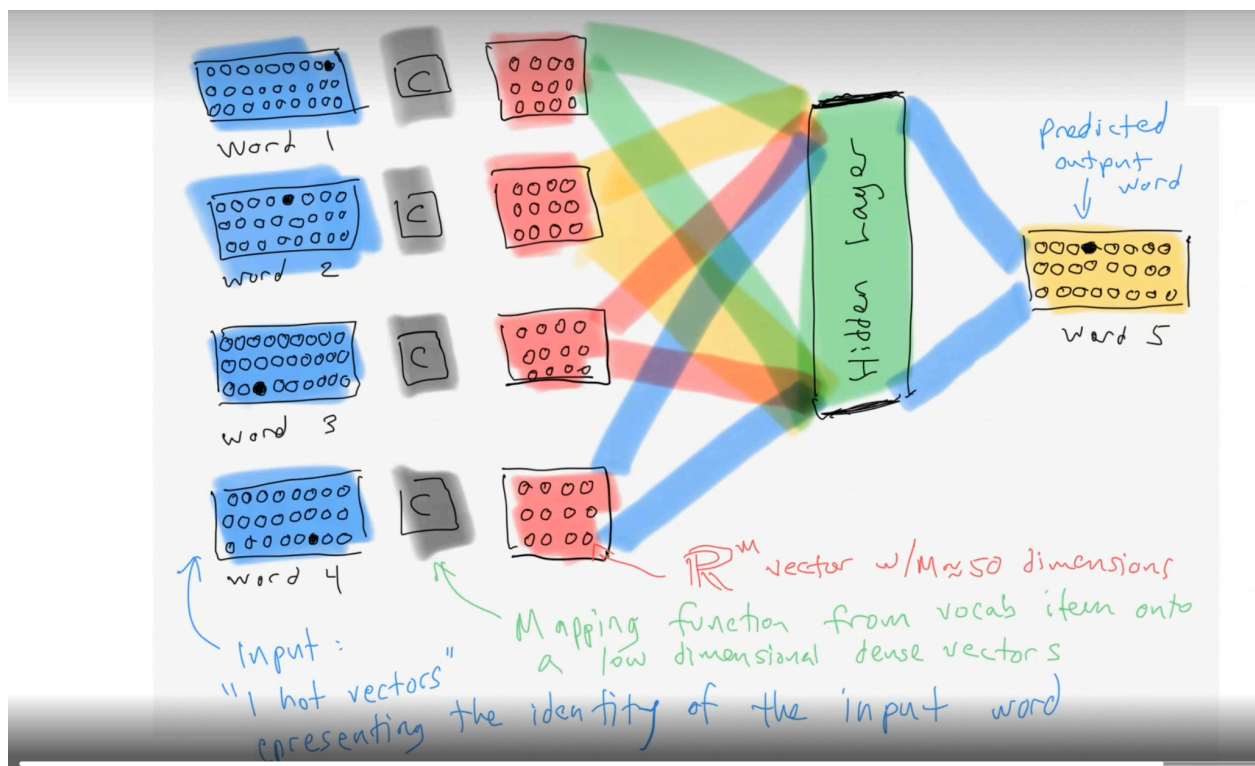
Similarly for,  $f(x) = u(v(w(x)))$ , the derivative of  $f(x)$  is:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx}$$

## Backward pass



## Module 13 notes:





# Distributional Hypothesis

If we consider *optometrist* and *eye-doctor* we find that, as our corpus of utterances grows, these two occur in almost the same environments. In contrast, there are many sentence environments in which *optometrist* occurs but *lawyer* does not...

It is a question of the relative frequency of such environments, and of what we will obtain if we ask an informant to substitute any word he wishes for *optometrist* (not asking what words have the same meaning).

These and similar tests all measure the probability of particular environments occurring with particular elements... If A and B have almost identical environments we say that they are synonyms.

–Zellig Harris (1954)

Textbook readings 5.1 for Jursasky Textbook:

Chapter 5.1 – Lexical Semantics

Detailed Study Guide Summary

## Main Idea of the Section

This section introduces **lexical semantics**, the study of **word meaning**. It begins with the problem that representing meaning by just repeating the word itself — for example representing “dog” as **DOG** — is not actually a useful semantic representation. A good meaning representation should capture:

- similarity between words
- oppositeness between words
- different perspectives on the same event

- emotional or evaluative meaning
- structured relations among words
- and the ability to support inference in language understanding tasks

The section introduces key concepts such as:

- **lemmas**
- **wordforms**
- **word senses**
- **polysemy**
- **synonymy**
- **word similarity**
- **word relatedness**
- **semantic fields**
- **connotation**
- and the early idea that word meaning can be represented as a **vector in space**

This section lays the conceptual foundation for later chapters on **embeddings** and **vector semantics**.

## 1. Why Simple Symbolic Representations of Meaning Are Not Enough

The section begins by criticizing a very simple and traditional view of meaning:

- representing the meaning of “dog” as **DOG**
- representing the meaning of “cat” as **CAT**

This kind of representation is unsatisfactory because it does not explain anything about the relationships between meanings.

A good model of word meaning should be able to tell us things like:

- cat is similar to dog
- cold is opposite to hot
- happy has a positive connotation
- sad has a negative connotation
- buy, sell, and pay refer to different perspectives on the same event

So meaning must be richer than just a symbolic label.

## 2. What We Want from a Model of Word Meaning

The section gives a set of **desiderata** for a word meaning representation.

A good model of word meaning should support:

### A. Similarity

It should know that some words have similar meanings:

- cat and dog

## **B. Opposition / Antonymy**

It should know that some words are opposites:

- cold and hot

## **C. Connotation / Affect**

It should know that some words have:

- positive connotations, like happy
- negative connotations, like sad

## **D. Different Perspectives on the Same Event**

It should know that:

- buy
- sell
- pay

describe different viewpoints on the same underlying commercial event.

## **E. Inference**

It should support meaning-related reasoning in tasks such as:

- question answering
- dialogue
- semantic interpretation

So lexical semantics is not just about dictionary definitions; it is about building representations that support broader language understanding.

## **3. Lexical Semantics**

The field that studies word meaning is called:

### **lexical semantics**

This is the branch of linguistics and NLP concerned with:

- what words mean
- how word meanings are related
- how these meanings are structured

This section provides a first overview of that field.

## 4. Lemmas and Wordforms

The section begins the formal discussion of word meaning with an example from a dictionary entry for the word **mouse**.

### Lemma

The dictionary entry form is called the:

### lemma

also called the:

### citation form

For example:

- **mouse** is the lemma for both mouse and mice
- **sing** is the lemma for sing, sang, sung
- in Spanish, **dormir** is the lemma for duermes

So the lemma is the canonical form used in dictionaries.

### Wordform

Actual inflected surface forms are called:

### wordforms

Examples:

- sung
- carpets
- sing
- duermes

So:

- **lemma** = canonical dictionary form
- **wordform** = actual morphological form appearing in text

This distinction is very important in lexical semantics and NLP.

## 5. Word Senses

A single lemma can have multiple meanings.

Example:

- **mouse** can mean:

1. the animal
2. the computer input device

Each of these meanings is called a:

### **word sense**

So:

- lemma = the word entry itself
- word senses = the distinct meanings of that lemma

## **6. Polysemy**

The fact that one lemma can have multiple senses is called:

### **polysemy**

A polysemous word has more than one sense.

Example:

- mouse is polysemous because it refers both to an animal and to a device

### **Why Polysemy Matters**

Polysemy creates interpretation difficulty.

Example:

- if someone searches for “mouse info,” are they asking about:
  - a pet?
  - or a computer device?

This motivates the problem of:

### **word sense disambiguation**

which is the task of determining which sense of a word is intended in a particular context.

## **7. Synonymy**

Another key concept is:

### **synonymy**

Two word senses are synonyms if:

- they have identical or nearly identical meanings

Examples given:

- couch / sofa
- vomit / throw up
- filbert / hazelnut
- car / automobile

### **More Formal Definition**

Two words are synonymous if they can be substituted for each other in a sentence **without changing the truth conditions** of the sentence.

### **Truth conditions**

Truth conditions are:

- the situations in which a sentence would be true

So if replacing one word with another does not change when the sentence is true, the two words are synonymous in that context.

## **8. Synonymy Is Usually Approximate, Not Perfect**

The section emphasizes that true perfect synonymy is extremely rare.

Even pairs like:

- car / automobile
- water / H<sub>2</sub>O

are not completely identical in meaning.

Why?

Because words differ in:

- style
- genre
- register
- connotation
- usage context

Example:

- H<sub>2</sub>O sounds scientific
- water sounds normal and everyday

So even when two words are truth-conditionally equivalent, they may differ in other aspects of meaning.

## **9. Principle of Contrast**

This leads to an important semantic principle:

### **principle of contrast**

It says:

a difference in linguistic form is always associated with some difference in meaning

So if two forms are different, there is generally some meaning-related distinction between them.

This is why truly identical synonyms are probably very rare.

In practice, then, the word **synonym** usually means:

### **rough or approximate synonymy**

not absolute identity.

## **10. Word Similarity**

Because exact synonymy is rare, it is often more useful to talk about:

### **word similarity**

For example:

- cat is not a synonym of dog
- but cat and dog are clearly similar

Similarity is broader than synonymy.

### **Why Similarity Is Useful**

Knowing word similarity helps in larger semantic tasks like:

- question answering
- paraphrasing
- summarization
- phrase similarity
- sentence similarity

So similarity is one of the most useful semantic relations in practical NLP.

## **11. Human Judgments of Similarity**

One way to define or measure word similarity is to ask humans to score pairs of words.

The section mentions the **SimLex-999** dataset, where word pairs are rated on a 0–10 similarity scale.

Examples:

- vanish / disappear = 9.8
- belief / impression = 5.95
- muscle / bone = 3.65
- modest / flexible = 0.98
- hole / agreement = 0.3

These examples show the range from:

- near-synonyms  
to
- almost unrelated words

This kind of human similarity judgment becomes important later when evaluating vector semantic models.

## 12. Word Relatedness

The section next distinguishes similarity from:

### **word relatedness**

also traditionally called:

### **word association**

Two words may be related even if they are not similar.

### **Example: coffee and cup**

- coffee is a beverage or plant
- cup is a physical container

They are not similar in meaning, but they are clearly related because they often occur together in the same event or scenario.

So relatedness is broader than similarity.

### **Another Example**

- scalpel and surgeon

These are not similar words, but they are related because a surgeon typically uses a scalpel.

### **Key Distinction**

### **Similarity**



Words share semantic properties:

- cat / dog

## **Relatedness**

Words are associated through events, domains, or co-occurrence:

- coffee / cup
- surgeon / scalpel

This distinction is very important in lexical semantics and in evaluating embeddings.

## **13. Semantic Fields**

A very important type of relatedness is belonging to the same:

### **semantic field**

A semantic field is:

- a set of words covering a particular semantic domain
- with structured relations among them

Examples:

- hospital field: surgeon, scalpel, nurse, anesthetic, hospital
- restaurant field: waiter, menu, plate, food, chef
- house field: door, roof, kitchen, family, bed

Words in the same semantic field are often related because they belong to the same conceptual domain.

## **Why Semantic Fields Matter**

Semantic fields are useful for:

- discovering topic structure
- organizing vocabulary by domain
- understanding relatedness within discourse

## **14. Topic Models and Semantic Fields**

The section connects semantic fields to:

### **topic models**

such as:

- **Latent Dirichlet Allocation (LDA)**

Topic models use unsupervised learning on large document collections to induce groups of associated words.

So topic models can help discover:

- themes
- semantic domains
- structured groups of related words

This links lexical semantics to unsupervised learning.

## 15. Other Lexical Relations (Mentioned Briefly)

The section notes that later material will introduce more lexical relations, including:

- **hypernymy** / IS-A relations
- **antonymy** (opposites)
- **meronymy** (part-whole relations)

These are part of the broader structure of lexical semantics, even though they are only previewed here.

## 16. Connotation

The section then introduces:

### **connotation**

Connotation refers to the affective or evaluative aspects of meaning connected to:

- emotions
- sentiment
- opinions
- attitudes
- evaluation

This is different from basic denotational meaning.

### **Examples of Connotation**

Some words have:

- positive connotations: wonderful
- negative connotations: dreary

Even near-similar words can differ strongly in connotation.

Examples:

- fake / knockoff / forgery  
vs
- copy / replica / reproduction

And:

- innocent (positive)
- naive (negative)

So connotation is a very important dimension of lexical meaning.

## 17. Sentiment as Part of Word Meaning

The section highlights:

### **sentiment**

as a special kind of connotational meaning.

Examples:

- positive evaluation words: great, love
- negative evaluation words: terrible, hate

Word-level sentiment is important in tasks such as:

- sentiment analysis
- stance detection
- political language analysis
- consumer review analysis

So lexical semantics is directly relevant to modern NLP applications.

## 18. Early Work on Affective Meaning

The section describes an influential early model of affective meaning from Osgood et al. (1957).

They proposed that words vary along three dimensions:

### **1. Valence**

How pleasant or unpleasant the word is

Examples:

- high valence: happy, satisfied
- low valence: unhappy, annoyed

### **2. Arousal**

How intense or emotionally activating the word is

Examples:

- high arousal: excited
- low arousal: calm

### 3. Dominance

How much control or power the stimulus conveys

Examples:

- high dominance: controlling
- low dominance: awed, influenced

## 19. Words as Points in a 3-Dimensional Space

In Osgood's model, each word is represented by three numbers:

- one for valence
- one for arousal
- one for dominance

Example values:

- courageous = [8.0, 5.5, 7.4]
- music = [7.7, 5.6, 6.5]
- heartbreak = [2.5, 5.7, 3.6]
- cub = [6.7, 4.0, 4.2]

This means each word can be viewed as:

**a point in a three-dimensional space**

This is a major conceptual breakthrough.

## 20. Importance of the Vector Idea

The section ends by stressing that Osgood's work was revolutionary because it represented word meaning as:

**a vector**

rather than as just a symbolic label.

For example, part of the meaning of **heartbreak** can be represented as:

[2.5, 5.7, 3.6]

This is the beginning of:

### **vector semantics**

which will become the central idea of embeddings later in the chapter.

## **21. Big Conceptual Transition to Embeddings**

This section is essentially the conceptual bridge from:

- symbolic views of word meaning

to:

- vector-based views of word meaning

The final idea is:

### **Word meaning can be represented as a point in space**

This insight is the foundation of:

- embeddings
- distributional semantics
- modern NLP representations

So this section is historically and conceptually very important.

## **High-Yield Terms and Definitions**

### **Lexical Semantics**

The study of word meaning and relations between words.

### **Lemma / Citation Form**

The canonical dictionary form of a word.

### **Wordform**

A specific inflected or surface form of a word.

### **Word Sense**

One distinct meaning of a lemma.

### **Polysemy**

The property of a lemma having multiple senses.

## **Word Sense Disambiguation**

Determining which sense of a word is intended in context.

## **Synonym**

Words or senses with identical or nearly identical meaning.

## **Truth Conditions**

The conditions under which a sentence is true.

## **Principle of Contrast**

Different linguistic forms imply some difference in meaning.

## **Word Similarity**

Degree to which two words share semantic properties.

## **Word Relatedness / Association**

Broader connection between words beyond similarity.

## **Semantic Field**

A domain-based set of related words with structured relations.

## **Connotation**

Affective, emotional, evaluative, or attitudinal aspects of meaning.

## **Sentiment**

Positive or negative evaluative meaning.

## **Valence**

Pleasantness of a word.

## **Arousal**

Intensity of feeling associated with a word.

## **Dominance**

Degree of control or power associated with a word.

## **Vector Semantics**

Representing word meaning as a point/vector in a multidimensional space.

## Most Important Distinctions

### Lemma vs Wordform

- lemma = dictionary form
- wordform = actual inflected form in text

### Lemma vs Word Sense

- lemma = word entry
- sense = one meaning of that entry

### Similarity vs Relatedness

- similarity = shared semantic properties
- relatedness = association, domain, event connection

### Denotation vs Connotation

- denotation = core referential meaning
- connotation = emotional/evaluative meaning

## Chapter 5.2 – Vector Semantics: The Intuition

### Detailed Study Guide Summary

#### Main Idea of the Section

This section introduces **vector semantics**, which is the standard modern way to represent **word meaning in NLP**. The central idea is:

A word's meaning can be represented as a **point in a multidimensional space**, where that point is derived from the word's **distributional behavior** in language.

In simpler terms:

- words that occur in similar contexts tend to have similar meanings
- so if two words appear near similar neighboring words, their vectors should be close in space

This section explains:

- where vector semantics comes from historically
- the intuition behind the distributional view of meaning
- why embeddings are useful

- how simple count-based vectors work
- why those vectors are often sparse and high-dimensional
- why later methods such as **word2vec** learn dense vectors
- and why **cosine similarity** becomes an important tool

This section is the conceptual bridge from traditional lexical semantics to modern **embeddings**.

## 1. What Vector Semantics Is

### Definition

**Vector semantics** is the standard approach in NLP for representing word meaning using vectors.

Instead of representing a word by:

- just its spelling
- or just an ID in a vocabulary

vector semantics represents a word as:

**a point in a multidimensional semantic space**

This means each word corresponds to a vector of numbers, and those numbers encode something about the word's meaning.

## 2. Why Vector Semantics Matters

Vector semantics helps model many of the aspects of word meaning introduced in the previous section, including:

- similarity
- semantic relatedness
- neighborhood structure in meaning space
- and potentially connotation or other semantic properties

The core reason it is useful is:

**words with similar meanings can be represented as nearby points in space**

This gives a concrete computational way to reason about meaning.

## 3. Historical Roots of Vector Semantics

The section says vector semantics comes from the convergence of **two major ideas** from the 1950s.

### Idea 1: Osgood's Vector Representation of Meaning



From Osgood (1957):

- words can be represented as points in a space
- originally using dimensions such as:
  - valence
  - arousal
  - dominance

This introduced the basic notion that:

- meaning can be encoded numerically
- a word can be represented as a vector

## Idea 2: Distributional Linguistics

From Joos, Harris, and Firth:

- a word's meaning can be defined by its **distribution in language use**

This means:

- look at the neighboring words and grammatical environments in which a word appears
- words that occur in similar distributions tend to have similar meanings

This is the famous **distributional hypothesis**.

## 4. Distributional Hypothesis

The key intuition is:

Two words that occur in very similar contexts tend to have similar meanings.

This is one of the most important ideas in NLP.

Instead of asking directly:

- “What does this word mean?”

we ask:

- “What kinds of words tend to appear around it?”
- “What contexts does it occur in?”

Then we use those contextual patterns to infer meaning.

## 5. Ongchoi Example: Learning Meaning from Context

The section illustrates the distributional idea with the unfamiliar word:

**ongchoi**

Suppose you do not know what ongchoi means, but you see it in contexts like:

- ongchoi is delicious sautéed with garlic
- ongchoi is superb over rice
- ongchoi leaves with salty sauces

Then suppose you have also seen words like:

- spinach
- chard
- collard greens

in very similar contexts:

- sautéed with garlic
- over rice
- leaves
- salty
- leafy greens

From this, you could infer that:

- ongchoi is probably a leafy green similar to spinach, chard, or collard greens

This is exactly the distributional intuition.

### **Main Point of the Example**

You can infer a word's meaning from:

- the company it keeps
- the words that occur near it
- the contexts it participates in

This is the basis of count-based and embedding-based semantics.

## **6. Computational Version of the Distributional Idea**

The section says we can implement this intuition computationally by:

### **counting words in the context of a target word**

So for a word like ongchoi, we count how often nearby words such as:

- garlic
- rice
- leaves
- salty

appear in its context.

Then those counts become part of the vector representing *ongchoi*.

So the vector for a word is built from:

- contextual evidence
- neighboring words
- co-occurrence statistics

## 7. Word Meaning as a Point in Semantic Space

The section gives the main formal intuition:

**A word is represented as a point in a multidimensional semantic space derived from distributions of word neighbors.**

This means:

- each dimension corresponds in some way to contextual information
- the word's coordinates reflect how it behaves in language
- semantic similarity becomes spatial proximity

So:

- similar words → nearby vectors
- dissimilar words → distant vectors

This is the basic geometry of embeddings.

## 8. Embeddings

Vectors used to represent words are called:

**embeddings**

This is the standard modern term.

### Why the Name “Embedding”?

Historically, “embedding” comes from mathematics, where it means:

- mapping one space or structure into another

In NLP, the term now means:

- a vector representation of a word (or sentence, or document) in some continuous space

So:

- embeddings = learned vector representations

## 9. Example of Embeddings in Practice

The section refers to a visualization of **word2vec embeddings** where neighbors of the word **sweet** include words like:

- honey
- candy
- juice
- chocolate

These nearby words are semantically related and similar in taste-related or positive descriptive contexts.

This visualization supports the key idea:

**semantically similar words become neighbors in vector space**

## 10. Why This Is Powerful

This geometric organization of words in high-dimensional space gives NLP systems enormous power.

The section gives an important example:

**In a classic classifier, the exact same words must often appear in both training and test data.**

But if words are represented with embeddings:

- a model can generalize across similar words
- even if the exact word was not seen during training

For example:

- if a classifier learned that “wonderful” is positive
- and “fantastic” has a similar embedding
- it may still classify “fantastic” positively even if that exact word was absent from training

This is one of the biggest advantages of vector semantics.

## 11. Benefit for Sentiment Classification

The section specifically connects embeddings to sentiment classifiers from the previous chapter.

Without embeddings:

- classifiers often depend on exact lexical overlap between training and test sets

With embeddings:

- classifiers can generalize based on semantic similarity

This means:

- meaning is more flexible than exact word matching
- models become more robust to vocabulary variation

This is a key practical motivation for embeddings.

## 12. Learning Embeddings Automatically

Another major point:

**vector semantic models can be learned automatically from text without supervision**

This is extremely important.

It means:

- no human has to manually assign coordinates to words
- instead, the system learns word meaning from raw text data

So vector semantics is powerful not just because it represents meaning well, but also because it can be induced automatically from large corpora.

## 13. First Pedagogical Model: Count-Based Context Vectors

The section says the chapter will begin with a **simple pedagogical model** in which:

- the meaning of a word is defined by a vector of counts of nearby words

This is meant as a conceptual introduction.

So the first type of vector semantics is:

**count-based vector semantics**

where a word vector records how often context words appear near the target word.

### Why Start with This Simple Model?

Because it helps explain:

- what a vector is
- how a vector can encode meaning
- how context leads to representation
- why spatial similarity reflects semantic similarity

This model is useful pedagogically, even though more advanced versions are used in practice.

## 14. Sparse High-Dimensional Vectors

The section says that simple count-based vectors are often:

**very long**

and

**sparse**

### Why Long?

Because the vocabulary can be huge, and each possible context word may correspond to a dimension.

So vectors may have thousands or more dimensions.

### Why Sparse?

Because most words do **not** appear in the context of most other words.

So most entries in the vector are zero.

Thus:

- high-dimensional
- mostly zeros
- sparse representation

This is a very important property of simple distributional vectors.

## 15. Dense Vectors and word2vec

Later in the chapter, the book introduces:

**word2vec**

and related models that produce:

**short, dense vectors**

### Dense vs Sparse

**Sparse vector:**

- many dimensions

- mostly zero values

### **Dense vector:**

- fewer dimensions
- most entries are nonzero
- more compact and often more useful

word2vec and similar methods learn dense embeddings that often capture semantic properties better than raw count vectors.

## **16. Why Dense Vectors Are Useful**

Dense vectors are useful because they:

- compress semantic information
- make similarity computations easier
- generalize better
- often encode meaningful semantic patterns
- are more practical for machine learning models

So the chapter is setting up a progression from:

- simple count-based vectors  
to
- more sophisticated dense embeddings

## **17. Cosine Similarity**

The section says it will also introduce:

### **cosine**

This refers to:

### **cosine similarity**

which is the standard way to compare embeddings.

Cosine similarity is used to measure:

- similarity between two words
- similarity between sentences
- similarity between documents

This becomes one of the most important tools in practical embedding-based NLP.

## **18. Why Cosine Matters**

Once words are represented as vectors, we need a way to compare them.

Cosine similarity becomes the standard because:

- it measures similarity of direction rather than raw magnitude
- words with similar semantic patterns tend to point in similar directions in vector space

So cosine is the standard operational notion of semantic similarity in vector semantics.

## **19. Conceptual Flow of the Chapter**

This section previews the chapter's structure:

### **First:**

introduce a simple count-based model of word meaning

### **Then:**

show that such vectors are sparse and high-dimensional

### **Later:**

introduce word2vec, which learns short dense vectors

### **Also:**

introduce cosine similarity for comparing vectors

This progression is important for understanding how modern embeddings evolve out of the distributional intuition.

## **20. Big Conceptual Shift**

This section marks the shift from older symbolic or dictionary-like views of lexical semantics to:

### **distributional, geometric, vector-based semantics**

The core idea is now:

Meaning is represented not by a symbolic definition, but by a location in a space induced from contextual usage.

## **Textbook 5.3-5.4 Jurasky**

### **1. Simple Count-Based Embeddings**

**The first embedding model introduced in the chapter is based on the:**



**co-occurrence matrix**

**A co-occurrence matrix records how often words appear together.**

**The specific version introduced here is the:**

**word-context matrix**

**This is one of the most basic count-based representations of word meaning.**

## **2. What a Word-Context Matrix Is**

**A word-context matrix is organized as follows:**

- **each row corresponds to a target word**
- **each column corresponds to a context word**
- **each cell stores how often the context word appears near the target word in a training corpus**

**So if the matrix entry in:**

- **row = cherry**
- **column = pie**

**is large, that means the word pie often appears near cherry.**

**Dimensionality**

**If the vocabulary size is  $|V|$ , then the full matrix has shape:**

$$|V| \times |V|$$

**because:**

- **every word can be a target word**
- **every word can also be a context word**

**So this is a very large matrix in realistic corpora.**

## **3. What “Nearby” Means: Context Window**

**The section defines context in a simple way using a:**

**context window**

**Example:**

- **4 words to the left**
- **and 4 words to the right**

**This is written as a:**

**$\pm 4$  word window**

**So a context word counts as co-occurring with a target word if it appears within that window.**

### **Important Point**

**The meaning representation depends on:**

- **how we define context**
- **how wide the context window is**

**In this simple model, nearby means:**

- **physically close in the text**

## **4. Example Words Used**

**The section illustrates the method using four target words:**

- **cherry**
- **strawberry**
- **digital**
- **information**

**and some example context windows taken from text.**

**These examples are used to show how co-occurrence counts are built.**

## **5. Mini Example of Context Counting**

**The book simplifies the example by only considering three context words:**

- **a**
- **computer**
- **pie**

**Then it counts how often each of these appears in the window of each target word.**

**This yields the following example vectors:**

- **cherry = [1, 0, 1]**
- **strawberry = [0, 0, 2]**
- **digital = [0, 1, 0]**
- **information = [1, 1, 0]**

**Each row is a word vector.**

## 6. Each Row Is a Vector

A key idea:

each row of the word-context matrix is a vector representing a word

So, for example:

- cherry is represented by the vector [1,0,1]
- information is represented by [1,1,0]

This is the actual embedding in this simple count-based model.

Thus:

- target word meaning = row vector of context counts

## 7. What a Vector Is

The section briefly reviews linear algebra terminology.

A:

vector

is simply a list or array of numbers.

So in this setting, a vector is a numerical representation of the contextual behavior of a word.

## 8. Vector Space and Dimensionality

A:

vector space

is a collection of vectors, characterized by its:

dimension

The dimension of the space is the number of features or coordinates in each vector.

In the simplified example:

- the space is 3-dimensional  
because there are 3 context-word dimensions:
- a
- computer
- pie

**In real NLP applications:**

- **the dimension is usually much larger**
- **often equal to  $|V| \times |V|$ , the vocabulary size**

## **9. Meaning of Dimensions**

**Each position in the vector corresponds to one context dimension.**

**Example:**

- **the third dimension corresponds to how often pie occurs nearby**
- **the second dimension corresponds to how often computer occurs nearby**

**So vector entries are interpretable:**

- **each number tells us about one specific contextual association**

**This is one of the nice properties of count-based embeddings.**

## **10. Real Word Vectors Are Built from Entire Corpora**

**The section emphasizes that in practice:**

- **we do not compute vectors from a single window**
- **we count context co-occurrences across a whole corpus**

**So a real word vector is based on:**

- **aggregate corpus statistics**
- **not just one sentence or occurrence**

**That gives a more stable and meaningful representation.**

## **11. Larger Corpus Example**

**The section then shows a more realistic subset of a word-context matrix built from Wikipedia for the same words:**

- **cherry**
- **strawberry**
- **digital**
- **information**

**with context dimensions such as:**

- **aardvark**
- **computer**
- **data**
- **result**

- pie
- sugar

This illustrates what real count vectors look like.

### Example Pattern

The counts show:

- cherry and strawberry have higher counts on dimensions like:
  - pie
  - sugar
- digital and information have higher counts on dimensions like:
  - computer
  - data

This means:

- cherry and strawberry are similar to each other
- digital and information are similar to each other

This matches intuition, which shows that the count-based representation is capturing meaning.

## 12. Word Meaning as Location in Space

The section emphasizes again that a word vector can be thought of as:

a point in high-dimensional space

So:

- cherry is one point
- digital is another point
- information is another point

The closer two points are, the more similar the words may be.

This geometric view is one of the most important ideas in vector semantics.

## 13. Vocabulary Size and High Dimensionality

In realistic settings, the dimensionality of count-based word vectors is often:

- the vocabulary size  $|V|$

This may be:

- 10,000
- 50,000

- or more

Usually, only the most frequent words are kept, because beyond the top 50,000 or so words, the added context dimensions often do not help much.

So real word-context matrices are:

- large
- high-dimensional
- expensive but manageable with sparse methods

## 14. Sparse Vectors

Because most words do not occur near most other words, these vectors are mostly zeros.

Therefore count-based embeddings are:

sparse vector representations

This means:

- many dimensions
- but only a small number of nonzero values

Sparse vectors are common in traditional NLP and require specialized storage and algorithms for efficiency.

## 15. Weighting the Counts

The section notes that we do not always use raw counts directly.

Various:

weighting functions

can be applied to the matrix entries.

The most important one mentioned is:

tf-idf

which is introduced later in the book.

So count-based embeddings can be improved by reweighting raw frequency counts to make them more informative.

## 16. Need for a Similarity Metric

Once each word is represented as a vector, we need a way to compare two vectors.

That means we need a:

similarity metric

This should tell us:

- how similar two target words are based on their vectors

The standard choice is:

cosine similarity

## 17. Dot Product

Cosine similarity is based on the:

dot product

also called the:

inner product

For vectors  $\mathbf{v}$  and  $\mathbf{w}$ , the dot product is:

$$\mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i \quad \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i$$

This means:

- multiply corresponding dimensions
- sum the products

### Why Dot Product Captures Similarity

The dot product is large when:

- both vectors have large values in the same dimensions

So it naturally reflects shared contextual behavior.

If two vectors are:

- high on the same context words
- low on the same context words

their dot product tends to be large.

If they are active in different dimensions, the dot product tends to be small.

## 18. Problem with Raw Dot Product

The raw dot product has a major problem:

it favors long vectors

A vector's:

length

is:

$$|v| = \sqrt{\sum_{i=1}^N v_i^2} \quad |v| = \sqrt{\sum_{i=1}^N v_i^2}$$

Words that are more frequent in the corpus tend to:

- co-occur with more words
- have larger count values
- therefore have longer vectors

So frequent words may get large dot products simply because they are frequent, not because they are semantically similar.

This makes raw dot product a poor standalone similarity metric.

## 19. Cosine Similarity

To fix this, we normalize by vector length.

The result is:

cosine similarity

$$\text{cosine}(v, w) = \frac{v \cdot w}{|v| |w|} \quad \text{cosine}(v, w) = \frac{v \cdot w}{|v| |w|}$$

Expanded:

$$\text{cosine}(v, w) = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}} \quad \text{cosine}(v, w) = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

This is the normalized dot product.

## 20. Geometric Meaning of Cosine

Cosine similarity is equal to the cosine of the angle between the two vectors.

From linear algebra:

$$a \cdot b = |a| |b| \cos \theta \quad a \cdot b = |a| |b| \cos \theta$$

So dividing by the lengths leaves:



$\cos\theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$

Thus cosine similarity measures:

- whether two vectors point in the same direction

not just whether they are long.

## 21. Why Cosine Is Better Than Dot Product

Cosine removes the effect of overall vector magnitude.

That means:

- frequent words no longer automatically seem more similar
- similarity depends on pattern of distribution, not sheer size

This is why cosine is the standard similarity metric in NLP vector semantics.

## 22. Range of Cosine Similarity

Cosine values range from:

- 1: vectors point in the same direction
- 0: vectors are orthogonal (completely unrelated in direction)
- -1: vectors point in opposite directions

However, in raw count-based word vectors:

- entries are nonnegative
- so cosine is typically between 0 and 1

## 23. Unit Vectors

Sometimes vectors are pre-normalized to length 1, producing:

unit vectors

If a vector has unit length, then:

- its dot product with another unit vector equals the cosine

So normalizing in advance can simplify similarity calculations.

## 24. Example Cosine Computation

The section gives a concrete example comparing:

- cherry

- digital
- information

using only the dimensions:

- pie
- data
- computer

It computes:

$\cos(\text{cherry}, \text{information}) = 0.018$   
 $\cos(\text{cherry}, \text{information}) = 0.018$   
 $\cos(\text{digital}, \text{information}) = 0.996$   
 $\cos(\text{digital}, \text{information}) = 0.996$

This means:

- information is much more similar to digital than to cherry

This result matches our intuition and shows the method is working sensibly.

## 25. Geometric Interpretation of the Example

The section explains that:

- when the angle between two vectors is smaller
- the cosine is larger

So:

- digital and information point in nearly the same direction
- cherry and information point in very different directions

Thus cosine similarity corresponds nicely to semantic closeness in the vector space.

## 26. Applications of Cosine Similarity

Cosine similarity can be used for many semantic tasks, including:

- estimating word similarity
- finding paraphrases
- tracking changes in word meaning
- discovering meanings in different corpora
- comparing words, sentences, or documents

So cosine is not just a theoretical tool — it is extremely practical.

## 27. Finding Most Similar Words

A standard use of cosine is:

given a target word *www*, compute cosine similarity between *www* and every other word in the vocabulary, then sort and inspect the top neighbors

This is a classic way to:

- explore embeddings
- evaluate semantic quality
- retrieve semantically similar words

So nearest-neighbor search in cosine space becomes a basic semantic operation.

## 28. Big Conceptual Takeaways

Count-based embeddings operationalize the distributional hypothesis

They turn “words occurring in similar contexts have similar meanings” into a concrete numeric representation.

Each word is a vector of contextual counts

Meaning is represented as a point in high-dimensional space.

Similarity becomes geometry

Words are similar when their vectors point in similar directions.

Cosine is the standard similarity metric

Because it corrects for vector length and focuses on contextual pattern.

Sparse, high-dimensional vectors are the first step

They are simple and interpretable, even though later methods improve on them.

## High-Yield Terms and Definitions

### Co-occurrence Matrix

A matrix recording how often words occur together.

### Word-Context Matrix

A co-occurrence matrix where rows are target words and columns are context words.

### Context Window

The span of nearby words used to define co-occurrence.

## Count-Based Embedding

A word vector built from counts of neighboring context words.

## Vector

A list/array of numbers representing an object.

## Vector Space

A collection of vectors sharing the same dimensions.

## Dimension

One coordinate/feature of the vector.

## Sparse Vector

A vector with mostly zero entries.

## Dot Product / Inner Product

$$\mathbf{v} \cdot \mathbf{w} = \sum_i v_i w_i \quad \mathbf{v} \cdot \mathbf{w} = \sum_i v_i w_i$$

## Vector Length

$$|\mathbf{v}| = \sqrt{\sum_i v_i^2} \quad |\mathbf{v}| = \sqrt{\sum_i v_i^2}$$

## Cosine Similarity

Normalized dot product measuring angle similarity between vectors.

## Unit Vector

A vector normalized to have length 1.

## Most Important Formulas

### Dot product

$$\mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i \quad \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i$$

### Vector length

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^N v_i^2} \quad |\mathbf{v}| = \sqrt{\sum_{i=1}^N v_i^2}$$

### Dot product and angle

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos \theta \quad \mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos \theta$$

## Cosine similarity

$$\text{cosine}(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}| |\mathbf{w}|} \quad \text{cosine}(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}| |\mathbf{w}|}$$

Expanded:

$$\text{cosine}(\mathbf{v}, \mathbf{w}) = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}} \quad \text{cosine}(\mathbf{v}, \mathbf{w}) = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

Word2vec algorithm example:

## Negative sampling

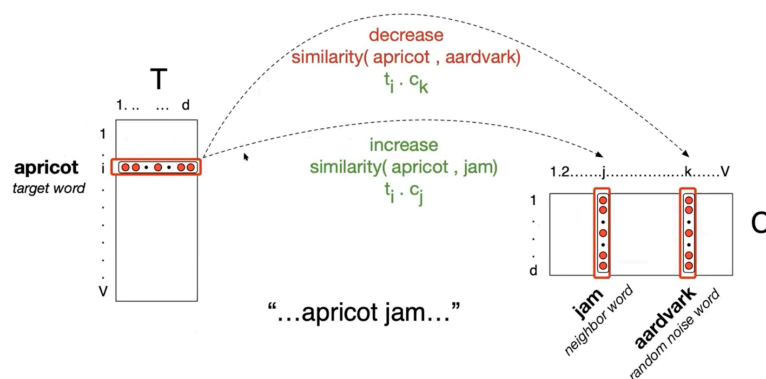
lemon, a [tablespoon of apricot preserves or] jam  
 c1 c2 w c3 c4

We want predictions  
of these words to be high

And these words to be low

[cement metaphysical dear coaxial apricot attendant whence forever puddle]  
 n1 n2 n3 n4 n5 n6 n7 n8

## Neural Network



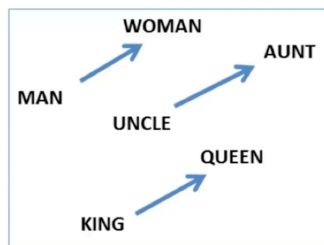
# Properties of Embeddings

Nearest Neighbors are surprisingly good

|                |                    |                        |               |             |              |
|----------------|--------------------|------------------------|---------------|-------------|--------------|
| <b>target:</b> | Redmond            | Havel                  | ninjutsu      | graffiti    | capitulate   |
|                | Redmond Wash.      | Vaclav Havel           | ninja         | spray paint | capitulation |
|                | Redmond Washington | president Vaclav Havel | martial arts  | grafitti    | capitulated  |
|                | Microsoft          | Velvet Revolution      | swordsmanship | taggers     | capitulating |

## Embeddings capture relational meanings

$\text{vector}(\text{'king'}) - \text{vector}(\text{'man'}) + \text{vector}(\text{'queen'}) \cong \text{vector}(\text{'woman'})$



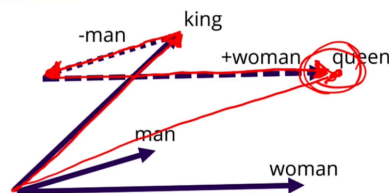
Vector arithmetic to solve word analogies!

## Vector Arithmetic

$\mathbf{a}:\mathbf{a}^*$  as  $\mathbf{b}:\mathbf{b}^*$ .  $\mathbf{b}^*$  is a hidden vector.

$\mathbf{b}^*$  should be similar to the vector  $\mathbf{b} - \mathbf{a} + \mathbf{a}^*$

$\text{vector}(\text{'king'}) - \text{vector}(\text{'man'}) + \text{vector}(\text{'woman'}) \approx \text{vector}(\text{'queen'})$



$$\arg \max_{\mathbf{b}^* \in V} (\cos(\mathbf{b}^*, \mathbf{b} - \mathbf{a} + \mathbf{a}^*))$$

## Textbook 5.5 Jurasky

### Chapter 5.5 – Word2vec

This section introduces **word2vec**, one of the most important methods for learning **dense word embeddings**. Earlier sections described **count-based embeddings**, where each word was represented by a very long, sparse vector of contextual counts. Word2vec moves to a more powerful representation:

- **short vectors**
- **dense vectors**
- **real-valued entries**
- dimensions usually between **50 and 1000**
- dimensions that no longer have direct human-interpretable meanings

The specific version introduced here is:

#### **skip-gram with negative sampling (SGNS)**

The central idea is:

Instead of directly counting context words, train a **binary classifier** to decide whether a candidate context word is likely to appear near a target word.

Then:

- the classifier's learned weights become the embeddings

This is a major breakthrough because it shows that useful semantic vectors can be learned from **raw running text** using **self-supervision**, without any human-labeled semantic data.

The section covers:

- why dense embeddings are better than sparse vectors
- how skip-gram formulates the learning problem
- how positive and negative training examples are constructed
- how the classifier uses dot product and sigmoid
- the loss function and SGD updates
- why word2vec learns two embeddings per word
- and extensions such as **fastText** and **GloVe**

### 1. From Sparse Vectors to Dense Embeddings

The previous sections represented a word as:

- a long sparse vector
- with one dimension per vocabulary word
- mostly zeros
- based on context counts

Word2vec instead introduces:

### **short dense vectors**

These have:

- dimensionality ddd, often between **50 and 1000**
- entries that are real-valued
- values that may be positive or negative
- dimensions that do **not** have clear individual interpretations

So word2vec embeddings are more compact and more abstract than count-based vectors.

## **2. Why Dense Vectors Are Better Than Sparse Vectors**

The section states that:

### **dense vectors work better in every NLP task than sparse vectors**

The text says we do not fully understand all the reasons, but offers several important intuitions.

#### **A. Fewer Parameters to Learn**

If a word is represented by:

- a 300-dimensional dense vector

instead of:

- a 50,000-dimensional sparse vector

then any classifier built on those embeddings needs to learn many fewer parameters.

This likely helps:

- generalization
- computational efficiency
- avoiding overfitting

#### **B. Better Capture of Similarity and Synonymy**

In sparse count vectors:

- car and automobile are different dimensions
- their similarity may not be captured directly



So a model might fail to recognize that:

- a context involving **car**
- is semantically similar to one involving **automobile**

Dense embeddings can encode these similarities more naturally, because similar words can end up with similar vectors.

### 3. Word2vec and SGNS

The method introduced is:

#### **skip-gram with negative sampling (SGNS)**

This is often loosely called:

#### **word2vec**

because it is one of the two main algorithms in the original word2vec software package.

So in practice:

- “word2vec” may refer to the package
- or specifically to skip-gram embeddings

### 4. Static Embeddings

Word2vec produces:

#### **static embeddings**

This means:

- each word has exactly one fixed vector in the vocabulary
- the vector does not change depending on context

For example:

- the word bank would have one fixed embedding, regardless of whether it means riverbank or financial bank

This is in contrast to later models like BERT, which produce:

#### **dynamic contextual embeddings**

where the vector for a word changes depending on the surrounding context.

### 5. Core Intuition of Word2vec

Instead of asking:

- “How often does context word ccc occur near target word www?”

word2vec asks:

**“Is word ccc likely to show up near word www?”**

So it turns distributional learning into a **binary classification task**.

Example:

- target word = apricot
- candidate context word = jam

Question:

- is jam a real nearby context word for apricot?

This is a much more indirect way of learning meaning, but it turns out to work extremely well.

## **6. Why This Is Revolutionary: Self-Supervision**

The training signal comes directly from running text.

If a context word really occurs near the target word in the corpus, then that pair acts as a positive training example.

So the model learns from:

### **self-supervision**

This means:

- the supervision signal is generated automatically from the data itself
- no hand-labeled semantic annotations are needed

This is one of the most important ideas in modern NLP.

## **7. Historical Connection to Neural Language Models**

The section notes that this self-supervised idea had already appeared in early neural language models.

Those models learned to:

- predict the next word from previous words

and in doing so:

- learned useful embeddings as part of the prediction process

Word2vec simplifies that idea.

## 8. Why Word2vec Is Simpler Than a Neural Language Model

Word2vec is simpler in two important ways:

### A. Simpler Task

Instead of full next-word prediction over the whole vocabulary, it does:

#### binary classification

asking whether a context word is likely or unlikely near the target

### B. Simpler Architecture

Instead of a multi-layer neural network with hidden layers, it trains:

#### a logistic regression classifier

So word2vec is much easier and faster to train than early neural language models.

## 9. Skip-Gram Intuition in Four Steps

The section summarizes skip-gram as follows:

1. Treat the target word and a real neighboring context word as a **positive example**
2. Sample random other words as **negative examples**
3. Train a logistic regression classifier to distinguish positive from negative pairs
4. Use the learned weights as the word embeddings

This is the heart of SGNS.

## 10. Classifier Setup

Suppose we have a sentence fragment with target word **apricot** and window size  $\pm 2$ :

- nearby context words are  $c_1, c_2, c_3, c_4$

The classifier is trained on tuples:

$(w, c)$

where:

- $w$  = target word
- $c$  = candidate context word

The classifier outputs:

$$P(+|w,c)P(+|\text{mid } w,c)P(+|w,c)$$

the probability that ccc is a real context word for www.

The negative-class probability is:

$$P(-|w,c)=1-P(+|w,c)P(-|\text{mid } w,c)=1-P(+|\text{mid } w,c)P(-|w,c)=1-P(+|w,c)$$

## 11. Similarity via Dot Product

The skip-gram model assumes that:

- a context word is likely to appear near a target word if their embeddings are similar

Similarity is approximated by:

### dot product

$$\text{Similarity}(w,c) \approx c \cdot w \quad \text{Similarity}(w,c) \approx c \cdot w \quad \text{Similarity}(w,c) \approx c \cdot w$$

So:

- if the target and context embeddings have a high dot product
- then they are considered likely neighbors

## 12. Turning Dot Product into a Probability

The dot product is just a real number, ranging from:

- $-\infty$  to  $+\infty$

It is not yet a probability.

So word2vec applies the:

### sigmoid function

$$\sigma(x) = \frac{1}{1 + \exp(-x)} \quad \sigma(x) = \frac{1}{1 + \exp(-x)} \quad \sigma(x) = \frac{1}{1 + \exp(-x)}$$

Then:

$$P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)} \quad P(+|\text{mid } w,c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)} \quad P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

This gives a probability between 0 and 1.

### Negative-Class Probability

Similarly:

$$P(-|w,c) = 1 - P(+|w,c) = \sigma(-c \cdot w) = \frac{1}{1 + \exp(c \cdot w)} \quad P(+|w,c) = \sigma(c \cdot w) = \frac{\exp(c \cdot w)}{1 + \exp(c \cdot w)}$$

So the classifier is exactly a logistic regression model on the dot product between word embeddings.

### 13. Multiple Context Words

A context window contains multiple words.

Skip-gram makes a simplifying assumption:

**all context words are independent**

So the probability of the full context window is:

$$P(+|w, c_{1:L}) = \prod_{i=1}^L \sigma(c_i \cdot w) \quad P(+|w, c_{\{1:L\}}) = \prod_{i=1}^L \sigma(c_i \cdot w)$$

Taking logs gives:

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^L \log \sigma(c_i \cdot w) \quad \log P(+|w, c_{\{1:L\}}) = \sum_{i=1}^L \log \sigma(c_i \cdot w)$$

So the log-probability of the full context is just the sum of the log-probabilities of each context word.

### 14. Two Embeddings Per Word

A very important detail:

**skip-gram stores two embeddings for each word**

For each vocabulary word  $w$ , there is:

- a **target embedding**  $w_i$
- a **context embedding**  $c_i$

These are stored in two matrices:

- $W$ : target embeddings
- $C$ : context embeddings

So every word has two learned representations during training.

### Why Two Embeddings?

Because the model treats:

- being a target word  
and
- being a context word

as different roles.

Thus:

- one vector is used when the word is the center word
- another vector is used when the word is being considered as a context candidate

## 15. Parameters of the Model

The total parameter set is:

$$\theta = \{W, C\} \quad \theta = \{W, C\}$$

This can be viewed as a matrix containing:

- $2|V| \times |V|$  vectors
- each of dimension  $d$

where:

- $|V|$  = vocabulary size
- $d$  = embedding dimension

## 16. How Learning Begins

The learning algorithm starts with:

- a corpus
- a chosen vocabulary size  $N$

Initially:

- every vocabulary word is assigned a **random embedding vector**

Then training iteratively adjusts the vectors so that:

- words occurring near each other move closer
- words that do not occur near each other move farther apart

## 17. Positive Training Examples

For a target word  $w$ , every actual context word in the window produces a:

### positive example

Example with target = apricot and context window  $\pm 2$ :

Positive examples:

- (apricot, tablespoon)
- (apricot, of)
- (apricot, jam)
- (apricot, a)

These are word pairs that really occurred together in the corpus.

## 18. Negative Training Examples

To train a binary classifier, we also need:

### negative examples

For each positive pair  $(w, c_{pos})$ , SGNS samples  $k$  negative words  $c_{neg}$ .

Each negative example looks like:

- (apricot, aardvark)
- (apricot, seven)
- (apricot, my)
- etc.

These negative words are:

- random words from the lexicon
- constrained not to be the target word itself

This is the “negative sampling” part of SGNS.

## 19. Negative Sampling Ratio

The number of negative samples per positive sample is controlled by a hyperparameter:

$k$

If  $k=2$ , then for each positive example:

- 2 negative examples are created

Usually:

- there are more negatives than positives

This helps train the classifier to distinguish true context from random words.

## 20. Noise Distribution for Negative Sampling

Negative words are not sampled uniformly.

Instead they are sampled according to a:

### weighted unigram distribution

$$P_{\alpha}(w) = \frac{\text{count}(w)^{\alpha}}{\sum_{w'} \text{count}(w')^{\alpha}}$$

The usual choice is:

$$\alpha = 0.75$$

### Why Use $\alpha = 0.75$ ?

This slightly increases the probability of rare words compared with their raw unigram probability.

That is:

- rare words are sampled a bit more often than they would be under plain frequency sampling

This improves performance.

### Intuition

If we used raw unigram probabilities:

- extremely frequent words would dominate the negative samples

Using exponent 0.75:

- softens this dominance
- gives somewhat more variety
- especially boosts rarer words

## 21. Learning Objective

The goal of training is:

- maximize similarity between target and true context words
- minimize similarity between target and negative sampled words

For one positive pair  $(w, c_{\text{pos}})$  and  $k$  negative words  $c_{\text{neg1}}, \dots, c_{\text{negk}}$ , the loss is:

$$L(w, c_{\text{pos}}, c_{\text{neg}^*}) = -[\log \sigma(c_{\text{pos}} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{\text{neg}i} \cdot w)]$$



This is the SGNS loss for one training instance.

## Interpretation of the Loss

The loss encourages:

### Positive term:

$$\log \sigma(c_{\text{pos}} \cdot w) \log \sigma(c_{\text{pos}} \cdot w)$$

to be large

→ so  $c_{\text{pos}} \cdot w$  should be large

→ target and true context should be close

### Negative terms:

$$\log \sigma(-c_{\text{negi}} \cdot w) \log \sigma(-c_{\text{negi}} \cdot w)$$

to be large

→ so  $-c_{\text{negi}} \cdot w$  should be small

→ target and negative samples should be far apart

So SGNS learns by pulling real neighbors together and pushing fake neighbors apart.

## 22. Optimization with Stochastic Gradient Descent

The loss is minimized using:

### stochastic gradient descent (SGD)

Training repeatedly:

1. samples a target/context configuration
2. computes loss
3. computes gradients
4. updates the embeddings

Thus word2vec learning is an instance of logistic-regression-style SGD over word pairs.

## 23. Gradients for SGNS

The section gives the derivatives of the loss with respect to:

- the positive context embedding  $c_{\text{pos}}$
- the negative context embeddings  $c_{\text{neg}_i}$
- the target embedding  $w$

They are:

$$\begin{aligned}\frac{\partial L}{\partial \mathbf{c}_{pos}} &= [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1] \mathbf{w} \frac{\partial L}{\partial \mathbf{c}_{pos}} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1] \mathbf{w} \frac{\partial L}{\partial \mathbf{c}_{neg_i}} = [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})] \mathbf{w} \frac{\partial L}{\partial \mathbf{c}_{neg_i}} \\ &= [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})] \mathbf{w} \frac{\partial L}{\partial \mathbf{c}_{neg_i}} \\ \frac{\partial L}{\partial \mathbf{w}} &= [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1] \mathbf{c}_{pos} + \sum_{i=1}^k [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})] \mathbf{c}_{neg_i} \frac{\partial L}{\partial \mathbf{w}} = \\ &= [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1] \mathbf{c}_{pos} + \sum_{i=1}^k [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})] \mathbf{c}_{neg_i} \frac{\partial L}{\partial \mathbf{w}}\end{aligned}$$

These gradients control how embeddings move during learning.

## 24. Intuition of the Update Direction

The update rules do the following:

### For the positive context word

Move  $\mathbf{c}_{pos}$  closer to  $\mathbf{w}$

This increases:

$$\mathbf{c}_{pos} \cdot \mathbf{w}$$

So true neighbors become more similar.

### For negative words

Move  $\mathbf{c}_{neg}$  away from  $\mathbf{w}$

This decreases:

$$\mathbf{c}_{neg} \cdot \mathbf{w}$$

So fake neighbors become less similar.

### For the target word

Move it:

- toward true context words
- away from negative sampled words

So the target embedding is shaped by both attractive and repulsive forces.

## 25. SGD Update Equations

The update rules from time  $t$  to  $t+1$  are:

$$\begin{aligned}\mathbf{c}_{pos}^{t+1} &= \mathbf{c}_{pos}^t - \eta [\sigma(\mathbf{c}_{pos}^t \cdot \mathbf{w}^t) - 1] \mathbf{w}^t \\ \mathbf{c}_{neg_i}^{t+1} &= \mathbf{c}_{neg_i}^t - \eta [\sigma(\mathbf{c}_{neg_i}^t \cdot \mathbf{w}^t)] \mathbf{w}^t \\ \mathbf{w}^{t+1} &= \mathbf{w}^t + \eta [\sigma(\mathbf{c}_{pos}^t \cdot \mathbf{w}^t) - 1] \mathbf{c}_{pos}^t + \eta \sum_{i=1}^k [\sigma(\mathbf{c}_{neg_i}^t \cdot \mathbf{w}^t)] \mathbf{c}_{neg_i}^t\end{aligned}$$

$$\eta[\sigma(c_{\text{neg}_i}^t \cdot w^t)]w^{t+1} = c_{\text{neg}_i} - \eta[\sigma(c_{\text{neg}_i} \cdot w^t)]w^t$$

$$w^{t+1} = w^t - \eta([\sigma(c_{\text{post}} \cdot w^t) - 1]c_{\text{post}} + \sum_{i=1}^k [\sigma(c_{\text{neg}_i} \cdot w^t)]c_{\text{neg}_i})w^{t+1} = w^t - \eta \left( [\sigma(c_{\text{pos}}^t \cdot w^t) - 1]c_{\text{pos}}^t + \sum_{i=1}^k [\sigma(c_{\text{neg}_i}^t \cdot w^t)]c_{\text{neg}_i}^t \right)w^{t+1} = w^t - \eta([\sigma(c_{\text{post}} \cdot w^t) - 1]c_{\text{post}} + \sum_{i=1}^k [\sigma(c_{\text{neg}_i} \cdot w^t)]c_{\text{neg}_i})$$

These are the concrete SGNS learning equations.

## 26. What Word Vector Is Used After Training?

Since each word has:

- target embedding  $w_i$
- context embedding  $c_i$

we must decide what to use as the final representation.

Common options:

### Option 1

Use:

$$w_i + c_i$$

This combines both target and context information.

### Option 2

Discard CCC and use only:

$$w_i$$

Both are common choices.

## 27. Context Window as a Hyperparameter

Just like in count-based models, the:

### context window size

matters for skip-gram performance.

A larger window may capture:

- broader semantic relations

A smaller window may capture:

- more local syntactic relations

So window size is often tuned on a development set.

## 28. Other Static Embeddings: fastText

The section then introduces:

### **fastText**

an extension of word2vec.

fastText addresses a key weakness of ordinary word2vec:

### **unknown words / out-of-vocabulary words**

Word2vec assigns embeddings only to words seen during training, so unseen words are a problem.

### **fastText Solution: Subword Modeling**

fastText represents each word as:

- the word itself
- plus a bag of character n-grams

Example for “where” with  $n=3$ :

- <wh
- whe
- her
- ere
- re>

(with boundary markers)

Then:

- an embedding is learned for each n-gram
- the word representation is the sum of its constituent n-gram embeddings

### **Why fastText Helps**

Because even unseen words can still be represented if their character n-grams were seen.

This is especially useful for:

- unknown words
- morphologically rich languages
- rare inflected forms

So fastText improves robustness and handles word sparsity much better.

## 29. Other Static Embeddings: GloVe

Another major static embedding method is:

### GloVe

short for:

### Global Vectors

GloVe is based on:

- global corpus statistics
- ratios of probabilities from the co-occurrence matrix

So unlike word2vec, which is framed as a predictive classifier, GloVe is more directly tied to matrix statistics.

## 30. Relationship Between Dense and Count-Based Embeddings

The section ends with an important theoretical point:

**dense embeddings like word2vec are mathematically related to count-based embeddings**

In particular:

- word2vec can be viewed as implicitly optimizing a function of a count matrix
- with a particular PPMI-based weighting

This means:

- predictive and count-based models are more closely related than they first appear

This is an important bridge between the two traditions.

## High-Yield Terms and Definitions

### Dense Embedding

A short real-valued vector with mostly nonzero entries.

### Static Embedding

A fixed vector for each word, regardless of context.

### word2vec

A software package / family of methods for learning dense word embeddings.

## **Skip-Gram**

A model that predicts whether context words occur near a target word.

## **SGNS**

Skip-gram with negative sampling.

## **Self-Supervision**

Using structure in raw text itself as the supervision signal.

## **Positive Example**

A real target-context pair observed in the corpus.

## **Negative Example**

A target paired with a randomly sampled noise word.

## **Negative Sampling**

Training by contrasting real context words with randomly sampled fake context words.

## **Target Embedding**

The embedding used when a word is the center/target word.

## **Context Embedding**

The embedding used when a word is considered as a context word.

## **Noise Word**

A random word used to create a negative example.

## **fastText**

A static embedding model that incorporates subword character n-grams.

## **GloVe**

A static embedding model based on global co-occurrence statistics.

## **Most Important Formulas**

## **Positive-class probability**

$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)} \\ P(-|w, c) = \sigma(-c \cdot w) = \frac{1}{1 + \exp(c \cdot w)}$$

### Negative-class probability

$$P(-|w, c) = 1 - P(+|w, c) = \sigma(-c \cdot w) \\ P(-|w, c) = 1 - P(+|w, c) = \sigma(-c \cdot w)$$

### Full context probability

$$P(+|w, c_{1:L}) = \prod_{i=1}^L \sigma(c_i \cdot w) \\ P(+|w, c_{1:L}) = \prod_{i=1}^L \sigma(c_i \cdot w)$$

### Log probability

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^L \log \sigma(c_i \cdot w) \\ \log P(+|w, c_{1:L}) = \sum_{i=1}^L \log \sigma(c_i \cdot w)$$

### Negative-sampling distribution

$$P_{\alpha}(w) = \frac{\text{count}(w)^{\alpha}}{\sum_{w'} \text{count}(w')^{\alpha}} \\ P_{\alpha}(w) = \frac{\text{count}(w)^{\alpha}}{\sum_{w'} \text{count}(w')^{\alpha}}$$

### SGNS loss

$$L(w, c_{\text{pos}}, c_{\text{neg}}) = -[\log \sigma(c_{\text{pos}} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{\text{neg}_i} \cdot w)] \\ L(w, c_{\text{pos}}, c_{\text{neg}}) = -[\log \sigma(c_{\text{pos}} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{\text{neg}_i} \cdot w)]$$

### Gradients

#### Juraska chapter 5.8:

Bias and Embeddings In addition to their ability to learn word meaning from text, embeddings, alas, also reproduce the implicit biases and stereotypes that were latent in the text. As the prior section just showed, embeddings can roughly model relational similarity: ‘queen’ as the closest word to ‘king’ - ‘man’ + ‘woman’ implies the analogy man:woman::king:queen. But these same embedding analogies also exhibit gender stereotypes. For example Bolukbasi et al. (2016) find that the closest occupation to ‘computer programmer’ - ‘man’ + ‘woman’ in word2vec embeddings trained on news text is ‘homemaker’, and that the embeddings similarly suggest the analogy ‘father’ is to ‘doctor’ as ‘mother’ is to ‘nurse’. This could result in what Crawford (2017) and Blodgett et al. (2020) call an allocational harm, when a system allocates resources (jobs or credit) unfairly to different groups. For example algorithms that use embeddings as part of a search for hiring potential programmers or doctors might thus incorrectly downweight documents with women’s names. It turns out that embeddings don’t just reflect the statistics of their input, but also amplify bias; gendered terms become more gendered in embedding space than they bias amplification were in the input text statistics (Zhao et al. 2017, Ethayarajah et al. 2019b, Jia et al. 2020), and biases are more exaggerated than in actual

labor employment statistics (Garg et al., 2018). Embeddings also encode the implicit associations that are a property of human reasoning. The Implicit Association Test (Greenwald et al., 1998) measures people's associations between concepts (like 'flowers' or 'insects') and attributes (like 'pleasantness' and 'unpleasantness') by measuring differences in the latency with which they label words in the various categories.<sup>3</sup> Using such methods, people in the United States have been shown to associate African-American names with unpleasant words (more than European-American names), male names more with mathematics and female names with the arts, and old people's names with unpleasant words (Greenwald et al. 1998, Nosek et al. 2002a, Nosek et al. 2002b). Caliskan et al. (2017) replicated all these findings of implicit associations using GloVe vectors and cosine similarity instead of human latencies. For example African-American names like 'Leroy' and 'Shaniqua' had a higher GloVe cosine with unpleasant words while European-American names ('Brad', 'Greg', 'Courtney') had a higher cosine with pleasant words. These problems with embeddings are an example of a representational harm (Crawford 2017, Blodgett et al. 2020), which is a harm caused by representational harm a system demeaning or even ignoring some social groups. Any embedding-aware algorithm that made use of word sentiment could thus exacerbate bias against African Americans. Recent research focuses on ways to try to remove these kinds of biases, for example by developing a transformation of the embedding space that removes gender stereotypes but preserves definitional gender (Bolukbasi et al. 2016, Zhao et al. 2017) or changing the training procedure (Zhao et al., 2018). However, although debiasing these sorts of debiasing may reduce bias in embeddings, they do not eliminate it (Gonen and Goldberg, 2019), and this remains an open problem. Historical embeddings are also being used to measure biases in the past. Garg et al. (2018) used embeddings from historical texts to measure the association between embeddings for occupations and embeddings for names of various ethnicities or genders (for example the relative cosine similarity of women's names versus men's to occupation words like 'librarian' or 'carpenter') across the 20th century. They found that the cosines correlate with the empirical historical percentages of women or ethnic groups in those occupations. Historical embeddings also replicated old surveys of ethnic stereotypes; the tendency of experimental participants in 1933 to associate adjectives like 'industrious' or 'superstitious' with, e.g., Chinese ethnicity, correlates with the cosine between Chinese last names and those adjectives using embeddings trained on 1930s text. They also were able to document historical gender biases, such as the fact that embeddings for adjectives related to competence ('smart', 'wise', 'thoughtful', 'resourceful') had a higher cosine with male than female words, and showed that this bias has been slowly decreasing since 1960. We return in later chapters to this question about the role of bias in natural language processing.

Skim chapter 7 nicely:

<https://web.stanford.edu/~jurafsky/slp3/7.pdf>

**API Key for hw10:**

sk-proj-I4xjbHmjVS\_4q7ue\_uOX2V9WLRCWrjYaq6CxinT4I7-E90YEOwFcuGLlpaTqoFsftqYjddp78T3BlbkFJkYch3cGrwBIxmvUJsr3tokJoP8R0IkeYmvhqJEuZp4wqx-iHeU5AkmUDFOUic\_\_cTmAOf-yu8A